

KHOA TOÁN KINH TẾ



**TIỂU LUẬN CUỐI KỲ
MÔN GIỚI THIỆU HỌC MÁY**

**PHÂN LOẠI GIAO DỊCH CÓ LỢI NHUẬN BẰNG HỌC MÁY
ỨNG DỤNG LOGISTIC REGRESSION, DECISION TREE,
RANDOM FOREST VÀ GRADIENT BOOSTING**

GVHD: GVC.ThS. Trương Quang Nhật

SVTH: Huỳnh Thị Phương Dung

MSSV: K234131474



LỜI CẢM ƠN

Kính gửi Thầy Trương Quang Nhật,

Em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Thầy, người đã tận tình giảng dạy và hướng dẫn em trong suốt học phần Giới thiệu học máy.

Không thể phủ nhận rằng đây là một môn học đầy thử thách, từ lý thuyết cho đến việc triển khai các mô hình học máy phức tạp. Tuy nhiên, nhờ những bài giảng được trình bày một cách rõ ràng, có hệ thống cùng với sự nhiệt huyết và kiên nhẫn của Thầy, em đã có thể hoàn thành bài tiểu luận này.

Những tri thức và tư duy mà Thầy truyền đạt sẽ là hành trang quý báu, tiếp sức cho em trên chặng đường học tập và nghiên cứu phía trước.

Dẫu đã hết sức cố gắng, song bài tiểu luận khó tránh khỏi những thiếu sót nhất định. Em rất mong nhận được những lời nhận xét, góp ý từ Thầy để có thể rút kinh nghiệm và hoàn thiện hơn trong tương lai. Em xin chân thành cảm ơn Thầy.

Trân trọng./.

MỤC LỤC

DANH MỤC HÌNH ẢNH	iv
PHẦN I. GIỚI THIỆU VỀ DỮ LIỆU	1
1.1. Mô tả tổng quan bộ dữ liệu, kích thước dữ liệu	1
1.2. Danh sách các biến, kiểu dữ liệu của từng biến, ý nghĩa thực tiễn và Khoảng dữ liệu của từng biến	1
1.3. Thống kê mô tả	4
1.3.1. Nhóm biến định lượng	4
1.3.2. Nhóm biến định tính	5
1.4. Khám phá dữ liệu (EDA)	8
1.4.1. Xác định giá trị duy nhất cho biến phân loại	8
1.4.2. Heatmap tương quan	10
1.4.3. Biểu đồ phát hiện ngoại lai	11
1.4.4. Scatter Discount vs Profit (phân loại theo Category)	18
PHẦN II. GIỚI THIỆU BÀI TOÁN	21
2.1. Bài toán đang giải quyết	21
2.2. Mô hình được chọn và lý do lựa chọn	23
2.2.1. Hồi quy Logistic (Logistic Regression)	23
2.2.2. Decision Tree	24
2.2.3. Random Forest	25
2.2.4. Gradient Boosting	26
2.3. Sơ đồ quy trình thực hiện (Pipeline)	27
PHẦN III. XỬ LÝ DỮ LIỆU	28
3.1. Xử lý dữ liệu rỗng (Missing Values)	28
3.2. Xử lý dữ liệu trùng lặp	29
3.3. Kiểm tra tính hợp lý	30
3.4. Feature Engineering	31
3.5. Xử lý giá trị ngoại lai (Outliers)	33
3.6. Tạo biến mục tiêu + binning	35
3.7. Lựa chọn đặc trưng	36

3.8. Mã hóa dữ liệu (Data Encoding)	37
3.8.1. Mã hóa biến danh nghĩa (One-Hot Encoding).....	37
3.8.2. Kích thước và kiểm tra tương quan sau mã hóa	39
3.8.3 Kiểm tra tương quan sau encoding	39
3.9. Tách dữ liệu thành tập train, test.....	43
3.10. Scale dữ liệu.....	44
3.11. Xử lý mất cân bằng dữ liệu (SMOTE)	46
PHẦN IV. MÔ HÌNH HÓA DỮ LIỆU.....	50
4.1. Cơ sở lý thuyết và mô hình	50
4.1.1. Logistic Regression	50
4.1.2. Decision Tree.....	54
4.1.3. Random Forest.....	58
4.1.4. Gradient Boosting	62
4.2. Xây dựng mô hình.....	65
4.3. Đánh giá mô hình bằng kỹ thuật Kiểm chứng chéo (Cross Validation)	67
4.4. Huấn luyện mô hình và diễn giải kết quả huấn luyện	69
PHẦN V. ĐÁNH GIÁ MÔ HÌNH.....	72
5.1. Một số dự báo minh họa trên tập test	72
5.2. Đánh giá bằng các thang đo phù hợp và so sánh lựa chọn mô hình tối ưu	74
5.2.1. Đánh giá bằng các chỉ số.....	75
5.2.2. Ma trận nhầm lẫn	78
5.2.3. Đường cong ROC.....	80
PHẦN VI. KẾT LUẬN CHUNG.....	84
6.1. Những phát hiện chính từ quá trình phân tích dữ liệu và mô hình	84
6.2. Ưu điểm và hạn chế của nghiên cứu	85
6.3. Đề xuất hướng phát triển	86
TÀI LIỆU THAM KHẢO.....	87

DANH MỤC HÌNH ẢNH

Hình 1. 1. Kích thước dữ liệu	1
Hình 1. 2. Thông tin chi tiết về các thuộc tính và kiểu dữ liệu của bộ dữ liệu.....	2
Hình 1. 3. Thống kê mô tả biến định lượng.....	4
Hình 1. 4. Thống kê mô tả biến định tính.....	5
Hình 1. 5. Số lượng và danh sách các giá trị duy nhất của các biến	9
Hình 1. 6. Phân tích tương quan giữa các biến định lượng	10
Hình 1. 7. Ngoại lai trong doanh số (Sales)	13
Hình 1. 8. Ngoại lai trong lợi nhuận (Profit).....	14
Hình 1. 9. Phân bố và biến động Profit theo nhóm sản phẩm	15
Hình 1. 10. Ngoại lai trong biến chiết khấu (Discount)	16
Hình 1. 11. Ngoại lai trong biến số lượng (Quantity)	17
Hình 1. 12. Mối quan hệ giữa tỷ lệ chiết khấu (Discount) và lợi nhuận (Profit)	18
Hình 2. 1. Sơ đồ quy trình thực hiện	27
Hình 3. 1. Tổng hợp giá trị thiếu theo từng biến	29
Hình 3. 2. Số lượng dòng trước và sau khi xóa trùng lặp.....	29
Hình 3. 3. Kiểm tra tính hợp lệ của dữ liệu	31
Hình 3. 4. Thông tin thống kê cơ bản của Sales, Quantity, Discount và Profit.....	31
Hình 3. 5. Kết quả feature engineering biến thời gian giao hàng (Ship_Duration)	33
Hình 3. 6. Xử lý ngoại lai bằng phương pháp Winsorization.....	34
Hình 3. 7. Biến mới sau quá trình Feature Engineering	35
Hình 3. 8. Tổng quan tập đặc trưng và biến mục tiêu	37
Hình 3. 9. Kết quả sau khi mã hóa dữ liệu	38
Hình 3. 10. Kích thước dữ liệu sau mã hóa	39
Hình 3. 11. Kiểm tra tương quan sau mã hóa	40
Hình 3. 12. Mức độ tương quan của các đặc trưng với biến mục tiêu Is_Profit	42
Hình 3. 13. Kích thước tập Train/Test và phân bố biến mục tiêu Is_Profit.....	44
Hình 3. 14. Kết quả chuẩn hóa dữ liệu bằng Min-Max Scaling.....	46
Hình 3. 15. Kết quả cân bằng dữ liệu bằng phương pháp SMOTE.....	48
Hình 3. 16. Biểu đồ so sánh phân bố lớp trước và sau SMOTE	48
Hình 4. 1. Hàm sigmoid: Ánh xạ logit sang xác suất.....	51
Hình 4. 2. Hàm mất mát Cross-Entropy khi nhãn thật bằng 1	51
Hình 4. 3. Gradient Descent tìm điểm tối ưu của hàm mất mát.....	52
Hình 4. 4. Minh họa cây quyết định phân loại nhị phân	54
Hình 4. 5. Độ đo Gini cho bài toán phân loại nhị phân.....	56
Hình 4. 6. Minh họa cách hoạt động của Random Forest	59
Hình 4. 7. Cơ chế đánh giá mô hình Random Forest bằng OOB.....	60
Hình 4. 8. So sánh khả năng chống overfitting: Decision Tree và Random Forest	61
Hình 4. 9. Minh họa nguyên lý hoạt động của Gradient Boosting.....	62
Hình 4. 10. Nguyên lý Bagging và Boosting trong ensemble learning.....	64

Hình 4. 11. Thiết lập tham số cơ bản cho các mô hình học máy.....	65
Hình 4. 12. Kết quả Cross Validation 5-fold trên tập Train sau SMOTE.....	68
Hình 4. 13. So sánh hiệu suất Train và Cross Validation của các mô hình	70
Hình 5. 1. So sánh dự đoán của các mô hình trên một số mẫu dữ liệu	72
Hình 5. 2. Số lượng và tỷ lệ dự đoán đúng của các mô hình.....	73
Hình 5. 3. So sánh các chỉ số đánh giá hiệu suất của các mô hình.....	77
Hình 5. 4. Phân tích ma trận nhầm lẫn cho mô hình	79
Hình 5. 5. Đường cong ROC và giá trị AUC của các mô hình	81

PHẦN I. GIỚI THIỆU VỀ DỮ LIỆU

1.1. Mô tả tổng quan bộ dữ liệu, kích thước dữ liệu

Bộ dữ liệu được sử dụng trong nghiên cứu này có tên Sample Superstore, mô phỏng hoạt động bán hàng của một siêu thị tại Hoa Kỳ, cung cấp cái nhìn toàn diện về các giao dịch bán hàng trong suốt 4 năm (từ năm 2014 đến năm 2017).

Kết quả kiểm tra kích thước dữ liệu cho thấy:

```
print(f"Kích thước dữ liệu: {df.shape[0]} dòng x {df.shape[1]} cột")
```

Kích thước dữ liệu: 9994 dòng x 21 cột

Hình 1. 1. Kích thước dữ liệu

Với 9.994 giao dịch được ghi nhận, bộ dữ liệu này có quy mô đủ lớn để đảm bảo tính thống kê có ý nghĩa khi xây dựng các mô hình học máy. Mỗi giao dịch được mô tả bởi 21 thuộc tính khác nhau, bao gồm thông tin về đơn hàng, sản phẩm, khách hàng, phương thức vận chuyển và các chỉ số tài chính quan trọng như doanh số, chiết khấu và lợi nhuận.

Sự đa dạng về số lượng biến và số lượng quan sát tạo điều kiện thuận lợi cho việc phân tích sâu, từ khám phá dữ liệu đến xây dựng và đánh giá các mô hình dự báo. Bộ dữ liệu này đặc biệt phù hợp cho bài toán phân loại nhị phân, nơi mục tiêu là dự đoán liệu một giao dịch có mang lại lợi nhuận hay không.

1.2. Danh sách các biến, kiểu dữ liệu của từng biến, ý nghĩa thực tiễn và Khoảng dữ liệu của từng biến

Để hiểu rõ cấu trúc và bản chất của dữ liệu, bảng tổng hợp dưới đây trình bày chi tiết các biến, bao gồm: kiểu dữ liệu, số lượng giá trị duy nhất, khoảng giá trị (đối với biến số) và số lượng giá trị thiếu.

```
info_df = pd.DataFrame({
    "Kiểu dữ liệu": df.dtypes,
    "Giá trị thiếu": df.isnull().sum(),
    "Số lượng Unique": df.nunique()
})
# Tính toán Khoảng dữ liệu (Min - Max) cho từng biến
min_vals = []
max_vals = []

for col in df.columns:
    if pd.api.types.is_numeric_dtype(df[col]) or
pd.api.types.is_datetime64_any_dtype(df[col]):
        min_vals.append(df[col].min())
```

```

        max_vals.append(df[col].max())
    else:
        min_vals.append("-")
        max_vals.append("-")

info_df["Giá trị Min"] = min_vals
info_df["Giá trị Max"] = max_vals

styled_info = info_df.style.set_properties(**{
    'background-color': '#f0f7ff',
    'color': '#003366',
    'border-color': 'white'
}).set_table_styles([
    {'selector': 'th', 'props': [('background-color', '#003366'),
    ('color', 'white')]}
])
display(styled_info)

```

	Kiểu dữ liệu	Giá trị thiếu	Số lượng Unique	Giá trị Min	Giá trị Max
Row ID	int64	0	9994	1	9994
Order ID	object	0	5009	-	-
Order Date	object	0	1237	-	-
Ship Date	object	0	1334	-	-
Ship Mode	object	0	4	-	-
Customer ID	object	0	793	-	-
Customer Name	object	0	793	-	-
Segment	object	0	3	-	-
Country	object	0	1	-	-
City	object	0	531	-	-
State	object	0	49	-	-
Postal Code	int64	0	631	1040	99301
Region	object	0	4	-	-
Product ID	object	0	1862	-	-
Category	object	0	3	-	-
Sub-Category	object	0	17	-	-
Product Name	object	0	1850	-	-
Sales	float64	0	5825	0.444000	22638.480000
Quantity	int64	0	14	1	14
Discount	float64	0	12	0.000000	0.800000
Profit	float64	0	7287	-6599.978000	8399.976000

Hình 1. 2. Thông tin chi tiết về các thuộc tính và kiểu dữ liệu của bộ dữ liệu

Nhóm biến định lượng:

- **Sales (Doanh số):** Biến này phản ánh giá trị tiền tệ của mỗi giao dịch. Khoảng giá trị dao động từ 0.444 đến 22.638,48 đô la, thể hiện sự chênh lệch lớn giữa các giao dịch nhỏ lẻ và các đơn hàng có giá trị cao. Số lượng giá trị duy nhất là 5.825, cho thấy tính đa dạng của doanh số. Trong bối cảnh phân tích kinh doanh, đây là biến đầu vào quan trọng nhằm phản ánh quy mô giao dịch.

- **Quantity (Số lượng):** Số lượng sản phẩm trong mỗi đơn hàng dao động từ 1 đến 14, với giá trị trung bình khoảng 3-4 sản phẩm. Biến này có 14 giá trị duy nhất, tương ứng với các mức số lượng phổ biến. Nó cung cấp thông tin về quy mô đơn hàng, có liên quan đến doanh số và chi phí vận chuyển.

- **Discount (Chiết khấu):** Tỷ lệ chiết khấu áp dụng cho giao dịch, dao động từ 0% đến 80%. Sự tồn tại của chiết khấu cao (80%) cho thấy các chương trình khuyến mãi mạnh có thể đã được áp dụng. Biến này có 40 giá trị duy nhất, phản ánh các mức chiết khấu khác nhau và trong nghiên cứu thì chiết khấu được xem là yếu tố có thể ảnh hưởng đến lợi nhuận.

- **Profit (Lợi nhuận):** Lợi nhuận thu được từ mỗi giao dịch, dao động từ mức lỗ sâu (-6.599,98 đô la) đến mức lãi rất cao (8.399,98 đô la). Biến này có 6.611 giá trị duy nhất, thể hiện sự phân tán lớn. Trong bài toán phân loại, Profit sẽ được chuyển đổi thành biến nhị phân Is_Profit để xác định trạng thái có lợi nhuận hay không.

Nhóm biến định tính:

- **Ship Mode:** Gồm 4 phương thức vận chuyển (First Class, Same Day, Second Class, Standard Class), phản ánh sự lựa chọn dịch vụ giao hàng của khách hàng. Phương thức vận chuyển có thể ảnh hưởng đến chi phí và thời gian giao hàng, từ đó tác động đến lợi nhuận.

- **Segment:** Gồm 3 phân khúc khách hàng (Consumer, Corporate, Home Office). Việc phân nhóm này giúp so sánh hành vi mua sắm và hiệu quả kinh doanh giữa các đối tượng khách hàng khác nhau.

- **Category:** Gồm 3 danh mục sản phẩm chính (Furniture, Office Supplies, Technology), là cơ sở để phân tích hiệu suất theo nhóm hàng, từ đó đưa ra các chiến lược kinh doanh phù hợp.

- **Sub-Category:** Gồm 17 danh mục con, cung cấp thông tin chi tiết hơn về loại sản phẩm và hỗ trợ phân tích sâu hơn nếu cần.

- **Country:** Chỉ có 1 giá trị duy nhất (United States), do đó biến này không mang lại thông tin hữu ích cho mô hình và sẽ được loại bỏ trong quá trình lựa chọn đặc trưng.

- **City và State:** Có số lượng giá trị duy nhất lần lượt là 531 và 49, có thể được sử dụng để phân tích địa lý. Tuy nhiên, do số lượng lớn nên việc đưa trực tiếp vào mô hình có thể gây tăng chiều dữ liệu và yêu cầu các phương pháp mã hóa phù hợp.

- **Postal Code:** Biến này có thể mang tính định danh cao, không phù hợp để đưa trực tiếp vào mô hình.

1.3. Thống kê mô tả

1.3.1. Nhóm biến định lượng

```
print("\nThống kê mô tả biến định lượng:")  
display(df.describe().T.style.background_gradient(cmap='GnBu'))
```

Thống kê mô tả biến định lượng:								
	count	mean	std	min	25%	50%	75%	max
Row ID	9994.000000	4997.500000	2885.163629	1.000000	2499.250000	4997.500000	7495.750000	9994.000000
Postal Code	9994.000000	55190.379428	32063.693350	1040.000000	23223.000000	56430.500000	90008.000000	99301.000000
Sales	9994.000000	229.858001	623.245101	0.444000	17.280000	54.490000	209.940000	22638.480000
Quantity	9994.000000	3.789574	2.225110	1.000000	2.000000	3.000000	5.000000	14.000000
Discount	9994.000000	0.156203	0.206452	0.000000	0.000000	0.200000	0.200000	0.800000
Profit	9994.000000	28.656896	234.260108	-6599.978000	1.728750	8.666500	29.364000	8399.976000

Hình 1. 3. Thống kê mô tả biến định lượng

Row ID và Postal Code: Hai biến này mang tính định danh, không có ý nghĩa trong việc dự đoán lợi nhuận. Row ID đơn thuần là số thứ tự, Postal Code là mã bưu điện, cả hai đều không được sử dụng trong mô hình. Tuy nhiên, việc thống kê chúng cho thấy không có giá trị thiếu và dữ liệu được ghi nhận đầy đủ.

Sales (Doanh số):

- Giá trị trung bình (mean = 229.86) lớn hơn đáng kể so với trung vị (median = 54.49), cho thấy phân phối của doanh số bị lệch phải. Điều này nghĩa là tồn tại một số lượng nhỏ các giao dịch có doanh số rất cao, kéo giá trị trung bình lên cao hơn so với giá trị đại diện cho phần lớn dữ liệu.

- Độ lệch chuẩn (std = 623.25) rất lớn, gần gấp 3 lần giá trị trung bình, khẳng định sự biến động mạnh của doanh số giữa các giao dịch. Khoảng biến thiên từ 0.444 đến 22.638,48 đô la càng làm rõ sự chênh lệch này.

- Khoảng tứ phân vị (IQR = $Q3 - Q1 = 209.94 - 17.28 = 192.66$) cho biết 50% giao dịch ở giữa có doanh số dao động trong khoảng 17.28 đến 209.94 đô la. Phân vị 25% ($Q1 = 17.28$) cho biết 25% giao dịch có doanh số dưới 17.28 đô la.

Quantity (Số lượng):

- Giá trị trung bình (3.79) và trung vị (3) khá gần nhau, cho thấy phân phối tương đối đối xứng. Độ lệch chuẩn (2.225) phản ánh mức độ biến động vừa phải.

- Giá trị nhỏ nhất là 1 và lớn nhất là 14, cho thấy đây là các giao dịch bán lẻ. Phân vị 75% ($Q3 = 5$) cho biết 75% giao dịch có số lượng không quá 5 sản phẩm.

Discount (Chiết khấu):

- Trung vị (median = 0.2) bằng với phân vị 75% ($Q3 = 0.2$), đây là một điểm đặc biệt đáng lưu ý, cho thấy có một lượng lớn giao dịch có chiết khấu đúng bằng 20% (có thể

là một mức chiết khấu tiêu chuẩn của siêu thị). Cụ thể, có ít nhất 25% giao dịch có chiết khấu bằng 20% (từ phân vị 50% đến 75%).

- Giá trị trung bình (0.156) nhỏ hơn trung vị (0.2), cho thấy phân phối bị lệch trái một chút do ảnh hưởng của các giao dịch có chiết khấu bằng 0.

- Phân vị 25% ($Q1 = 0$) cho biết 25% giao dịch không được áp dụng chiết khấu. Giá trị lớn nhất là 0.8 (80%), cho thấy có những chương trình khuyến mãi rất sâu.

Profit (Lợi nhuận):

- Tương tự Sales, phân phối của lợi nhuận bị lệch phải mạnh ($\text{mean} = 28.66 > \text{median} = 8.67$), với độ lệch chuẩn rất lớn (234.26). Khoảng biến thiên từ -6.599,98 đến 8.399,98 cho thấy sự hiện diện của cả giao dịch lỗ sâu và lãi cao.

- Phân vị 25% ($Q1 = 1.73$) dương, nghĩa là 75% giao dịch có lợi nhuận dương trên 1.73 đô la. Tuy nhiên, 25% giao dịch còn lại có lợi nhuận dưới 1.73 đô la, bao gồm cả các giao dịch thua lỗ.

- Giá trị nhỏ nhất ($\text{min} = -6599.98$) là một con số đáng chú ý, cho thấy có những giao dịch thua lỗ nghiêm trọng. Ngược lại, giá trị lớn nhất ($\text{max} = 8399.98$) cho thấy tiềm năng sinh lời rất lớn từ một số giao dịch..

1.3.2. Nhóm biến định tính

```
print("\nThông kê mô tả biến định tính:")
display(df.describe(include=['O']).T.style.set_properties(**{'background-color': '#e0f2f1', 'color': '#084594'}))
```

Thông kê mô tả biến định tính:				
	count	unique	top	freq
Order ID	9994	5009	CA-2017-100111	14
Order Date	9994	1237	9/5/2016	38
Ship Date	9994	1334	12/16/2015	35
Ship Mode	9994	4	Standard Class	5968
Customer ID	9994	793	WB-21850	37
Customer Name	9994	793	William Brown	37
Segment	9994	3	Consumer	5191
Country	9994	1	United States	9994
City	9994	531	New York City	915
State	9994	49	California	2001
Region	9994	4	West	3203
Product ID	9994	1862	OFF-PA-10001970	19
Category	9994	3	Office Supplies	6026
Sub-Category	9994	17	Binders	1523
Product Name	9994	1850	Staple envelope	48

Hình 1. 4. Thông kê mô tả biến định tính

Nhóm biến về đơn hàng và thời gian:

- **Order ID (Mã đơn hàng):** Có 5.009 giá trị duy nhất trên tổng số 9.994 quan sát, cho thấy trung bình mỗi mã đơn hàng xuất hiện khoảng 2 lần. Điều này phản ánh thực tế rằng một đơn hàng có thể bao gồm nhiều sản phẩm khác nhau, mỗi sản phẩm được ghi nhận thành một dòng riêng biệt trong dữ liệu. Mã đơn hàng CA-2017-100111 xuất hiện nhiều nhất với 14 lần, cho thấy đây là một đơn hàng lớn với nhiều mặt hàng.

- **Order Date (Ngày đặt hàng):** Có 1.237 ngày đặt hàng khác nhau, phản ánh hoạt động bán hàng diễn ra liên tục trong suốt giai đoạn 2014-2017. Ngày 05/9/2016 có tần suất cao nhất với 38 giao dịch, có thể do đây là ngày diễn ra chương trình khuyến mãi hoặc sự kiện đặc biệt thu hút khách hàng.

- **Ship Date (Ngày giao hàng):** Có 1.334 ngày giao hàng khác nhau, nhiều hơn so với số ngày đặt hàng, cho thấy có sự phân tán trong thời gian vận chuyển. Ngày 16/12/2015 có tần suất cao nhất với 35 giao dịch, có thể do lượng hàng tồn kho được giao đồng loạt vào ngày này.

Nhóm biến về vận chuyển và khách hàng:

- **Ship Mode (Phương thức vận chuyển):** Có 4 phương thức vận chuyển khác nhau. Standard Class chiếm ưu thế tuyệt đối với 5.968 giao dịch (khoảng 59,7%), phản ánh sự ưu tiên về chi phí của phần lớn khách hàng. Các phương thức thời gian giao nhanh hơn như First Class (khoảng 1.482 giao dịch), Second Class (khoảng 1.895 giao dịch) và Same Day (khoảng 649 giao dịch) có tỷ lệ thấp hơn, phù hợp với nhu cầu cấp bách của một bộ phận khách hàng.

- **Customer ID và Customer Name:** Có 793 khách hàng duy nhất, với khách hàng có mã WB-21850 (William Brown) có tần suất mua hàng cao nhất (37 giao dịch). Điều này cho thấy sự trung thành của một số khách hàng, đồng thời gợi ý rằng việc phân tích hành vi theo từng khách hàng có thể mang lại những hiểu biết sâu sắc về giá trị vòng đời khách hàng.

Nhóm biến về phân khúc và địa lý:

- **Segment (Phân khúc khách hàng):** Có 3 phân khúc chính. Consumer (khách hàng cá nhân) chiếm tỷ trọng lớn nhất với 5.191 giao dịch (khoảng 52%), phù hợp với bản chất của một siêu thị. Corporate (khách hàng doanh nghiệp) và Home Office (văn phòng tại nhà) có tỷ lệ thấp hơn nhưng vẫn đóng góp đáng kể, cho thấy siêu thị phục vụ đa dạng đối tượng khách hàng.

- **Country (Quốc gia):** Chỉ có 1 giá trị duy nhất (United States), khẳng định phạm vi hoạt động của siêu thị chỉ giới hạn trong lãnh thổ Hoa Kỳ. Biến này sẽ được loại bỏ trong quá trình lựa chọn đặc trưng vì không mang lại thông tin phân biệt.

- **City (Thành phố):** Có 531 thành phố xuất hiện trong dữ liệu. New York City dẫn đầu với 915 giao dịch, phản ánh vị thế là trung tâm kinh tế lớn nhất nước Mỹ. Sự đa dạng về thành phố cho thấy siêu thị có mạng lưới phân phối rộng khắp.

- **State (Tiểu bang):** Có 49 tiểu bang xuất hiện. California là tiểu bang có số lượng giao dịch lớn nhất (2.001), tiếp theo là New York (khoảng 1.200), Texas, Pennsylvania, Florida, Illinois. Sự phân bố này phản ánh quy mô dân số và mức độ phát triển kinh tế của các tiểu bang.

- **Region (Khu vực):** Có 4 khu vực địa lý. West (miền Tây) dẫn đầu với 3.203 giao dịch, tiếp theo là East (2.881), Central (2.315), và South (1.595). Sự chênh lệch này có thể do quy mô dân số, mật độ siêu thị hoặc các yếu tố văn hóa - kinh tế đặc thù của từng vùng miền.

Nhóm biến về sản phẩm:

- **Product ID (Mã sản phẩm):** Có 1.862 mã sản phẩm duy nhất, cho thấy danh mục sản phẩm của siêu thị khá đa dạng. Mã sản phẩm OFF-PA-10001970 xuất hiện nhiều nhất với 19 lần, đây có thể là một sản phẩm văn phòng phẩm bán chạy.

- **Category (Danh mục sản phẩm):** Có 3 danh mục chính. Office Supplies (văn phòng phẩm) chiếm tỷ trọng áp đảo với 6.026 giao dịch (khoảng 60,3%), cho thấy đây là nhóm hàng chủ lực, có nhu cầu ổn định và thường xuyên. Furniture (nội thất) có 2.127 giao dịch, Technology (công nghệ) có 1.841 giao dịch, tỷ lệ thấp hơn nhưng giá trị đơn hàng có thể cao hơn.

- **Sub-Category (Danh mục con):** Có 17 danh mục con. Binders (bìa kẹp) là danh mục con phổ biến nhất với 1.523 giao dịch, phù hợp với xu hướng chung của nhóm Office Supplies. Các danh mục con khác như Paper (giấy), Storage (lưu trữ), Furnishings (trang trí nội thất), Phones (điện thoại) cũng có tần suất cao, cho thấy sự đa dạng trong cơ cấu sản phẩm.

- **Product Name (Tên sản phẩm):** Có 1.850 tên sản phẩm duy nhất. Sản phẩm "Staple envelope" xuất hiện nhiều nhất với 48 lần, đây có thể là mặt hàng được mua thường xuyên với số lượng nhỏ, phục vụ nhu cầu văn phòng hàng ngày.

Những điểm đáng chú ý và định hướng xử lý:

- Biến **Country** chỉ có một giá trị duy nhất, không mang lại thông tin phân biệt, do đó sẽ được loại bỏ khỏi tập đặc trưng.

- Các biến **Order ID**, **Customer ID**, **Product ID** mang tính định danh cao, có số lượng giá trị duy nhất rất lớn (5009, 793, 1862). Việc đưa trực tiếp các biến này vào mô hình có thể gây tăng chiều dữ liệu và không mang lại giá trị tổng quát hóa. Do đó, các biến này sẽ được loại bỏ.

- Các biến **City** và **State** có số lượng giá trị duy nhất lần lượt là 531 và 49. Việc đưa vào mô hình có thể làm tăng độ phức tạp, nhưng có thể được xem xét nếu có ý nghĩa trong phân tích địa lý. Trong nghiên cứu này, biến **Region** (4 giá trị) sẽ được ưu tiên sử dụng thay vì **City** và **State**, vì nó vừa phản ánh yếu tố địa lý vừa đảm bảo tính khả thi trong mô hình hóa.

- Các biến **Ship Mode**, **Segment**, **Category**, **Sub-Category** có số lượng giá trị duy nhất vừa phải (4, 3, 3, 17), rất phù hợp để đưa vào mô hình sau khi được mã hóa bằng kỹ thuật *One-Hot Encoding*.

- Sự phân bố không đồng đều giữa các nhóm trong nhiều biến định tính (ví dụ: **Consumer** chiếm 52%, **Office Supplies** chiếm 60,3%) cho thấy dữ liệu có sự mất cân bằng nhất định. Điều này cần được lưu ý trong quá trình xây dựng mô hình để tránh hiện tượng thiên vị về lớp đa số.

1.4. Khám phá dữ liệu (EDA)

Sau khi đã có cái nhìn tổng quan về cấu trúc và thống kê mô tả của bộ dữ liệu, bước tiếp theo là khám phá sâu hơn các mối quan hệ tiềm ẩn, phát hiện những điểm bất thường và xác định các đặc trưng có ảnh hưởng lớn đến biến mục tiêu. Quá trình này được thực hiện thông qua các biểu đồ trực quan và các phép tính thống kê bổ sung.

1.4.1. Xác định giá trị duy nhất cho biến phân loại

Trước khi đưa các biến phân loại vào mô hình, việc xác định số lượng giá trị duy nhất của từng biến là cần thiết để lựa chọn phương pháp mã hóa phù hợp. Bảng dưới đây liệt kê chi tiết các biến phân loại quan trọng:

```
# Liệt kê các cột phân loại cần kiểm tra
cat_cols = ['Ship Mode', 'Segment', 'Country', 'Region', 'Category',
            'Sub-Category']

unique_data = []
for col in cat_cols:
    unique_data.append({
        "Tên biến": col,
        "Số lượng Unique": df[col].nunique(),
        "Danh sách mẫu (tất cả giá trị)": ", ".join(map(str,
df[col].unique()))),
    })

df_unique = pd.DataFrame(unique_data)

display(df_unique.style.set_properties(**{
    'background-color': '#e0f2f1',
    'color': '#084594',
    'border-color': 'white',
```

```

)).set_table_styles([
    {'selector': 'th', 'props': [('background-color', '#003366'),
    ('color', 'white')]}
]))

```

	Tên biến	Số lượng Unique	Danh sách mẫu (tất cả giá trị)
0	Ship Mode	4	Second Class, Standard Class, First Class, Same Day
1	Segment	3	Consumer, Corporate, Home Office
2	Country	1	United States
3	Region	4	South, West, Central, East
4	Category	3	Furniture, Office Supplies, Technology
5	Sub-Category	17	Bookcases, Chairs, Labels, Tables, Storage, Furnishings, Art, Phones, Binders, Appliances, Paper, Accessories, Envelopes, Fasteners, Supplies, Machines, Copiers

Hình 1. 5. Số lượng và danh sách các giá trị duy nhất của các biến

Ship Mode: Với 4 giá trị duy nhất, biến này phản ánh các lựa chọn dịch vụ vận chuyển từ tiêu chuẩn đến cao cấp. Số lượng nhỏ giúp việc mã hóa dễ dàng, có thể sử dụng One-Hot Encoding hoặc thậm chí là Label Encoding nếu có thứ tự (Standard < Second < First < Same Day) nhưng thứ tự này không đảm bảo tính chất tuyến tính, vì vậy *One-Hot* sẽ an toàn hơn.

Segment: Ba phân khúc khách hàng cho phép phân tích hành vi mua sắm theo nhóm đối tượng. Đây là biến quan trọng trong các mô hình marketing và dự báo lợi nhuận.

Country: Chỉ có một giá trị duy nhất, do đó biến này không có giá trị phân biệt và sẽ được *loại bỏ* hoàn toàn khỏi tập đặc trưng để tránh gây nhiễu cho mô hình.

Region: Bốn khu vực địa lý của Hoa Kỳ. Biến này có thể chứa thông tin về sự khác biệt về chi phí vận chuyển, thuế, và hành vi tiêu dùng theo vùng miền.

Category: Ba danh mục sản phẩm chính. Đây là biến phân loại quan trọng nhất vì các danh mục có biên lợi nhuận và độ nhạy cảm với chiết khấu khác nhau.

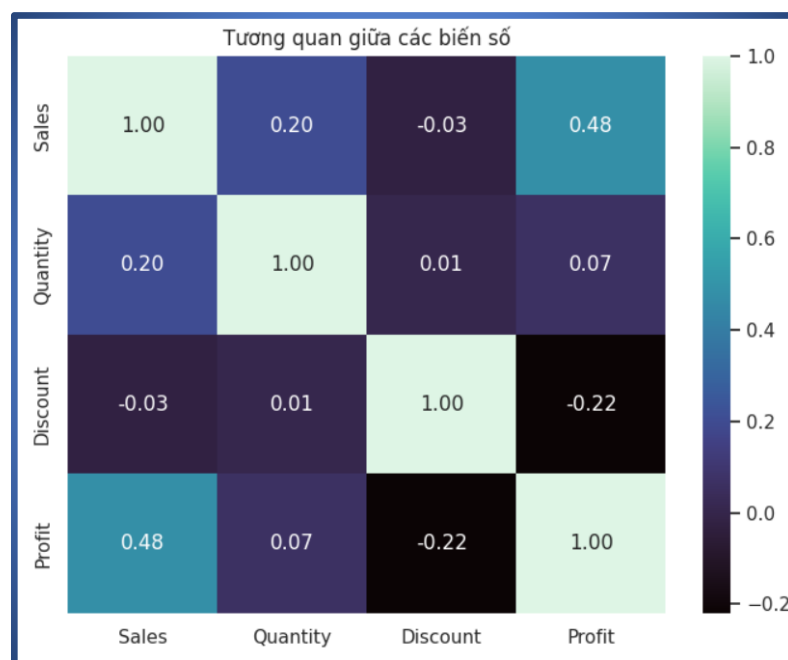
Sub-Category: Với 17 giá trị, đây là biến có số lượng nhóm lớn nhất trong số các biến định tính. Việc đưa trực tiếp Sub-Category vào mô hình sẽ làm tăng số chiều dữ liệu đáng kể (sau One-Hot sẽ thêm 16 cột). Tuy nhiên, thông tin về từng loại sản phẩm cụ thể có thể rất hữu ích. Trong nghiên cứu này, chúng ta sẽ cân nhắc sử dụng Category thay vì Sub-Category để giảm độ phức tạp, nhưng vẫn giữ lại Sub-Category trong các phân tích EDA để hiểu rõ hơn về cấu trúc bán hàng.

Định hướng xử lý: Các biến Ship Mode, Segment, Region, Category sẽ được giữ lại và mã hóa bằng One-Hot Encoding. Biến Country bị loại bỏ. Biến Sub-Category có thể được xem xét đưa vào nếu sau khi thử nghiệm mô hình thấy cần thiết, nhưng trong khuôn khổ báo cáo này, chúng ta tập trung vào các biến có độ phân giải vừa phải để tránh overfitting.

1.4.2. Heatmap tương quan

Ma trận tương quan là công cụ thống kê quan trọng để đo lường mức độ và chiều hướng của mối quan hệ tuyến tính giữa các biến định lượng. Hệ số tương quan Pearson (r) có giá trị nằm trong khoảng $[-1, 1]$, trong đó giá trị tuyệt đối càng gần 1 cho thấy mối quan hệ tuyến tính càng mạnh, dấu dương thể hiện mối quan hệ đồng biến, dấu âm thể hiện mối quan hệ nghịch biến.

```
plt.figure(figsize=(8, 6))
sns.heatmap(df[['Sales', 'Quantity', 'Discount', 'Profit']].corr(),
            annot=True, cmap='mako', fmt='.2f')
plt.title('Tương quan giữa các biến số')
plt.show()
```



Hình 1. 6. Phân tích tương quan giữa các biến định lượng

Phân tích hệ số tương quan:

- **Sales và Profit (hệ số 0.48):** Mối tương quan dương ở mức trung bình giữa doanh số và lợi nhuận cho thấy rằng khi doanh số tăng, lợi nhuận có xu hướng tăng theo. Tuy nhiên, hệ số 0.48 không phải là một mối quan hệ tuyến tính mạnh (chỉ ở mức vừa phải), có ý nghĩa quan trọng là *không phải giao dịch nào có doanh số cao cũng đảm bảo lợi nhuận cao*. Có những yếu tố khác (đặc biệt là chiết khấu, chi phí vận chuyển hoặc đặc tính của từng danh mục sản phẩm) có thể làm giảm đáng kể lợi nhuận ngay cả khi doanh số lớn. Vậy Sales là một yếu tố dự báo quan trọng nhưng không phải là yếu tố duy nhất.

- **Sales và Quantity (hệ số 0.20):** Tương quan dương yếu giữa doanh số và số lượng sản phẩm bán ra, cho thấy việc bán được nhiều sản phẩm hơn có xu hướng làm tăng doanh số, nhưng mức độ tác động không mạnh. Nguyên nhân có thể là do sự chênh lệch về giá trị đơn vị của các sản phẩm: bán 10 chiếc bút chì (giá thấp) có thể tạo ra doanh số thấp

hơn so với bán 1 chiếc máy tính (giá cao). Do đó, số lượng không phải là chỉ số đại diện tốt cho quy mô tài chính của giao dịch.

- ***Sales và Discount (hệ số -0.03)***: Hệ số gần như bằng 0 cho thấy không có mối quan hệ tuyến tính đáng kể giữa chiết khấu và doanh số. Đây là một phát hiện thú vị và trái với trực giác kinh doanh thông thường (giảm giá thường kỳ vọng kích thích doanh số). Trong tập dữ liệu này, việc áp dụng chiết khấu không làm tăng doanh số một cách có hệ thống. Có thể lý giải rằng các chương trình giảm giá chưa đủ hấp dẫn để thay đổi hành vi mua hàng hoặc sản phẩm có độ co giãn cầu thấp hoặc khách hàng của siêu thị này ít nhạy cảm về giá. Điều này đặt ra câu hỏi về hiệu quả thực sự của các chiến lược khuyến mãi dựa trên giảm giá.

- ***Profit và Quantity (hệ số 0.07)***: Tương quan dương rất yếu, gần như không có ý nghĩa thực tiễn. Khẳng định rằng số lượng sản phẩm bán ra không phải là yếu tố quyết định đến lợi nhuận. Một đơn hàng số lượng lớn vẫn có thể thua lỗ nếu chiết khấu quá sâu hoặc chi phí vận chuyển cao, trong khi một đơn hàng số lượng nhỏ nhưng có biên lợi nhuận tốt vẫn có thể sinh lời.

- ***Profit và Discount (hệ số -0.22)***: Mối tương quan nghịch ở mức vừa phải giữa chiết khấu và lợi nhuận cho biết chiết khấu cao có xu hướng làm giảm lợi nhuận. Hệ số -0.22 không quá lớn, cho thấy tác động này không đồng nhất và có thể phụ thuộc vào các yếu tố khác như danh mục sản phẩm. Trên thực tế, như đã thấy trong biểu đồ phân tán, có những danh mục như Technology chịu được chiết khấu tốt hơn, trong khi Furniture rất nhạy cảm.

- ***Quantity và Discount (hệ số 0.01)***: Không có mối quan hệ tuyến tính giữa số lượng bán và mức chiết khấu được áp dụng, cho thấy chiết khấu không được sử dụng như một công cụ để kích thích mua số lượng lớn hoặc nếu có thì hiệu quả không thể hiện qua tương quan tuyến tính.

Ý nghĩa thực tiễn:

- Sales là yếu tố dự báo quan trọng nhất đối với Profit với hệ số tương quan 0.48.
- Chiết khấu có tác động tiêu cực đến lợi nhuận (hệ số -0.22), nhưng không làm tăng doanh số (hệ số -0.03).
- Số lượng bán (Quantity) có ảnh hưởng khá hạn chế đến cả doanh số và lợi nhuận.
- Không có hiện tượng đa cộng tuyến nghiêm trọng giữa các biến độc lập, do đó có thể đưa cả ba biến Sales, Quantity, Discount vào mô hình.

1.4.3. Biểu đồ phát hiện ngoại lai

Trong bối cảnh phân tích dữ liệu bán hàng, xác định các outlier giúp nhận diện những giao dịch đặc biệt có thể ảnh hưởng đến kết quả thống kê và hiệu suất mô hình.

```

flier_style = dict(marker='o', markerfacecolor='#3d5a80', markersize=7,
                    markeredgecolor='w', alpha=0.7)

plt.figure(figsize=(20, 12))
sns.set_style("whitegrid", {'axes.facecolor': '#eceff1'})

# Subplot 1: Sales
plt.subplot(2, 3, 1)
sns.boxplot(y=df['Sales'], color='#08415c', flierprops=flier_style,
            width=0.4)
plt.title('1. Ngoại lai: Doanh số (Sales)', fontsize=13,
          fontweight='bold', color='#212121')

# Subplot 2: Profit
plt.subplot(2, 3, 2)
sns.boxplot(y=df['Profit'], color='#2a9d8f', flierprops=flier_style,
            width=0.4)
plt.title('2. Ngoại lai: Lợi nhuận (Profit)', fontsize=13,
          fontweight='bold', color='#212121')

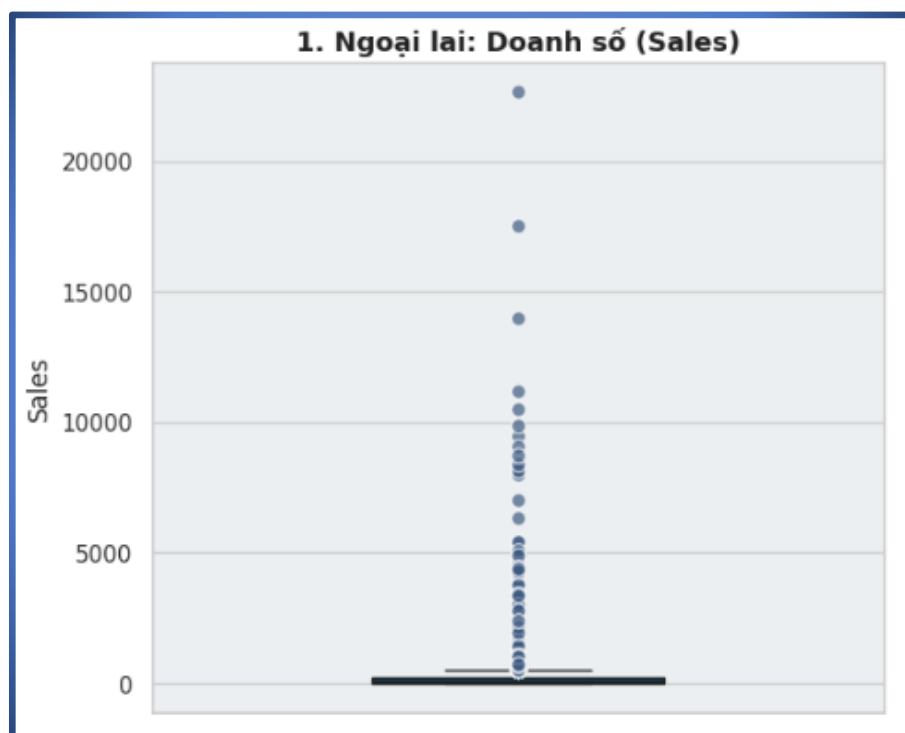
# Subplot 3: Profit theo Category (Theo nhóm hàng)
plt.subplot(2, 3, 3)
sns.boxplot(x='Category', y='Profit', data=df, palette='mako',
            flierprops=flier_style)
plt.title('3. Biến động Profit theo Category', fontsize=13,
          fontweight='bold', color='#212121')

# Subplot 4: Discount (Chiết khấu)
plt.subplot(2, 3, 4)
sns.boxplot(x=df['Discount'], color='#455a64', flierprops=flier_style)
plt.title('4. Ngoại lai: Chiết khấu (Discount)', fontsize=13,
          fontweight='bold', color='#212121')

# Subplot 5: Quantity (Số lượng)
plt.subplot(2, 3, 5)
sns.boxplot(y=df['Quantity'], color='#546e7a', flierprops=flier_style,
            width=0.4)
plt.title('5. Ngoại lai: Số lượng (Quantity)', fontsize=13,
          fontweight='bold', color='#212121')

plt.tight_layout(pad=4.0)
plt.show()

```



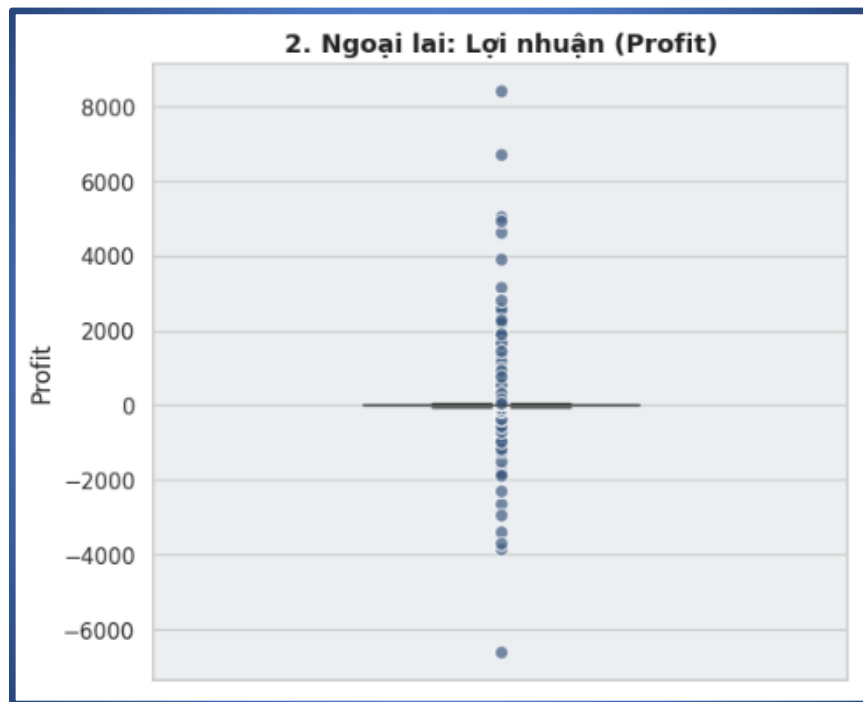
Hình 1. 7. Ngoại lai trong doanh số (Sales)

Ngoại lai: Doanh số (Sales)

Biểu đồ của biến Sales (Doanh số) cho thấy phân phối dữ liệu bị lệch phải rõ rệt, với phần lớn các quan sát tập trung ở mức doanh số thấp, chủ yếu gần 0. Trung vị nằm ở mức thấp và khoảng tứ phân vị (IQR) tương đối hẹp, phản ánh rằng đa số các giao dịch có giá trị nhỏ và mức độ biến động không lớn trong khoảng này.

Đáng chú ý là sự xuất hiện của nhiều giá trị ngoại lai ở phía trên của phân phối: Các điểm dữ liệu có doanh số cao phân bố rải rác từ vài nghìn đến trên 20.000, nằm cách xa phần thân. Đây có thể là các giao dịch có quy mô lớn, chẳng hạn như đơn hàng số lượng nhiều hoặc giao dịch với khách hàng doanh nghiệp. Mặc dù các giá trị này không nhất thiết là sai lệch, nhưng chúng khác biệt đáng kể so với phần lớn dữ liệu còn lại.

Sự tồn tại của các giá trị lớn này làm cho phân phối doanh số trở nên mất cân đối và có thể ảnh hưởng đến các chỉ tiêu thống kê như giá trị trung bình. Vì vậy, trong quá trình xử lý dữ liệu, cần cân nhắc áp dụng các phương pháp như biến đổi log hoặc giới hạn giá trị (winsorization) nhằm giảm tác động của các giá trị cực đoan, qua đó giúp kết quả phân tích và mô hình hóa phản ánh sát hơn đặc điểm chung của dữ liệu.



Hình 1. 8. Ngoại lai trong lợi nhuận (Profit)

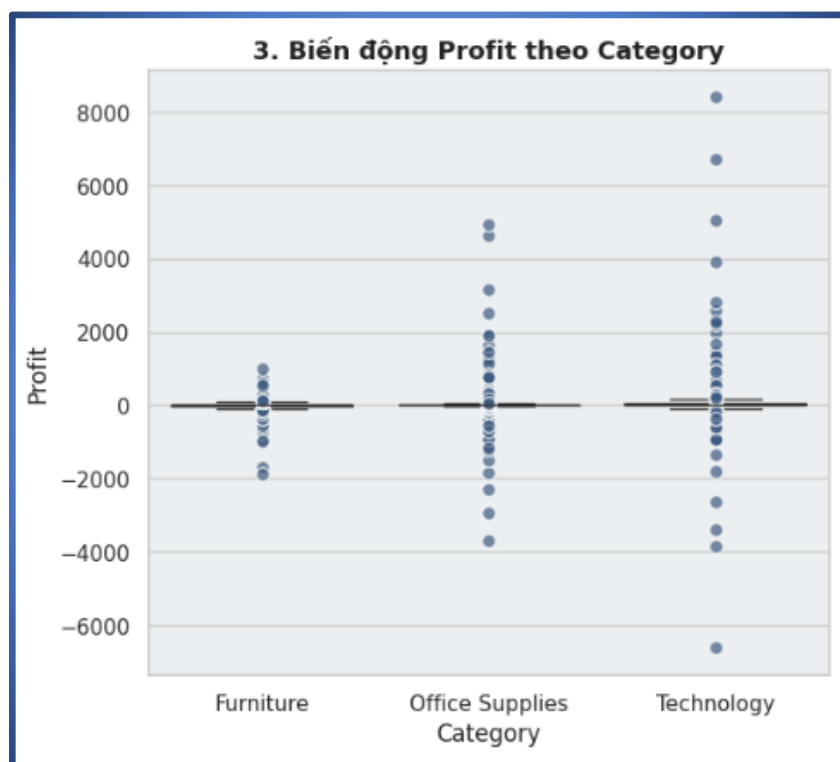
Ngoại lai: Lợi nhuận (Profit)

Biểu đồ của biến Profit (Lợi nhuận) cho thấy phân phối dữ liệu trải rộng hơn so với Sales và có sự phân tán đáng kể ở cả hai phía. Phần lớn các quan sát tập trung quanh mức gần 0, với trung vị xấp xỉ 0, cho thấy nhiều giao dịch chỉ đạt mức lợi nhuận rất thấp hoặc hòa vốn. Khoảng tứ phân vị (IQR) tương đối hẹp, phản ánh phần lớn dữ liệu không biến động mạnh trong vùng trung tâm.

Điểm đáng chú ý là sự xuất hiện của các giá trị ngoại lai ở cả hai phía của phân phối:

- Phía dưới: Có nhiều điểm lợi nhuận âm sâu, kéo dài xuống đến khoảng -6.000. Các giá trị này nằm xa phần thân, cho thấy đây là những giao dịch thua lỗ lớn. Nguyên nhân có thể liên quan đến chi phí cao, chiết khấu lớn hoặc cấu trúc giá chưa hợp lý. Những trường hợp này cần được xem xét kỹ để hạn chế rủi ro trong hoạt động kinh doanh.
- Phía trên: Đồng thời, cũng xuất hiện các giá trị lợi nhuận dương cao, lên đến hơn 8.000. Đây là những giao dịch mang lại lợi nhuận vượt trội so với mặt bằng chung, có thể đến từ các đơn hàng lớn hoặc sản phẩm có biên lợi nhuận cao.

Sự phân tán mạnh của lợi nhuận phản ánh tính bất định trong hoạt động kinh doanh. Mục tiêu của mô hình là dự đoán khả năng sinh lời (phân loại nhị phân), do đó việc xử lý outlier ở Profit sẽ được thực hiện đồng thời với Sales bằng Winsorization, nhằm giảm ảnh hưởng của các giá trị cực đoan mà không làm mất đi tính đại diện của dữ liệu.



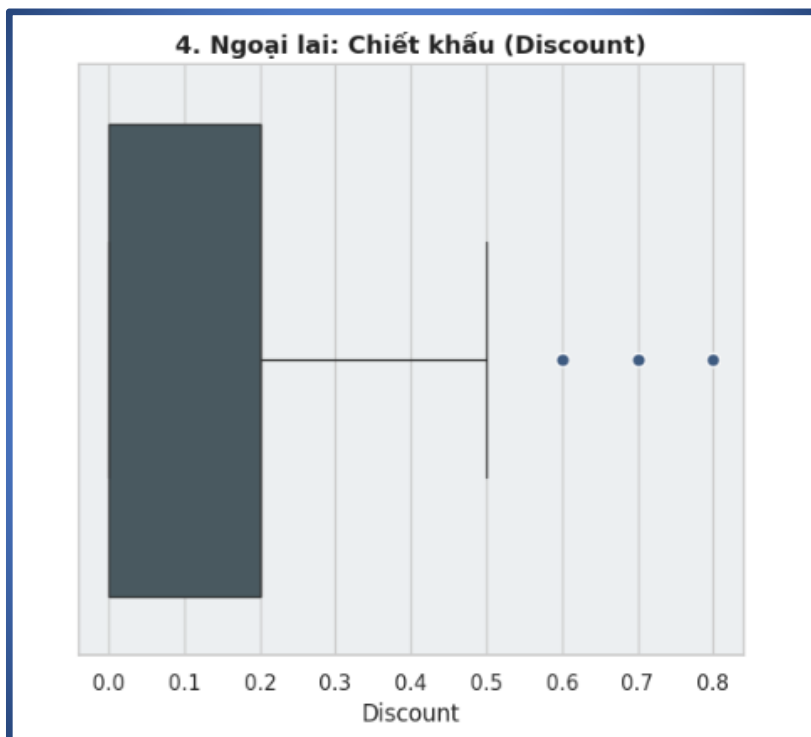
Hình 1. 9. Phân bố và biến động Profit theo nhóm sản phẩm

Biến động Profit theo Category

Biểu đồ thể hiện sự phân bố lợi nhuận theo từng nhóm sản phẩm (Category) cho thấy mức độ biến động khác nhau giữa ba nhóm: Furniture, Office Supplies và Technology. Nhìn chung, trung vị của cả ba nhóm đều nằm gần mức 0, cho thấy phần lớn các giao dịch chỉ tạo ra lợi nhuận thấp hoặc dao động quanh điểm hòa vốn. Khoảng tứ phân vị (IQR) của các nhóm tương đối hẹp, phản ánh rằng phần lớn dữ liệu tập trung trong một khoảng giá trị không quá rộng. Tuy nhiên, sự khác biệt rõ rệt nằm ở mức độ và phạm vi của các giá trị ngoại lai:

- **Furniture**: Nhóm này có ít outlier hơn và phạm vi biến động tương đối hẹp. Lợi nhuận chủ yếu dao động trong khoảng nhỏ, tuy vẫn xuất hiện một số giá trị âm đáng kể (dưới -1.500) và một số giá trị dương nhưng không quá lớn. Điều này cho thấy Furniture có mức sinh lời tương đối ổn định nhưng không cao.
- **Office Supplies**: Nhóm này có số lượng outlier nhiều hơn, với cả giá trị âm và dương trải rộng. Lợi nhuận âm có thể xuống dưới -3.000, trong khi lợi nhuận dương có thể đạt trên 4.000. Điều này phản ánh mức độ biến động trung bình, với cả rủi ro và cơ hội sinh lời đáng kể.
- **Technology**: Đây là nhóm có mức độ phân tán lớn nhất. Các giá trị ngoại lai xuất hiện dày đặc ở cả hai phía, đặc biệt có những điểm lợi nhuận rất cao (trên 8.000) và cũng có những điểm lỗ sâu (dưới -6.000). Điều này cho thấy Technology là nhóm có tiềm năng sinh lời cao nhất nhưng đồng thời cũng đi kèm rủi ro lớn nhất.

Qua đó có thể thấy tầm quan trọng của biến Category trong việc dự đoán lợi nhuận. Do đó, mô hình sẽ được thiết kế để có thể học được sự khác biệt này và trong quá trình xử lý dữ liệu thì Category sẽ được giữ lại như một đặc trưng quan trọng.



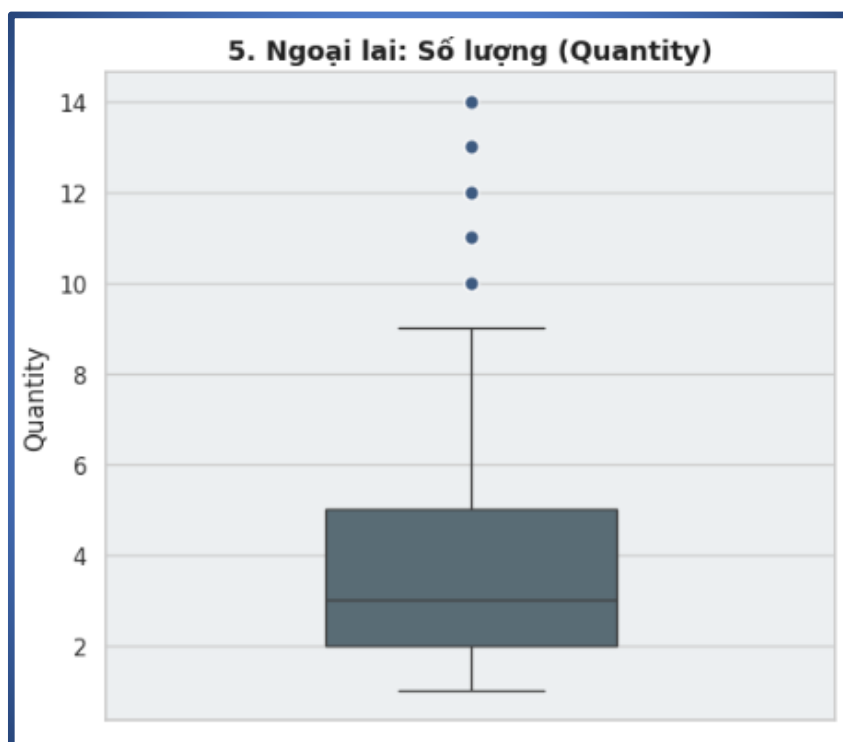
Hình 1. 10. Ngoại lai trong biến chiết khấu (Discount)

Ngoại lai: Chiết khấu (Discount)

Biểu đồ của biến Discount (Chiết khấu) cho thấy phân phối dữ liệu lệch phải, với phần lớn các giá trị tập trung ở mức thấp. Cụ thể, chiết khấu chủ yếu nằm trong khoảng từ 0 đến khoảng 0.2, với trung vị ở mức tương đối thấp, phản ánh rằng đa số các giao dịch chỉ áp dụng mức giảm giá nhỏ.

Đáng chú ý là sự xuất hiện của các giá trị ngoại lai ở phía trên của phân phối: Một số điểm dữ liệu có mức chiết khấu cao bất thường, dao động từ khoảng 0.6 đến 0.8, nằm tách biệt hoàn toàn so với phần còn lại của dữ liệu. Đây có thể là các trường hợp áp dụng chương trình khuyến mãi lớn, xả hàng tồn kho hoặc chiến lược kích cầu đặc biệt. Tuy nhiên, các giá trị này xuất hiện với tần suất rất thấp và không đại diện cho xu hướng chung.

Phân phối chiết khấu cho thấy phần lớn hoạt động kinh doanh diễn ra với mức giảm giá thấp, trong khi một số ít trường hợp áp dụng mức chiết khấu rất cao (60% - 80%), có thể là các chương trình thanh lý hàng tồn kho, khuyến mãi đặc biệt, hoặc áp dụng cho một số sản phẩm nhất định. Cần xem xét hiệu quả của các chương trình này: liệu chúng có thực sự kích thích doanh số hay chỉ đơn thuần làm xói mòn lợi nhuận? Kết hợp với phân tích tương quan (Discount - Profit: -0.22), có thể thấy chiết khấu cao thường đi kèm với lợi nhuận thấp, đặc biệt ở nhóm Furniture.



Hình 1. 11. Ngoại lai trong biến số lượng (Quantity)

Ngoại lai: Số lượng (Quantity)

Biểu đồ của biến Quantity (Số lượng) cho thấy phân phối tương đối tập trung và đối xứng. Phần thân nằm trong khoảng từ 2 đến 5 sản phẩm với trung vị là 3, phù hợp với đặc điểm bán lẻ. Phân vị 25% (Q1) là 2 và phân vị 75% (Q3) là 5, cho thấy 50% giao dịch ở giữa có số lượng từ 2 đến 5.

Râu trên của biểu đồ kéo dài đến khoảng 8-9 sản phẩm. Các điểm nằm phía trên râu (outlier) xuất hiện ở các mức số lượng từ 10 đến 14 sản phẩm. Đây là các đơn hàng có số lượng lớn bất thường, có thể là các giao dịch bán buôn hoặc khách hàng mua sắm tích trữ. Số lượng các outlier này rất ít so với tổng số quan sát.

Tuy nhiên, mức độ ảnh hưởng của các outlier này đến mô hình có thể không lớn, vì số lượng các giao dịch này ít và mối tương quan với Profit cũng rất yếu (0.07). Do đó, trong xử lý dữ liệu, có thể giữ nguyên các outlier này mà không cần can thiệp mạnh.

Kết luận về xử lý ngoại lai:

Từ các phân tích trên, nghiên cứu quyết định áp dụng phương pháp Winsorization cho các biến Sales và Profit, là những biến có ảnh hưởng lớn đến mô hình và có nhiều outlier cực đoan. Cụ thể:

- **Sales**: Chặn trên tại percentile 99% (khoảng 2.480 đô la) để giảm ảnh hưởng của các giao dịch có doanh số quá cao (trên 20.000 đô la), trong khi vẫn giữ lại phần lớn cấu trúc phân phối.

- **Profit**: Chặn dưới tại percentile 1% (khoảng -320 đô la) và chặn trên tại percentile 99% (khoảng 580 đô la) để loại bỏ các giá trị lỗ sâu (dưới -2.000 đô la) và lãi cực cao (trên 5.000 đô la), vốn có thể làm lệch mô hình.

Đối với biến **Discount**, chỉ các mức chiết khấu từ 60% đến 80% được xác định là outlier. Tuy nhiên, do đây là các giá trị có ý nghĩa thực tế (các chương trình khuyến mãi đặc biệt), việc giữ nguyên có thể mang lại thông tin hữu ích cho mô hình. Đối với biến **Quantity**, các outlier ở mức 10-14 sản phẩm có thể được giữ nguyên do số lượng ít và ảnh hưởng không lớn.

1.4.4. Scatter Discount vs Profit (phân loại theo Category)

```
plt.figure(figsize=(10, 6))

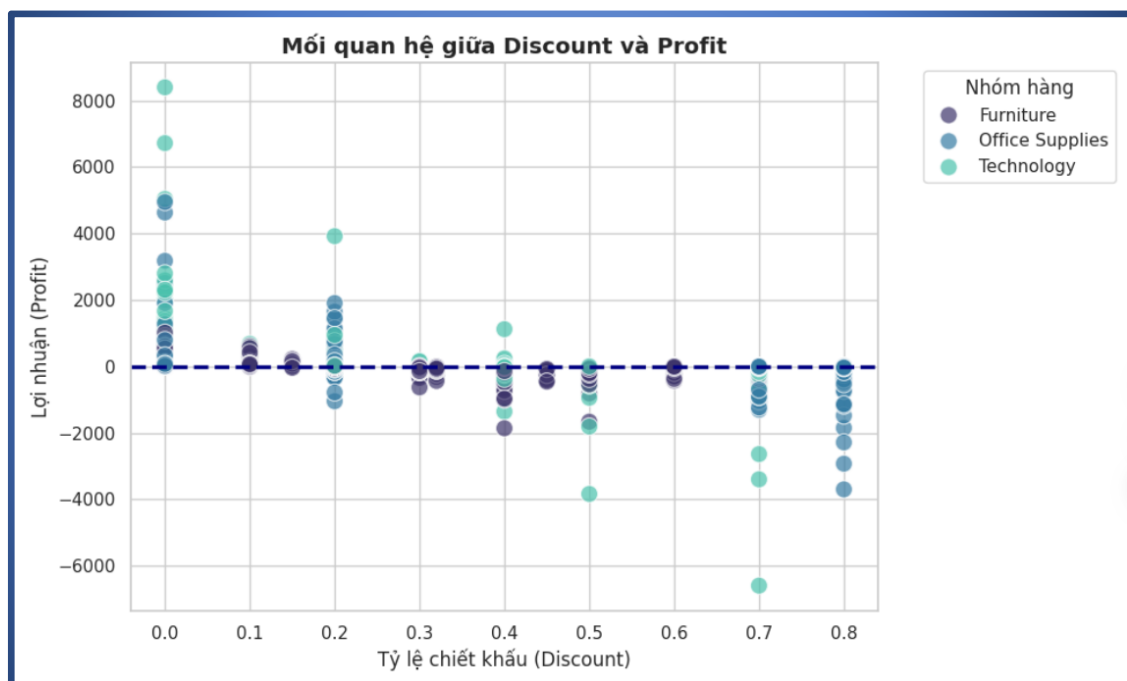
sns.scatterplot(data=df, x='Discount', y='Profit', hue='Category',
                palette='mako', s=120, edgecolor='w', alpha=0.7)

plt.axhline(0, color='#000080', linestyle='--', linewidth=2.5)

plt.title('Mối quan hệ giữa Discount và Profit', fontsize=14,
          fontweight='bold')
plt.xlabel('Tỷ lệ chiết khấu (Discount)')
plt.ylabel('Lợi nhuận (Profit)')

plt.legend(title='Nhóm hàng', bbox_to_anchor=(1.05, 1), loc='upper
left')

plt.tight_layout()
plt.show()
```



Hình 1. 12. Mối quan hệ giữa tỷ lệ chiết khấu (Discount) và lợi nhuận (Profit)

1.4.4.1. Phân tích tổng thể theo các vùng chiết khấu

Vùng chiết khấu thấp ($Discount \leq 0.2$)

Ở vùng chiết khấu từ 0% đến 20%, quan sát cho thấy phần lớn các điểm dữ liệu tập trung ở phía trên đường $y = 0$, tức là vùng lợi nhuận dương. Các điểm phân bố tương đối tập trung, với mức độ phân tán không quá lớn. Điều này cho thấy khi chiết khấu được giữ ở mức vừa phải (không quá 20%), các giao dịch có xu hướng mang lại lợi nhuận dương một cách ổn định. Đây có thể được xem là **vùng an toàn** cho các chính sách khuyến mãi.

Trong vùng này, cả ba danh mục sản phẩm đều có sự hiện diện, nhưng Technology và Office Supplies có mật độ điểm cao hơn ở vùng lợi nhuận dương, trong khi Furniture đã bắt đầu xuất hiện một số điểm ở vùng lợi nhuận âm dù chiết khấu còn thấp.

Vùng chiết khấu trung bình ($0.2 < Discount \leq 0.5$)

Khi chiết khấu vượt qua ngưỡng 20% thì tổng thể bắt đầu thay đổi rõ rệt. Các điểm dữ liệu bắt đầu phân tán mạnh, xuất hiện nhiều điểm ở cả hai phía của đường $y = 0$. Độ phân tán tăng dần khi chiết khấu tiến gần đến 0.5, thể hiện qua sự trải rộng của các điểm theo trục tung (lợi nhuận). Điều này cho thấy khi chiết khấu ở mức 20% đến 50%, kết quả kinh doanh trở nên khó dự đoán hơn: vẫn có những giao dịch mang lại lợi nhuận tốt, nhưng cũng xuất hiện nhiều giao dịch thua lỗ.

Ngưỡng 0.2 (20%) có thể được xem là một **điểm bẻ gãy** quan trọng, nơi rủi ro thua lỗ bắt đầu gia tăng đáng kể. Đây là vùng cần thận trọng, đòi hỏi các quyết định chiết khấu phải được cân nhắc kỹ lưỡng dựa trên đặc điểm của từng danh mục sản phẩm.

Vùng chiết khấu cao ($Discount > 0.5$)

Khi chiết khấu vượt quá 50%, số lượng quan sát giảm mạnh, cho thấy các mức chiết khấu này ít được áp dụng. Các điểm dữ liệu trong vùng này hầu hết nằm ở vùng lợi nhuận âm, đặc biệt là ở phía dưới sâu (từ -2.000 đến -6.000 đô la). Điều này cho thấy chiết khấu quá sâu (trên 50%) hầu như luôn dẫn đến thua lỗ nghiêm trọng. Đây là **vùng nguy hiểm**, các chương trình khuyến mãi ở mức này (nếu có) cần được xem xét lại vì hiệu quả kinh doanh rất thấp.

1.4.4.2. Phân tích theo từng danh mục sản phẩm

Furniture (Nội thất)

Danh mục Furniture thể hiện sự nhạy cảm cao nhất với chiết khấu trong cả ba nhóm:

- Ở vùng chiết khấu thấp (dưới 20%), đã xuất hiện một số điểm có lợi nhuận âm, cho thấy ngay cả khi chiết khấu khiêm tốn, nhóm hàng này vẫn tiềm ẩn rủi ro thua lỗ.

- Khi chiết khấu vượt quá 20%, số lượng điểm rơi vào vùng lợi nhuận âm tăng mạnh. Đặc biệt, các điểm có lợi nhuận âm sâu (dưới -2.000 đô la) tập trung chủ yếu ở danh mục này.

- Ở vùng chiết khấu trên 30%, hầu hết các giao dịch Furniture đều nằm dưới đường $y = 0$, cho thấy rủi ro thua lỗ gần như chắc chắn khi chiết khấu ở mức cao.

Nguyên nhân có thể do biên lợi nhuận của các sản phẩm nội thất thấp, chi phí vận chuyển và lưu kho lớn, hoặc do đặc thù cạnh tranh về giá trong ngành hàng này. Đối với nhóm Furniture, khuyến nghị là *hạn chế tối đa các chương trình chiết khấu*, đặc biệt khi chiết khấu vượt quá 20%.

Technology (Công nghệ)

Danh mục Technology thể hiện khả năng chịu chiết khấu tốt nhất trong ba nhóm:

- Ở vùng chiết khấu thấp và trung bình (0-50%), phần lớn các điểm nằm ở vùng lợi nhuận dương, với nhiều điểm có lợi nhuận cao (trên 2.000 đô la).

- Ngay cả khi chiết khấu lên đến 60%, vẫn có những giao dịch Technology giữ được lợi nhuận dương, mặc dù số lượng ít hơn.

- Các outlier dương (lợi nhuận rất cao) tập trung chủ yếu ở danh mục này, cho thấy tiềm năng sinh lời lớn.

Nguyên nhân có thể do các sản phẩm công nghệ có biên lợi nhuận cao, nhu cầu ít co giãn theo giá hoặc khách hàng sẵn sàng chi trả cho các sản phẩm có giá trị cao. Đối với nhóm Technology, có thể *linh hoạt áp dụng các chương trình chiết khấu* để kích thích doanh số mà vẫn đảm bảo lợi nhuận, đặc biệt là các chương trình giảm giá theo mùa hoặc theo dịp lễ.

Office Supplies (Văn phòng phẩm)

Danh mục Office Supplies có xu hướng trung gian giữa Furniture và Technology:

- Ở vùng chiết khấu thấp (dưới 20%), hầu hết các điểm nằm ở vùng lợi nhuận dương, thể hiện sự ổn định.

- Khi chiết khấu vượt quá 30%, bắt đầu xuất hiện nhiều điểm ở vùng lợi nhuận âm, nhưng vẫn còn một số điểm dương.

- Ở vùng chiết khấu trên 50%, số lượng giao dịch ít, nhưng các điểm còn lại chủ yếu nằm ở vùng lợi nhuận âm.

Nhóm Office Supplies có đặc điểm sản phẩm đa dạng, từ các mặt hàng giá trị thấp (bút, giấy) đến các mặt hàng giá trị cao hơn (máy in, máy tính). Do đó, tác động của chiết khấu có thể khác nhau giữa các sản phẩm trong cùng danh mục. Khuyến nghị là *có chính sách chiết khấu phân biệt* theo từng phân khúc sản phẩm trong nhóm Office Supplies.

PHẦN II. GIỚI THIỆU BÀI TOÁN

2.1. Bài toán đang giải quyết

Bộ dữ liệu Sample Superstore tái hiện hoạt động kinh doanh thực tế của một chuỗi siêu thị tại Hoa Kỳ trong khoảng thời gian 4 năm (2014-2017), bao gồm tổng cộng 9.994 giao dịch và 21 thuộc tính mô tả chi tiết từng giao dịch. Quá trình phân tích khám phá dữ liệu đã làm rõ nhiều đặc điểm quan trọng, đồng thời phát hiện những bất thường và các mối quan hệ mang ý nghĩa thực tiễn sâu sắc.

Biến lợi nhuận (Profit) đại diện cho kết quả tài chính cuối cùng của mỗi giao dịch thể hiện phân phối lệch phải rõ rệt. Sự lệch này được thể hiện qua khoảng cách đáng kể giữa giá trị trung bình (28,66 USD) và trung vị (8,67 USD), cùng với biên độ biến thiên cực lớn từ -6.599,98 USD đến 8.399,98 USD. Sự hiện diện đồng thời của các giao dịch lỗ nặng và các giao dịch lãi lớn cho thấy hoạt động kinh doanh của siêu thị tiềm ẩn cả rủi ro tài chính đáng kể lẫn cơ hội sinh lời cao. Đặc biệt, các giao dịch thua lỗ nghiêm trọng (giá trị âm sâu) đặt ra vấn đề cần được làm rõ: *nguyên nhân có xuất phát từ mức chiết khấu quá cao, đặc thù của một số danh mục sản phẩm hay sự kết hợp đồng thời của nhiều yếu tố?*

Phân tích tương quan giữa các biến định lượng đã làm nổi bật một số mối quan hệ có ý nghĩa. Hệ số tương quan Pearson giữa Discount và Profit đạt -0,22, cho thấy xu hướng nghịch biến: mức chiết khấu càng cao thì lợi nhuận càng có khả năng giảm. Tuy nhiên, giá trị tuyệt đối của hệ số này không quá lớn, ngụ ý rằng tác động của chiết khấu không đồng nhất mà chịu ảnh hưởng bởi các yếu tố điều tiết khác. Quan sát này được củng cố mạnh mẽ qua biểu đồ phân tán (scatter plot) giữa Discount và Profit, được phân loại theo danh mục sản phẩm (Category). Biểu đồ cho thấy sự khác biệt rất rõ nét: đối với danh mục Furniture, chỉ cần mức chiết khấu vượt quá 20% đã xuất hiện nhiều giao dịch rơi vào vùng lợi nhuận âm; trong khi đó, đối với danh mục Technology thì ngay cả khi chiết khấu đạt 50-60%, vẫn còn khá nhiều giao dịch duy trì được lợi nhuận dương. Như vậy, danh mục sản phẩm không chỉ đóng vai trò là một biến phân loại thông thường, mà còn là yếu tố điều tiết quan trọng ảnh hưởng đến mối quan hệ giữa chiết khấu và lợi nhuận.

Biểu đồ phân tách lợi nhuận theo danh mục sản phẩm càng làm rõ thêm sự khác biệt này. Danh mục Furniture có trung vị lợi nhuận gần bằng 0 và xuất hiện nhiều điểm ngoại lai âm sâu; Technology có trung vị dương cao nhất cùng các điểm ngoại lai tập trung chủ yếu ở vùng dương; trong khi Office Supplies nằm ở vị trí trung gian với mức độ ổn định tương đối nhưng không đạt được tiềm năng sinh lời cao như Technology. Những phát hiện này không chỉ mang giá trị thống kê mà còn có ý nghĩa quản trị thực tiễn sâu sắc: chúng gợi ý rằng chiến lược định giá, khuyến mãi và quản lý chiết khấu cần được xây dựng riêng biệt, phù hợp với đặc thù của từng danh mục sản phẩm.

Từ những phân tích trên, vấn đề kinh doanh cốt lõi được xác định rõ ràng: ***Liệu siêu thị có thể dự báo trước liệu một giao dịch sắp thực hiện sẽ mang lại lợi nhuận hay thua lỗ?*** Việc trả lời được câu hỏi này sẽ cung cấp cho nhà quản trị một công cụ hỗ trợ ra quyết định kịp thời và hiệu quả, chẳng hạn như: điều chỉnh mức chiết khấu ngay tại thời điểm nhập đơn hàng, ưu tiên xử lý các giao dịch có tiềm năng sinh lời cao hoặc cảnh báo nhân viên bán hàng về những giao dịch có nguy cơ thua lỗ để kịp thời can thiệp (thay đổi sản phẩm, giảm chiết khấu hoặc đề xuất phương thức vận chuyển kinh tế hơn).

Để chuyển hóa vấn đề kinh doanh này thành bài toán học máy cụ thể, nghiên cứu thực hiện chuyển đổi biến Profit (liên tục) thành biến nhị phân Is_Profit theo quy tắc:

- $Is_Profit = 1$ (có lợi nhuận) nếu $Profit > 0$
- $Is_Profit = 0$ (thua lỗ) nếu $Profit \leq 0$

Việc lựa chọn bài toán phân loại nhị phân thay vì bài toán hồi quy (dự báo giá trị lợi nhuận cụ thể) được đưa ra dựa trên nhiều lý do mang tính thực tiễn và phương pháp luận.

Về mặt diễn giải và ứng dụng, nhà quản trị thường quan tâm nhiều hơn đến câu hỏi nhị phân ***“giao dịch này có lãi hay không?”*** thay vì cần biết chính xác mức lãi là bao nhiêu. Một dự báo dạng nhị phân dễ hiểu, dễ truyền đạt đến các bên liên quan và dễ dàng tích hợp vào quy trình vận hành hàng ngày. Nhân viên bán hàng có thể nhận được cảnh báo tức thời từ hệ thống khi một đơn hàng được dự báo có nguy cơ thua lỗ, từ đó chủ động điều chỉnh chính sách chiết khấu, tư vấn sản phẩm thay thế hoặc thương lượng với khách hàng.

Ngoài ra, bài toán phân loại cho phép áp dụng hiệu quả các kỹ thuật xử lý mất cân bằng lớp chuyên biệt, điển hình là SMOTE - phương pháp tạo mẫu nhân tạo cho lớp thiểu số thông qua nội suy tuyến tính giữa các mẫu lân cận. Kỹ thuật này giúp mô hình học được đặc trưng của cả hai lớp một cách cân bằng hơn, giảm thiểu hiện tượng thiên vị về lớp đa số.

Cuối cùng, bài toán phân loại nhị phân tận dụng tối đa các phát hiện từ quá trình EDA: các mối quan hệ phi tuyến giữa Discount và Profit theo từng Category, sự khác biệt giữa các phân khúc khách hàng (Segment), ảnh hưởng của phương thức vận chuyển (Ship Mode) và thời gian vận chuyển (Ship_Duration) đến khả năng sinh lời. Tất cả những yếu tố này đều có thể được các mô hình phân loại học hỏi và sử dụng để nâng cao chất lượng dự báo.

Tóm lại, bài toán phân loại nhị phân “dự báo Is_Profit” được xác định là trọng tâm nghiên cứu. Nó không chỉ giải quyết trực tiếp vấn đề kinh doanh cốt lõi mà còn khai thác triệt để các hiểu biết đã có về dữ liệu, đồng thời đảm bảo tính khả thi khi triển khai thực tế và mức độ giải thích cao.

2.2. Mô hình được chọn và lý do lựa chọn

Để giải quyết bài toán phân loại nhị phân đã nêu, nghiên cứu lựa chọn bốn mô hình học máy đại diện cho các nhóm phương pháp từ đơn giản đến phức tạp, từ mô hình tuyến tính đến các kỹ thuật tập hợp. Việc lựa chọn dựa trên ba tiêu chí chính:

1. Khả năng xử lý các đặc trưng và mối quan hệ đã được phát hiện qua EDA (bao gồm tính phi tuyến, ngoại lai và mất cân bằng lớp),
2. Yêu cầu về tính giải thích trong bối cảnh ứng dụng kinh doanh,
3. Hiệu suất dự báo và tính ổn định của mô hình được đánh giá thông qua kiểm chứng chéo và tập kiểm tra độc lập.

Bốn mô hình được lựa chọn là: Logistic Regression, Decision Tree, Random Forest và Gradient Boosting. Mỗi mô hình đại diện cho một cấp độ khác nhau về độ phức tạp và khả năng mô hình hóa, tạo điều kiện cho việc so sánh toàn diện và lựa chọn phương án phù hợp nhất.

2.2.1. Hồi quy Logistic (*Logistic Regression*)

Hồi quy Logistic là mô hình nền tảng và phổ biến nhất cho các bài toán phân loại nhị phân. Về mặt bản chất, mô hình này vẫn mang tính tuyến tính, nhưng sử dụng hàm sigmoid để ánh xạ đầu ra của một tổ hợp tuyến tính vào khoảng $(0,1)$, biểu diễn xác suất một quan sát thuộc về lớp dương (có lợi nhuận). Công thức của mô hình được biểu diễn như sau:

$$P(y = 1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}}$$

Trong đó, các hệ số β_j thể hiện mức độ và chiều hướng ảnh hưởng của từng đặc trưng x_j lên log-odds của xác suất có lợi nhuận. Một hệ số dương có nghĩa là đặc trưng đó làm tăng xác suất có lợi nhuận và ngược lại.

Lý do lựa chọn Hồi quy Logistic xuất phát từ khả năng giải thích trực tiếp của nó. Hệ số hồi quy cho biết một cách trực quan: “khi đặc trưng X tăng lên một đơn vị, log-odds của xác suất có lợi nhuận thay đổi một lượng là β ”. Từ đó, nhà quản trị có thể ước lượng được mức độ ảnh hưởng của các yếu tố như doanh số, chiết khấu, danh mục sản phẩm đến khả năng sinh lời. Ví dụ, nếu hệ số của Discount là âm, điều đó khẳng định chiết khấu cao làm giảm xác suất có lợi nhuận; nếu hệ số của Category_Technology dương, điều đó cho thấy sản phẩm công nghệ có xu hướng sinh lời tốt hơn so với danh mục tham chiếu.

Bên cạnh đó, Hồi quy Logistic huấn luyện rất nhanh và ổn định. Với bộ dữ liệu có kích thước vừa phải (khoảng 10.000 quan sát và sau mã hóa khoảng 20-30 đặc trưng), mô hình hội tụ nhanh chóng, không yêu cầu nhiều tài nguyên tính toán. Điều này rất phù hợp cho bước thử nghiệm ban đầu và làm cơ sở để so sánh.

Hồi quy Logistic cũng đóng vai trò là đường cơ sở để đánh giá mức độ cải thiện của các mô hình phức tạp hơn. Nếu một mô hình phi tuyến như Random Forest hay Gradient Boosting chỉ cải thiện không đáng kể so với Logistic Regression, thì có thể lựa chọn mô hình đơn giản hơn vì tính dễ giải thích và hiệu quả tính toán. Ngược lại, nếu sự cải thiện là đáng kể, điều đó chứng tỏ dữ liệu có cấu trúc phi tuyến phức tạp mà mô hình tuyến tính không thể nắm bắt được.

Hơn nữa, Hồi quy Logistic phù hợp với dữ liệu đã được chuẩn hóa và mã hóa. Trong quy trình tiền xử lý, các biến định lượng được chuẩn hóa về khoảng $[0,1]$ bằng MinMaxScaler, các biến định tính được mã hóa One-Hot. Khi đó, các hệ số hồi quy có thể so sánh trực tiếp với nhau về mức độ ảnh hưởng, giúp xác định các yếu tố quan trọng nhất.

Tuy nhiên, Hồi quy Logistic có hạn chế lớn là không thể nắm bắt các mối quan hệ phi tuyến phức tạp giữa các biến. Như đã thấy trong phân tích scatter giữa Discount và Profit theo Category, tác động của chiết khấu lên lợi nhuận không đồng nhất mà phụ thuộc vào danh mục sản phẩm. Một mô hình tuyến tính thuần túy không thể học được sự tương tác này một cách tự nhiên, trừ khi người phân tích tạo thủ công các biến tương tác (ví dụ: $\text{Discount} \times \text{Category_Technology}$). Do đó, Hồi quy Logistic có thể không đạt hiệu suất cao bằng các mô hình phi tuyến.

2.2.2. Decision Tree

Decision Tree là một mô hình phi tuyến, hoạt động bằng cách xây dựng một cấu trúc cây dựa trên các quy tắc if-else. Tại mỗi nút của cây, thuật toán tìm kiếm đặc trưng và ngưỡng phân chia tốt nhất để tách dữ liệu thành hai nhóm con thuần nhất nhất có thể, dựa trên các độ đo như Gini impurity hoặc Information Gain. Quá trình này tiếp diễn cho đến khi đạt được điều kiện dừng (ví dụ: độ sâu tối đa, số lượng mẫu tối thiểu tại nút lá). Các nút lá sẽ gán nhãn dự đoán là lớp chiếm đa số trong nhóm.

Lý do lựa chọn Decision Tree xuất phát từ khả năng xử lý phi tuyến một cách tự nhiên. Như đã thấy trong biểu đồ phân tán Discount vs Profit, mối quan hệ giữa chiết khấu và lợi nhuận không chỉ đơn thuần là tuyến tính, mà còn phụ thuộc rất lớn vào danh mục sản phẩm. Decision Tree có thể học được các quy tắc như: “Nếu Category = Furniture và Discount > 0.2 thì dự đoán lỗ”, hoặc “Nếu Category = Technology và Discount ≤ 0.5 thì dự đoán lời”. Những quy tắc này không chỉ có giá trị dự báo mà còn rất dễ hiểu và dễ giải thích cho nhà quản trị.

Bên cạnh đó, Decision Tree không yêu cầu chuẩn hóa dữ liệu. Do chỉ dựa trên các ngưỡng phân chia, thang đo của các biến không ảnh hưởng đến kết quả. Điều này giúp đơn giản hóa quy trình tiền xử lý, mặc dù trong nghiên cứu này vẫn thực hiện chuẩn hóa để nhất quán giữa các mô hình.

Decision Tree cũng có thể được trực quan hóa một cách dễ dàng. Một cây với độ sâu vừa phải có thể được vẽ ra và trình bày trực tiếp cho ban lãnh đạo, giúp họ hiểu được quy trình ra quyết định của mô hình. Điều này đặc biệt quan trọng trong các ứng dụng kinh doanh, nơi yêu cầu về tính minh bạch và khả năng giải thích thường cao hơn so với các ứng dụng kỹ thuật thuần túy.

Tuy nhiên, Decision Tree có nhược điểm là dễ bị overfitting, tức là học quá kỹ dữ liệu huấn luyện đến mức không thể tổng quát hóa tốt trên dữ liệu mới. Để khắc phục, trong nghiên cứu này, Decision Tree được giới hạn với các siêu tham số: $\text{max_depth} = 5$ (độ sâu tối đa) và $\text{min_samples_split} = 20$ (số lượng mẫu tối thiểu để phân chia). Những ràng buộc này giúp cây không quá phức tạp, cân bằng giữa khả năng học các mẫu quan trọng và khả năng tổng quát hóa.

2.2.3. *Random Forest*

Random Forest là một kỹ thuật ensemble (tập hợp) mạnh mẽ, xây dựng nhiều Decision Tree và kết hợp kết quả của chúng. Cơ chế hoạt động của Random Forest dựa trên hai nguyên lý chính. Thứ nhất là Bagging (Bootstrap Aggregating): mô hình tạo ra nhiều mẫu bootstrap (lấy mẫu có hoàn lại) từ dữ liệu gốc, mỗi Decision Tree được huấn luyện trên một mẫu bootstrap khác nhau. Thứ hai là Random Feature Selection: tại mỗi nút của cây, chỉ một tập con ngẫu nhiên các đặc trưng được xem xét để phân chia, thay vì tất cả các đặc trưng. Điều này làm giảm sự tương quan giữa các cây, tăng tính đa dạng cho tổng thể. Kết quả cuối cùng được xác định bằng bình chọn đa số từ tất cả các cây.

Lý do lựa chọn Random Forest đến từ khả năng giảm overfitting đáng kể so với một Decision Tree đơn lẻ. Mặc dù mỗi cây riêng lẻ có thể học quá kỹ dữ liệu của mẫu bootstrap của nó, nhưng khi kết hợp nhiều cây, các sai lệch có xu hướng triệt tiêu lẫn nhau, tạo ra mô hình tổng thể ổn định và có khả năng tổng quát hóa tốt.

Random Forest cũng thường đạt độ chính xác cao trên dữ liệu dạng bảng như Sample Superstore. Trong nhiều bài toán phân loại thực tế, Random Forest là lựa chọn hàng đầu nhờ khả năng xử lý tốt các mối quan hệ phi tuyến và tương tác phức tạp giữa các biến.

Một ưu điểm quan trọng khác là Random Forest cung cấp chỉ số Feature Importance, là một thước đo về mức độ đóng góp của từng đặc trưng vào kết quả dự đoán. Có nhiều cách tính Feature Importance, phổ biến nhất là dựa trên mức độ giảm độ tạp (Gini impurity) khi một đặc trưng được sử dụng để phân chia. Chỉ số này có giá trị vô cùng lớn trong bối cảnh kinh doanh: nó giúp xác định chính xác các yếu tố then chốt ảnh hưởng đến lợi nhuận, từ đó tập trung nguồn lực cải thiện. Ví dụ, nếu Sales nổi bật nhất thì siêu thị nên tập trung vào các giao dịch có doanh số cao; nếu Discount_Bin quan trọng, cần kiểm soát chiết khấu chặt chẽ hơn; nếu Category quan trọng, cần có chiến lược riêng cho từng danh mục.

Random Forest cũng có khả năng xử lý tốt dữ liệu mất cân bằng, ngay cả trước khi áp dụng SMOTE. Cơ chế bình chọn đa số giúp giảm bớt thiên vị về lớp đa số, mặc dù trong nghiên cứu này vẫn áp dụng SMOTE để tối ưu hơn. Mô hình được thiết lập với $n_estimators = 100$ (số lượng cây) và $max_depth = 6$ (độ sâu tối đa). Các tham số này được chọn để cân bằng giữa thời gian huấn luyện và hiệu suất dự báo, đồng thời tránh overfitting.

2.2.4. Gradient Boosting

Gradient Boosting là một kỹ thuật ensemble khác, xây dựng các mô hình một cách tuần tự. Trong khi Random Forest xây dựng các cây độc lập và kết hợp chúng bằng bình chọn, Gradient Boosting xây dựng từng cây mới để sửa lỗi của tổ hợp các cây trước đó. Cơ chế hoạt động của Gradient Boosting có thể được tóm tắt như sau: mô hình bắt đầu với một dự đoán ban đầu (thường là giá trị trung bình của log-odds); tại mỗi bước lặp, một Decision Tree mới được huấn luyện để dự đoán phần dư, tức là sai số của tổ hợp các cây hiện tại; cây mới sau đó được thêm vào mô hình với một trọng số học (learning rate) nhỏ, giúp cải thiện dần dần; quá trình này lặp lại cho đến khi đạt được số lượng cây mong muốn hoặc khi phần dư không còn giảm đáng kể.

Lý do lựa chọn Gradient Boosting xuất phát từ hiệu suất dự báo vượt trội. Trong nhiều cuộc thi khoa học dữ liệu và các ứng dụng thực tế, Gradient Boosting (cùng với các biến thể hiện đại như XGBoost, LightGBM, CatBoost) thường đạt kết quả cao nhất trên dữ liệu dạng bảng. Điều này đến từ khả năng tối ưu hóa mạnh mẽ của thuật toán, cũng như tính linh hoạt trong việc điều chỉnh siêu tham số.

Gradient Boosting có khả năng nắm bắt các tương tác phức tạp giữa các biến. Trong bộ dữ liệu Sample Superstore, có nhiều mối quan hệ không chỉ đơn thuần là phi tuyến mà còn mang tính tương tác cao, chẳng hạn như tác động của thời gian vận chuyển (Ship_Duration) có thể khác nhau giữa các khu vực (Region) hoặc ảnh hưởng của chiết khấu có thể phụ thuộc vào cả danh mục sản phẩm và phân khúc khách hàng. Gradient Boosting, với cơ chế xây dựng cây tuần tự và tối ưu hóa gradient, có thể học được những tương tác này một cách tự nhiên.

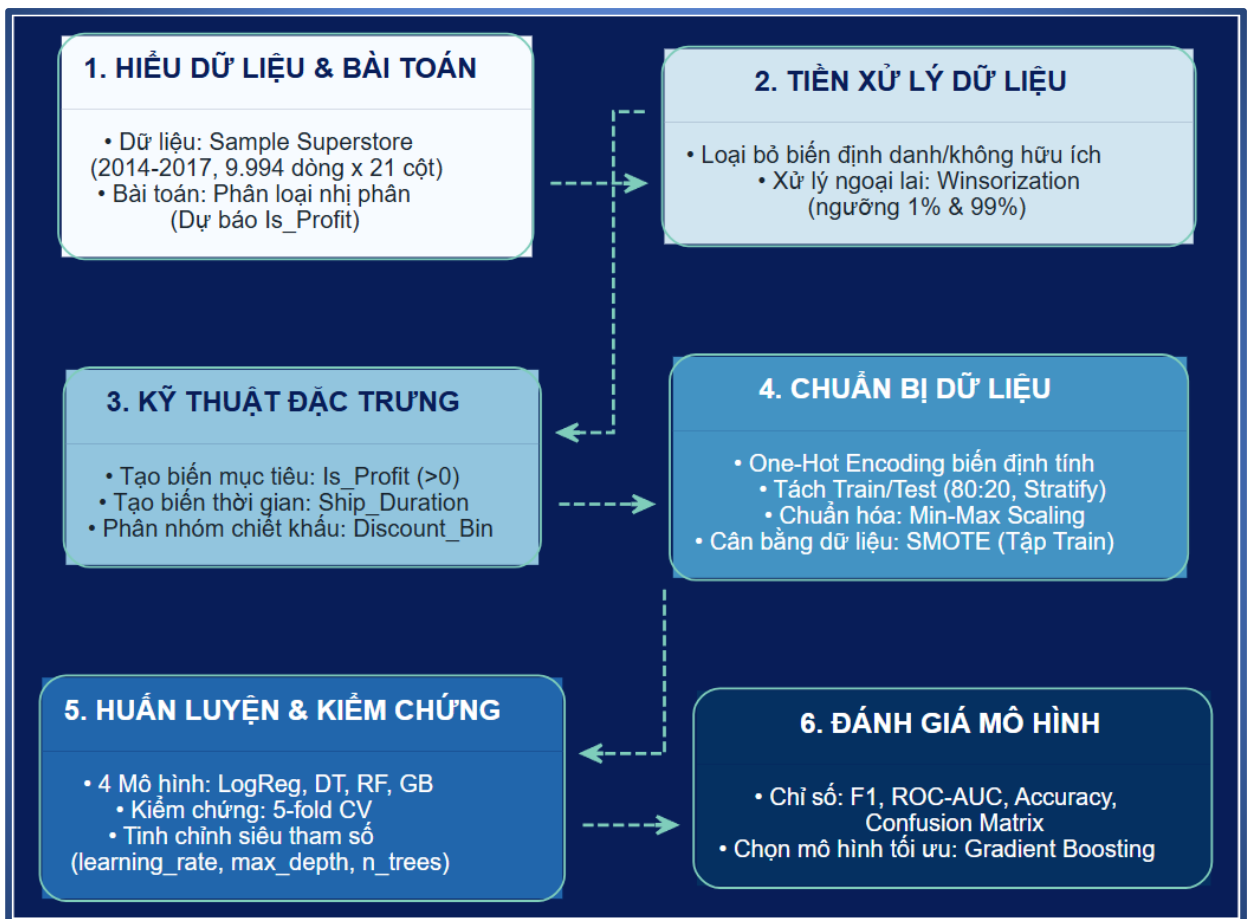
Mô hình này cũng rất linh hoạt trong việc điều chỉnh siêu tham số. Bằng cách thay đổi learning rate (tốc độ học), số lượng cây ($n_estimators$) và độ sâu của cây (max_depth), người phân tích có thể kiểm soát được sự cân bằng giữa tốc độ hội tụ và nguy cơ overfitting. Trong nghiên cứu này, mô hình được thiết lập với $n_estimators = 100$, $max_depth = 3$, và learning rate mặc định là 0.1. Các tham số này được chọn để mô hình có đủ độ phức tạp để học các mẫu quan trọng, nhưng vẫn đảm bảo khả năng tổng quát hóa.

Tuy nhiên, Gradient Boosting có hạn chế là dễ bị overfitting hơn Random Forest nếu không được điều chỉnh cẩn thận. Do các cây được xây dựng tuần tự và mỗi cây sau đều

cố gắng sửa lỗi của các cây trước, mô hình có thể trở nên quá phức tạp và học cả nhiễu trong dữ liệu. Thời gian huấn luyện cũng lâu hơn do tính chất tuần tự, không thể song song hóa hoàn toàn như Random Forest. Tuy nhiên, với kích thước dữ liệu vừa phải (khoảng 10.000 quan sát), đây vẫn là lựa chọn khả thi.

Như vậy, bốn mô hình được lựa chọn tạo thành một hệ thống so sánh đa chiều: từ mô hình tuyến tính (Hồi quy Logistic) đến mô hình phi tuyến đơn lẻ (Decision Tree); từ mô hình đơn lẻ đến mô hình tập hợp (Random Forest và Gradient Boosting); từ tập hợp ngẫu nhiên song song (Random Forest) đến tập hợp tăng cường tuần tự (Gradient Boosting). Cách tiếp cận này đảm bảo kết quả nghiên cứu không chỉ dừng lại ở việc chọn một mô hình duy nhất, mà còn cung cấp sự đánh giá toàn diện, từ đó đưa ra khuyến nghị phù hợp nhất với các tiêu chí về độ chính xác, khả năng giải thích và nguồn lực tính toán trong bối cảnh ứng dụng thực tiễn.

2.3. Sơ đồ quy trình thực hiện (Pipeline)



Hình 2. 1. Sơ đồ quy trình thực hiện

PHẦN III. XỬ LÝ DỮ LIỆU

Chất lượng của các mô hình học máy phụ thuộc rất lớn vào chất lượng của dữ liệu đầu vào. Nếu dữ liệu chứa nhiều sai sót, không nhất quán hoặc thiếu sót, thì mọi kết quả phân tích đều có thể bị ảnh hưởng nghiêm trọng. Chính vì vậy, giai đoạn tiền xử lý đóng vai trò nền tảng, quyết định độ tin cậy của toàn bộ nghiên cứu. Dựa trên những phát hiện từ quá trình khám phá dữ liệu, nghiên cứu xác định một số vấn đề cần được xử lý trước khi xây dựng mô hình, bao gồm: kiểm tra giá trị thiếu, xử lý trùng lặp, đảm bảo tính hợp lý của dữ liệu, phát hiện và xử lý ngoại lai, tạo biến mới, mã hóa biến phân loại, chuẩn hóa dữ liệu và xử lý mất cân bằng lớp.

3.1. Xử lý dữ liệu rỗng (Missing Values)

Việc kiểm tra và xử lý giá trị thiếu là bước đầu tiên và quan trọng nhất trong quy trình tiền xử lý, bởi sự tồn tại của các ô dữ liệu trống có thể gây ra lỗi trong quá trình tính toán hoặc làm sai lệch các ước lượng thống kê.

Trong bộ dữ liệu Sample Superstore, chúng ta tiến hành kiểm tra toàn bộ 21 biến để xác định số lượng giá trị thiếu ở từng cột. Kết quả kiểm tra cho thấy không có bất kỳ giá trị thiếu nào ở tất cả các biến, từ các biến định danh như Row ID, Order ID cho đến các biến định lượng quan trọng như Sales, Quantity, Discount, Profit. Điều này có nghĩa là dữ liệu đã được ghi nhận đầy đủ và không cần thực hiện các phương pháp điền giá trị như thay thế bằng trung bình, trung vị, hoặc sử dụng mô hình dự đoán. Tính toàn vẹn này là một lợi thế lớn, giúp đơn giản hóa quy trình tiền xử lý và đảm bảo rằng các thống kê mô tả phản ánh đúng bản chất của dữ liệu thực tế.

```
# Kiểm tra missing values
print("KIỂM TRA GIÁ TRỊ THIẾU:")
print(df.isnull().sum())

print("\nTổng giá trị thiếu trong toàn bộ dữ liệu:",
df.isnull().sum().sum())

if df.isnull().sum().sum() == 0:
    print("Không có giá trị thiếu, dữ liệu đã sạch.")
else:
    print("Có giá trị thiếu, cần xử lý.")
```

KIỂM TRA GIÁ TRỊ THIẾU:	
Row ID	0
Order ID	0
Order Date	0
Ship Date	0
Ship Mode	0
Customer ID	0
Customer Name	0
Segment	0
Country	0
City	0
State	0
Postal Code	0
Region	0
Product ID	0
Category	0
Sub-Category	0
Product Name	0
Sales	0
Quantity	0
Discount	0
Profit	0
Is_Profit	0
Ship_Duration	0
Discount_Bin	0
dtype: int64	
Tổng giá trị thiếu trong toàn bộ dữ liệu: 0	
Không có giá trị thiếu, dữ liệu đã sạch.	

Hình 3. 1. Tổng hợp giá trị thiếu theo từng biến

Mặc dù không có giá trị thiếu, việc kiểm tra này vẫn được thực hiện để xác nhận chất lượng dữ liệu đầu vào. Kết quả kiểm tra cũng cho thấy các biến định lượng có đầy đủ 9.994 quan sát, đảm bảo rằng không có sự mất mát thông tin nào trong quá trình thu thập. Đây là cơ sở vững chắc để tiến hành các bước xử lý tiếp theo mà không lo ngại về vấn đề dữ liệu khuyết.

3.2. Xử lý dữ liệu trùng lặp

Dữ liệu trùng lặp là các bản ghi giống hệt nhau trên tất cả các cột, có thể phát sinh do lỗi nhập liệu, quá trình tổng hợp dữ liệu không chính xác, hoặc do nhiều nguồn dữ liệu khác nhau được gộp lại. Sự tồn tại của các bản ghi trùng lặp có thể làm thổi phồng một cách giả tạo các chỉ số thống kê như tổng doanh số, số lượng bán, hoặc tạo trọng số không mong muốn cho các mẫu đó trong quá trình huấn luyện mô hình. Ngoài ra, dữ liệu trùng lặp còn có thể làm tăng phương sai của các ước lượng và gây nhiễu cho quá trình học.

```
# Xử lý dữ liệu trùng lặp
print("Số dòng trước khi xử lý:", df.shape[0])

df = df.drop_duplicates()
print("Số dòng sau khi xóa trùng lặp:", df.shape[0])
```

Số dòng trước khi xử lý: 9994
Số dòng sau khi xóa trùng lặp: 9994

Hình 3. 2. Số lượng dòng trước và sau khi xóa trùng lặp

Kết quả kiểm tra cho thấy không có dòng dữ liệu nào bị trùng lặp hoàn toàn trên tất cả các cột. Số lượng dòng trước và sau khi thực hiện lệnh `drop_duplicates()` đều là 9.994, chứng tỏ mỗi giao dịch là duy nhất và không có sự trùng lặp nào. Điều này phản ánh tính đa dạng của dữ liệu bán hàng, đồng thời khẳng định rằng quá trình thu thập và lưu trữ dữ liệu đã được thực hiện một cách cẩn thận. Việc không phát hiện dữ liệu trùng lặp cũng giúp chúng ta tiết kiệm thời gian và công sức, đồng thời đảm bảo rằng các thống kê mô tả không bị sai lệch do trọng số của các bản ghi trùng lặp.

3.3. Kiểm tra tính hợp lý

Sau khi xác nhận không có giá trị thiếu và dữ liệu trùng lặp, bước tiếp theo là kiểm tra tính hợp lý của dữ liệu nhằm phát hiện các giá trị bất thường có thể do lỗi nhập liệu hoặc sai sót trong quá trình thu thập. Các kiểm tra được thực hiện bao gồm: doanh số (Sales) không được âm, vì doanh số âm là giá trị không thể xảy ra trong thực tế; số lượng sản phẩm (Quantity) phải là số dương, vì không thể có đơn hàng với số lượng bằng 0 hoặc âm; chiết khấu (Discount) phải nằm trong khoảng từ 0 đến 1, tương ứng với 0% đến 100%, vì chiết khấu ngoài khoảng này là không hợp lý; và ngày giao hàng (Ship Date) không được sớm hơn ngày đặt hàng (Order Date), vì điều này vi phạm logic thời gian.

```
print("---- KIỂM TRA TÍNH HỢP LÝ CỦA DỮ LIỆU ----")

# Kiểm tra các giá trị âm không hợp lý
negative_sales = df[df['Sales'] < 0].shape[0]
negative_qty = df[df['Quantity'] <= 0].shape[0]
invalid_discount = df[(df['Discount'] < 0) | (df['Discount'] > 1)].shape[0]

# Kiểm tra logic ngày tháng (Ngày giao phải sau hoặc bằng ngày đặt)
df['Order Date'] = pd.to_datetime(df['Order Date'])
df['Ship Date'] = pd.to_datetime(df['Ship Date'])
date_error = df[df['Ship Date'] < df['Order Date']].shape[0]

check = pd.DataFrame({
    "Chi số kiểm tra": [
        "Doanh số (Sales) bị âm",
        "Số lượng (Quantity) <= 0",
        "Chiết khấu (Discount) ngoài khoảng [0, 1]",
        "Ngày giao hàng trước ngày đặt hàng"
    ],
    "Số lỗi phát hiện": [negative_sales, negative_qty,
invalid_discount, date_error],
    "Trạng thái": ["Hợp lệ" if x == 0 else "Cần xử lý" for x in
[negative_sales, negative_qty, invalid_discount, date_error]]
})

display(check)
print("Số dòng sau khi kiểm tra tính hợp lý:", df.shape[0])
```

--- KIỂM TRA TÍNH HỢP LÝ CỦA DỮ LIỆU ---			
	Chỉ số kiểm tra	Số lỗi phát hiện	Trạng thái
0	Doanh số (Sales) bị âm	0	Hợp lệ
1	Số lượng (Quantity) <= 0	0	Hợp lệ
2	Chiết khấu (Discount) ngoài khoảng [0, 1]	0	Hợp lệ
3	Ngày giao hàng trước ngày đặt hàng	0	Hợp lệ
Số dòng sau khi kiểm tra tính hợp lý: 9994			

Hình 3. 3. Kiểm tra tính hợp lệ của dữ liệu

Kết quả kiểm tra cho thấy tất cả các giá trị đều thỏa mãn các điều kiện trên, không phát hiện bất kỳ lỗi logic nào. Cụ thể, không có giao dịch nào có doanh số âm, không có giao dịch nào có số lượng bằng 0 hoặc âm, không có chiết khấu nào nằm ngoài khoảng [0,1], và không có trường hợp nào ngày giao hàng sớm hơn ngày đặt hàng. Điều này khẳng định dữ liệu đã được ghi nhận một cách hợp lý và có thể sử dụng trực tiếp cho các bước phân tích tiếp theo mà không cần điều chỉnh thủ công.

```
# Thống kê các giá trị số để xem có gì bất thường (Outliers cục đoạn)
print("\n Tóm tắt thống kê các biến số học:")
display(df[['Sales', 'Quantity', 'Discount', 'Profit']].describe().T)
```

Tóm tắt thống kê các biến số học:								
	count	mean	std	min	25%	50%	75%	max
Sales	9994.0	229.858001	623.245101	0.444	17.28000	54.4900	209.940	22638.480
Quantity	9994.0	3.789574	2.225110	1.000	2.00000	3.0000	5.000	14.000
Discount	9994.0	0.156203	0.206452	0.000	0.00000	0.2000	0.200	0.800
Profit	9994.0	28.656896	234.260108	-6599.978	1.72875	8.6665	29.364	8399.976

Hình 3. 4. Thông tin thống kê cơ bản của Sales, Quantity, Discount và Profit

Tuy nhiên, khi xem xét bảng thống kê mô tả, chúng ta nhận thấy sự chênh lệch lớn giữa giá trị trung bình và trung vị của các biến Sales và Profit, cùng với khoảng biến thiên rất rộng (từ giá trị âm sâu đến giá trị dương rất cao). Cụ thể, Sales có giá trị trung bình 229,86 USD nhưng trung vị chỉ 54,49 USD, và có giao dịch lên đến 22.638,48 USD. Tương tự, Profit có giá trị trung bình 28,66 USD nhưng trung vị chỉ 8,67 USD, với mức lỗ sâu -6.599,98 USD và lãi cao 8.399,98 USD. Những con số này cho thấy sự tồn tại của các giá trị ngoại lai (outliers) cần được xử lý ở bước sau.

3.4. Feature Engineering

Trước khi tiến hành xây dựng biến mục tiêu và các đặc trưng mới, cần đảm bảo các biến thời gian được xử lý đúng định dạng để có thể tính toán chênh lệch. Hai biến Order Date và Ship Date trong bộ dữ liệu ban đầu có kiểu dữ liệu chuỗi (object), do đó cần được

chuyển đổi sang định dạng datetime để có thể thực hiện các phép toán ngày tháng. Việc chuyển đổi này được thực hiện bằng phương thức `pd.to_datetime()`, giúp chuẩn hóa định dạng và cho phép tính toán khoảng cách thời gian giữa hai mốc.

Sau khi chuyển đổi, biến `Ship_Duration` được tạo ra bằng cách lấy hiệu giữa ngày giao hàng và ngày đặt hàng, tính theo đơn vị ngày. Biến này phản ánh thời gian vận chuyển, một yếu tố có thể ảnh hưởng đến trải nghiệm khách hàng và chi phí logistics. Những giao dịch có thời gian vận chuyển dài có thể phát sinh chi phí lưu kho cao hơn, trong khi các phương thức vận chuyển nhanh thường đi kèm với chi phí dịch vụ cao hơn, từ đó tác động đến lợi nhuận. Việc đưa biến này vào mô hình giúp nắm bắt được ảnh hưởng của hiệu quả vận chuyển đến khả năng sinh lời..

```
print("--- FEATURE ENGINEERING: SHIP DURATION ---")

# Thông tin trước khi xử lý
print(f"Số dòng ban đầu: {df.shape[0]}")

# Chuyển đổi kiểu ngày
df['Order Date'] = pd.to_datetime(df['Order Date'])
df['Ship Date'] = pd.to_datetime(df['Ship Date'])

# Tạo Ship_Duration
df['Ship_Duration'] = (df['Ship Date'] - df['Order Date']).dt.days

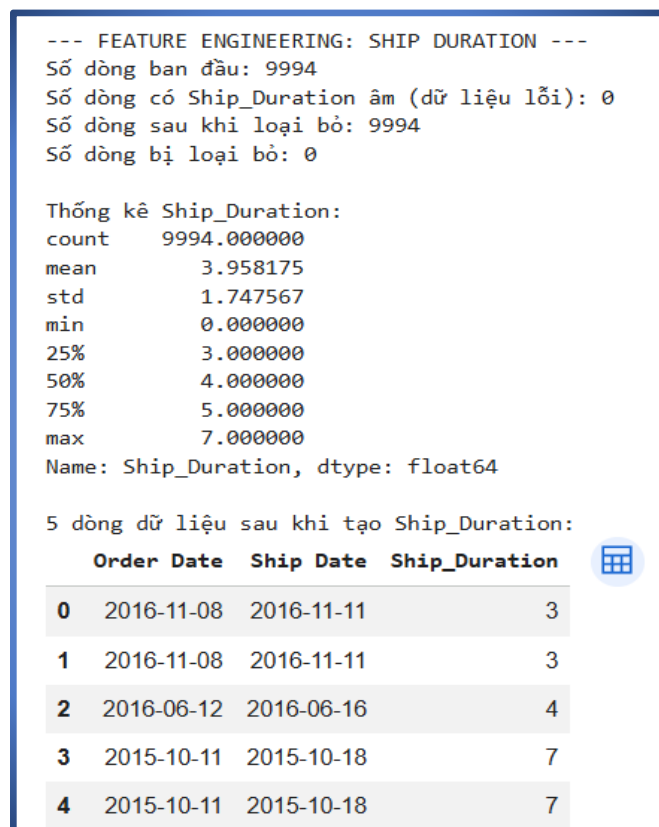
# Kiểm tra dữ liệu lỗi
invalid_rows = df[df['Ship_Duration'] < 0].shape[0]
print(f"Số dòng có Ship_Duration âm (dữ liệu lỗi): {invalid_rows}")

# Loại bỏ dữ liệu lỗi
df = df[df['Ship_Duration'] >= 0]

# Thông tin sau xử lý
print(f"Số dòng sau khi loại bỏ: {df.shape[0]}")
print(f"Số dòng bị loại bỏ: {invalid_rows}")

# Thống kê Ship_Duration
print("\nThống kê Ship_Duration:")
print(df['Ship_Duration'].describe())

# Hiển thị mẫu dữ liệu
print("\n5 dòng dữ liệu sau khi tạo Ship_Duration:")
display(df[['Order Date', 'Ship Date', 'Ship_Duration']].head())
```



Hình 3. 5. Kết quả feature engineering biến thời gian giao hàng (Ship_Duration)

Kết quả kiểm tra cho thấy không có dòng nào có Ship_Duration âm (giá trị âm có nghĩa là ngày giao hàng sớm hơn ngày đặt hàng, vi phạm logic thời gian). Điều này khẳng định dữ liệu đã được ghi nhận một cách hợp lý. Sau khi loại bỏ các dòng lỗi (nếu có), số lượng quan sát vẫn được giữ nguyên. Thống kê mô tả của Ship_Duration cho thấy thời gian vận chuyển trung bình khoảng 4 ngày, với giá trị nhỏ nhất là 0 (giao hàng cùng ngày) và lớn nhất lên đến 7 ngày. Đây là thông tin hữu ích để đánh giá hiệu suất của các phương thức vận chuyển.

3.5. Xử lý giá trị ngoại lai (Outliers)

Như đã phát hiện từ bảng thống kê mô tả và biểu đồ ở Phần I, các biến Sales và Profit có nhiều giá trị ngoại lai ở cả hai phía. Cụ thể, Sales có các điểm ngoại lai với giá trị từ khoảng 1.000 USD đến hơn 22.000 USD, trong khi Profit có các điểm ngoại lai âm sâu xuống đến -6.600 USD và các điểm dương cao lên đến 8.400 USD. Việc giữ nguyên các giá trị này có thể làm lệch các ước lượng thống kê và khiến mô hình học máy bị "kéo" về phía các giá trị cực đoan, dẫn đến giảm khả năng tổng quát hóa. Tuy nhiên, loại bỏ hoàn toàn các outlier cũng có thể làm mất đi thông tin quan trọng về các giao dịch đặc biệt, chẳng hạn như các đơn hàng lớn hoặc các khoản lỗ sâu cần được phân tích để rút ra bài học.

Để cân bằng giữa việc giảm ảnh hưởng của outlier và bảo toàn thông tin, nghiên cứu áp dụng phương pháp Winsorization (chặn giá trị).

- **Đối với Sales, chúng ta chặn phía trên tại percentile 99%**, nghĩa là tất cả các giá trị vượt quá ngưỡng này sẽ được thay thế bằng chính giá trị tại percentile 99%. Ngưỡng được xác định là 2.481,69 USD. Cách làm này giúp loại bỏ ảnh hưởng của những giao dịch có doanh số cực cao (trên 2.481 USD) nhưng vẫn giữ lại phần lớn cấu trúc phân phối. Việc chọn percentile 99% thay vì các ngưỡng khác dựa trên sự cân bằng giữa việc giảm nhiễu và giữ lại thông tin, đảm bảo rằng chỉ những giá trị thực sự cực đoan mới bị điều chỉnh.

- **Đối với Profit, chúng ta chặn cả hai phía:** chặn dưới tại percentile 1% (giá trị -319,26 USD) và chặn trên tại percentile 99% (giá trị 580,66 USD). Mọi giá trị Profit nhỏ hơn -319,26 USD được thay thế bằng -319,26 USD, và các giá trị lớn hơn 580,66 USD được thay thế bằng 580,66 USD. Việc chặn cả hai phía là cần thiết vì Profit có các outlier ở cả vùng lỗ sâu và lãi cao. Ngưỡng percentile 1% và 99% được chọn để loại bỏ ảnh hưởng của các giá trị cực đoan ở cả hai đầu phân phối.

```
def xu_ly_ngoai_lai(df):
    print("XỬ LÝ NGOẠI LAI BẰNG WINSORIZATION")
    print(f"Số dòng ban đầu: {df.shape[0]}")

    df_new = df.copy()

    # Sales: chỉ chặn trên (99%)
    sales_upper = df_new['Sales'].quantile(0.99)
    df_new['Sales'] = df_new['Sales'].clip(upper=sales_upper)
    print(f"- Sales: giới hạn trên = {sales_upper:.2f}")

    # Profit: chặn cả hai đầu (1% và 99%)
    profit_lower = df_new['Profit'].quantile(0.01)
    profit_upper = df_new['Profit'].quantile(0.99)
    df_new['Profit'] = df_new['Profit'].clip(lower=profit_lower,
    upper=profit_upper)
    print(f"- Profit: giới hạn dưới = {profit_lower:.2f} | trên = {profit_upper:.2f}")

    print(f"Số dòng sau xử lý: {df_new.shape[0]}")
    return df_new

df_clean = xu_ly_ngoai_lai(df)
```

```
XỬ LÝ NGOẠI LAI BẰNG WINSORIZATION
Số dòng ban đầu: 9994
- Sales: giới hạn trên = 2481.69
- Profit: giới hạn dưới = -319.26 | trên = 580.66
Số dòng sau xử lý: 9994
```

Hình 3. 6. Xử lý ngoại lai bằng phương pháp Winsorization

Kết quả sau Winsorization, số lượng quan sát vẫn được giữ nguyên (9.994 dòng), nhưng các giá trị cực đoan đã được điều chỉnh về ngưỡng hợp lý. Điều này giúp giảm thiểu ảnh hưởng của outlier đối với mô hình, đồng thời vẫn bảo toàn được thông tin từ các giao dịch có giá trị cao nhưng không quá bất thường. Việc chọn ngưỡng percentile 1% và 99% thay vì các ngưỡng khác dựa trên sự cân bằng giữa việc giảm nhiễu và giữ lại thông tin, đảm bảo rằng chỉ những giá trị thực sự cực đoan mới bị điều chỉnh.

3.6. Tạo biến mục tiêu + binning

```
# Tạo biến mục tiêu SAU khi xử lý outlier
df_clean['Is_Profit'] = (df_clean['Profit'] > 0).astype(int)

# Binning Discount (biến có thứ tự)
df_clean['Discount_Bin'] = pd.cut(
    df_clean['Discount'],
    bins=[-0.1, 0, 0.2, 0.5, 1],
    labels=['None', 'Low', 'Medium', 'High']
)

# Ép kiểu string để tránh NaN khi encode
df_clean['Discount_Bin'] = df_clean['Discount_Bin'].astype(str)

print("Các biến mới đã tạo:", ['Is_Profit', 'Ship_Duration',
                              'Discount_Bin'])
print(df_clean[['Is_Profit', 'Ship_Duration', 'Discount_Bin']].head())
```

	Is_Profit	Ship_Duration	Discount_Bin
0	1	3	None
1	1	3	None
2	1	4	None
3	0	7	Medium
4	1	7	Low

Hình 3. 7. Biến mới sau quá trình Feature Engineering

Sau khi xử lý ngoại lai, chúng ta tiến hành tạo biến mục tiêu `Is_Profit` từ biến `Profit` đã được điều chỉnh. Biến `Is_Profit` được xác định bằng cách kiểm tra điều kiện `Profit > 0`, với giá trị 1 được gán cho các giao dịch có lợi nhuận và giá trị 0 cho các giao dịch hòa vốn hoặc thua lỗ. Việc chuyển đổi này được thực hiện bằng phương thức `(df_clean['Profit'] > 0).astype(int)`, tạo ra một biến nhị phân rõ ràng. Đây là bước quan trọng không chỉ đơn giản hóa bài toán mà còn phù hợp với nhu cầu thực tế của nhà quản trị, khi họ quan tâm đến câu hỏi “giao dịch này có lãi hay không?” hơn là “lãi bao nhiêu?”.

Bên cạnh đó, chiết khấu (`Discount`) được phân nhóm thành bốn mức có thứ tự dựa trên các ngưỡng đã xác định từ phân tích EDA. Các ngưỡng này được chọn như sau: `None` (0%), `Low` (0% - 20%), `Medium` (20% - 50%), và `High` (>50%). Việc phân nhóm dựa

trên những phát hiện từ biểu đồ phân tán giữa Discount và Profit, trong đó ngưỡng 20% được xác định là điểm bẻ gãy quan trọng khi chiết khấu vượt quá 20%, rủi ro thua lỗ bắt đầu gia tăng đáng kể. Ngưỡng 50% được xem là ranh giới của vùng rủi ro cao, nơi hầu hết các giao dịch đều rơi vào vùng lợi nhuận âm. Các nhóm này được lưu dưới dạng chuỗi ký tự để thuận tiện cho việc mã hóa sau này. Việc phân nhóm giúp mô hình dễ dàng nhận diện các mức độ chiết khấu khác nhau thay vì xử lý chiết khấu như một biến liên tục, đồng thời phản ánh đúng bản chất phi tuyến của mối quan hệ giữa chiết khấu và lợi nhuận.

3.7. Lựa chọn đặc trưng

Sau khi làm sạch và tạo biến mới, việc lựa chọn đặc trưng phù hợp là bước quan trọng để giảm độ phức tạp của mô hình và tránh overfitting. Các biến định danh như Row ID, Order ID, Customer ID, Product ID, và Product Name có số lượng giá trị duy nhất rất lớn (lên đến hàng nghìn) và mang tính nhận dạng, không có giá trị tổng quát hóa cho các giao dịch mới, do đó bị loại bỏ. Biến Country chỉ có một giá trị duy nhất (United States) nên không mang lại thông tin phân biệt, cũng được loại bỏ. Biến City và State có thể hữu ích cho phân tích địa lý, nhưng để tránh tăng chiều dữ liệu quá mức và đảm bảo tính khả thi của mô hình, chúng ta sử dụng biến Region (4 giá trị) thay thế, vì nó vừa phản ánh yếu tố địa lý ở cấp độ vùng miền vừa có số lượng nhóm vừa phải.

```
features = ['Sales', 'Quantity', 'Discount_Bin', 'Ship_Duration',
            'Ship Mode', 'Segment', 'Region', 'Category']

X = df_clean[features].copy()
y = df_clean['Is_Profit']

print(f"1. Kích thước tập đặc trưng đầu vào (X): {X.shape}")
print(f"2. Kích thước tập biến mục tiêu (y): {y.shape}")
print(f"3. Danh sách các đặc trưng (Features): {' , '.join(features)}")

print("\n4. Tỷ lệ phân phối của biến mục tiêu (Is_Profit):")
ti_le_y = y.value_counts(normalize=True) * 100
for index, value in ti_le_y.items():
    nhan = "Có Lợi Nhuận (1)" if index == 1 else "Lỗ (0)"
    print(f"    - {nhan}: {value:.2f}%")

print("\n5. 5 dòng dữ liệu của tập X:")
display(X.head())
```

1. Kích thước tập đặc trưng đầu vào (X): (9994, 8)
2. Kích thước tập biến mục tiêu (y): (9994,)
3. Danh sách các đặc trưng (Features): Sales, Quantity, Discount_Bin, Ship_Duration, Ship Mode, Segment, Region, Category
4. Tỷ lệ phân phối của biến mục tiêu (Is_Profit):
 - Có Lợi Nhuận (1): 80.63%
 - Lỗ (0): 19.37%
5. 5 dòng dữ liệu của tập X:

	Sales	Quantity	Discount_Bin	Ship_Duration	Ship Mode	Segment	Region	Category
0	261.9600	2	None	3	Second Class	Consumer	South	Furniture
1	731.9400	3	None	3	Second Class	Consumer	South	Furniture
2	14.6200	2	None	4	Second Class	Corporate	West	Office Supplies
3	957.5775	5	Medium	7	Standard Class	Consumer	South	Furniture
4	22.3680	2	Low	7	Standard Class	Consumer	South	Office Supplies

Hình 3. 8. Tổng quan tập đặc trưng và biến mục tiêu

Các đặc trưng được giữ lại bao gồm:

- **Nhóm định lượng:** Sales, Quantity, Discount_Bin (đã phân nhóm), Ship_Duration.
- **Nhóm định tính:** Ship Mode (4 phương thức vận chuyển), Segment (3 phân khúc khách hàng), Region (4 khu vực địa lý), Category (3 danh mục sản phẩm).

Sau khi lựa chọn, tập đặc trưng đầu vào X có kích thước (9.994, 8). Đồng thời, chúng ta tính toán tỷ lệ phân phối của biến mục tiêu Is_Profit. Kết quả cho thấy có 6.446 giao dịch có lợi nhuận (chiếm 80,63%) và 1.549 giao dịch thua lỗ (chiếm 19,37%). Sự chênh lệch này là hiện tượng mất cân bằng dữ liệu, sẽ được xử lý ở bước SMOTE sau khi chia dữ liệu.

3.8. Mã hóa dữ liệu (Data Encoding)

Các mô hình học máy không thể xử lý trực tiếp dữ liệu dạng văn bản (categorical), do đó cần được chuyển đổi sang dạng số. Quá trình mã hóa được thực hiện với các phương pháp khác nhau tùy thuộc vào bản chất của từng biến.

3.8.1. Mã hóa biến danh nghĩa (One-Hot Encoding)

Đối với biến Discount_Bin, vốn có tính thứ tự (None < Low < Medium < High), chúng ta sử dụng ánh xạ (mapping) thủ công để chuyển các nhãn thành các số 0, 1, 2, 3. Cách làm này giữ nguyên thông tin về thứ tự của các mức chiết khấu, phù hợp với bản chất có thứ tự của biến này. Kiểm tra cho thấy không có giá trị nào ngoài mapping, đảm bảo tính chính xác.

Đối với các biến định tính không có thứ tự (nominal variables), bao gồm Ship Mode, Segment, Region, và Category, chúng ta áp dụng kỹ thuật One-Hot Encoding. Phương pháp này tạo ra các biến giả (dummy variables) cho từng giá trị của biến gốc, với mỗi biến giả nhận giá trị 1 nếu quan sát thuộc nhóm đó và 0 nếu không. Tham số drop_first=True được sử dụng để loại bỏ một cột (nhóm tham chiếu), nhằm tránh hiện

tượng đa cộng tuyến - tình trạng các biến giả có quan hệ tuyến tính với nhau, gây khó khăn cho việc ước lượng hệ số hồi quy.

```
print("\n--- MÃ HÓA DỮ LIỆU ---")

X_enc = X.copy()

# Ordinal Encoding cho Discount_Bin (có thứ tự)
mapping = {'None': 0, 'Low': 1, 'Medium': 2, 'High': 3}
X_enc['Discount_Bin'] = X_enc['Discount_Bin'].map(mapping)

# Check lỗi mapping
if X_enc['Discount_Bin'].isna().sum() > 0:
    raise ValueError("Discount_Bin có giá trị ngoài mapping!")

# One-Hot Encoding
X_encoded = pd.get_dummies(
    X_enc,
    columns=['Ship Mode', 'Segment', 'Region', 'Category'],
    drop_first=True,
    dtype=int
)

print("5 dòng dữ liệu sau khi mã hóa:")
display(X_encoded.head())
```

--- MÃ HÓA DỮ LIỆU ---
5 dòng dữ liệu sau khi mã hóa:

	Sales	Quantity	Discount_Bin	Ship_Duration	Ship Mode_Same Day	Ship Mode_Second Class	Ship Mode_Standard Class	Segment_Corporate	Segment_Home Office	Region_East	Region_South	Region_West	Category_Office Supplies	Category_Technology
0	261.9600	2	0	3	0	1	0	0	0	0	1	0	0	0
1	731.9400	3	0	3	0	1	0	0	0	0	1	0	0	0
2	14.6200	2	0	4	0	1	0	1	0	0	0	1	1	0
3	957.5775	5	2	7	0	0	1	0	0	0	1	0	0	0
4	22.3680	2	1	7	0	0	1	0	0	0	1	0	1	0

Hình 3. 9. Kết quả sau khi mã hóa dữ liệu

Kết quả sau One-Hot Encoding, bốn biến gốc được thay thế bằng 11 biến giả. Cụ thể:

- Ship Mode (4 giá trị) tạo ra 3 biến giả (First Class, Same Day, Second Class, với Standard Class là nhóm tham chiếu).
- Segment (3 giá trị) tạo ra 2 biến giả (Corporate, Home Office, với Consumer là nhóm tham chiếu).
- Region (4 giá trị) tạo ra 3 biến giả (East, South, West, với Central là nhóm tham chiếu).
- Category (3 giá trị) tạo ra 2 biến giả (Furniture, Office Supplies, với Technology là nhóm tham chiếu).

Việc lựa chọn nhóm tham chiếu dựa trên thứ tự xuất hiện trong dữ liệu và hệ số của các biến giả sau này sẽ được diễn giải là sự chênh lệch so với nhóm tham chiếu

3.8.2. Kích thước và kiểm tra tương quan sau mã hóa

```
print("\n--- KÍCH THƯỚC DỮ LIỆU SAU MÃ HÓA ---")
print(f"Kích thước ban đầu: {X.shape}")
print(f"Kích thước sau encoding: {X_encoded.shape}")
print(f"→ Số đặc trưng tăng từ {X.shape[1]} lên {X_encoded.shape[1]} cột.")

X = X_encoded.copy()
```

```
--- KÍCH THƯỚC DỮ LIỆU SAU MÃ HÓA ---
Kích thước ban đầu: (9994, 8)
Kích thước sau encoding: (9994, 14)
→ Số đặc trưng tăng từ 8 lên 14 cột.
```

Hình 3. 10. Kích thước dữ liệu sau mã hóa

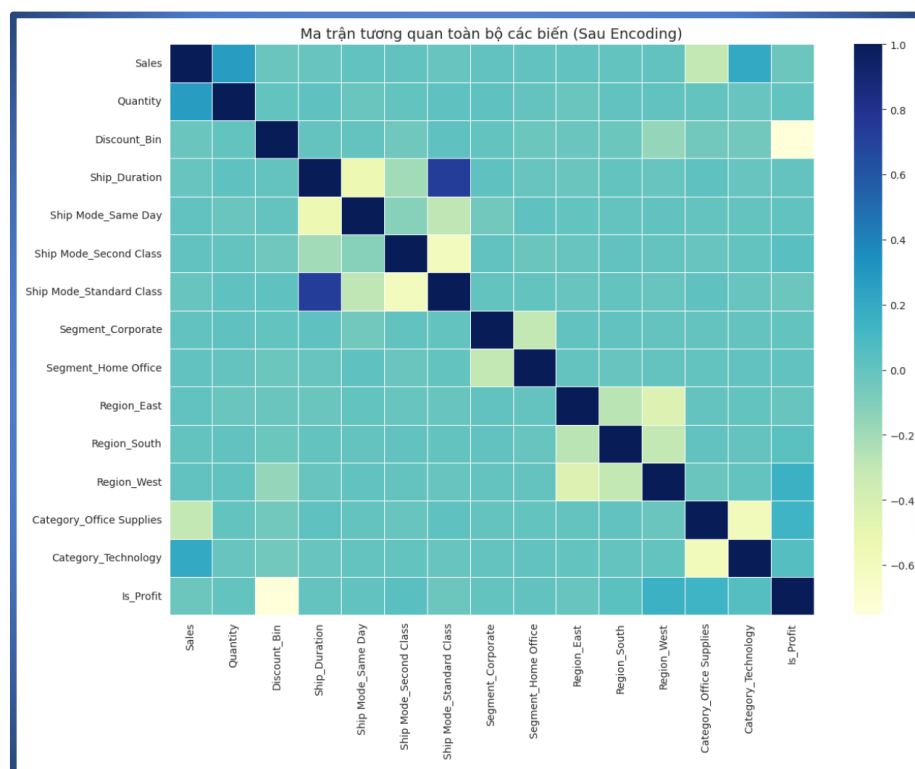
Sau khi mã hóa, số lượng đặc trưng tăng từ 8 lên 14. Sự gia tăng này là do các biến định tính được mở rộng thành nhiều biến giả. Kích thước tập dữ liệu mới là (9.994, 14), với các biến bao gồm các biến định lượng đã có (Sales, Quantity, Discount_Bin đã được ánh xạ, Ship_Duration) và các biến giả mới được tạo ra. Việc tăng số lượng đặc trưng có thể làm tăng độ phức tạp của mô hình, nhưng với số lượng quan sát khoảng 10.000 thì vẫn nằm trong khả năng xử lý của các thuật toán học máy.

3.8.3 Kiểm tra tương quan sau encoding

3.8.3.1. Ma trận tương quan toàn bộ các biến (Heatmap)

```
df_final = pd.concat([X, y], axis=1)

# Heatmap
plt.figure(figsize=(14, 10))
sns.heatmap(df_final.corr(), annot=False, cmap='YlGnBu',
            linewidths=0.5)
plt.title('Ma trận tương quan toàn bộ các biến (Sau Encoding)',
          fontsize=14)
plt.show()
```



Hình 3. 11. Kiểm tra tương quan sau mã hóa

Sau khi thực hiện mã hóa các biến định tính, ma trận tương quan giữa toàn bộ các đặc trưng và biến mục tiêu `Is_Profit` được trực quan hóa thông qua biểu đồ heatmap. Biểu đồ này thể hiện hệ số tương quan Pearson (r) giữa từng cặp biến, với giá trị dao động từ -1 đến 1. Màu sắc trong heatmap phản ánh mức độ và chiều hướng tương quan: các giá trị dương được biểu diễn bằng gam màu xanh, trong khi các giá trị âm thể hiện bằng gam màu vàng nhạt. Việc phân tích ma trận tương quan giúp phát hiện các mối quan hệ tuyến tính mạnh giữa các biến, từ đó đánh giá nguy cơ xảy ra hiện tượng đa cộng tuyến (multicollinearity) trong mô hình.

Kết quả từ heatmap cho thấy phần lớn các cặp biến có hệ số tương quan thấp, chủ yếu tập trung quanh giá trị 0. Điều này cho thấy không tồn tại mối quan hệ tuyến tính mạnh giữa các đặc trưng, và do đó nguy cơ đa cộng tuyến nghiêm trọng là không đáng kể. Đây là một tín hiệu tích cực, vì các mô hình học máy, đặc biệt là các mô hình tuyến tính, sẽ hoạt động ổn định hơn khi các biến đầu vào không có sự phụ thuộc tuyến tính cao. Trong bối cảnh của bài toán phân loại nhị phân này, việc các đặc trưng tương quan yếu với nhau giúp đảm bảo rằng mỗi biến đóng góp một lượng thông tin riêng biệt, không bị trùng lặp, từ đó nâng cao hiệu quả của mô hình.

Một số mối quan hệ đáng chú ý có thể quan sát được giữa các biến giả được tạo ra từ quá trình One-Hot Encoding. Ví dụ, các biến thuộc cùng một nhóm như Ship Mode (Same Day, Second Class, Standard Class) hoặc Region (East, South, West) có xu hướng tương quan âm với nhau. Điều này là hợp lý về mặt logic, vì trong mỗi quan sát, nếu một biến giả nhận giá trị 1 thì các biến còn lại trong cùng nhóm sẽ nhận giá trị 0. Cấu

trúc này là hệ quả tự nhiên của việc chuyển đổi biến phân loại thành các biến giả. Tuy nhiên, mức độ tương quan âm này không quá cao và đã được kiểm soát hiệu quả nhờ việc sử dụng tham số `drop_first=True` trong quá trình mã hóa, giúp tránh tình trạng các biến giả có quan hệ tuyến tính hoàn hảo với nhau, gây khó khăn cho việc ước lượng hệ số trong các mô hình hồi quy.

Ngoài ra, một số cặp biến định lượng cũng thể hiện mối tương quan ở mức trung bình, chẳng hạn như giữa `Ship_Duration` và một số biến về phương thức vận chuyển. Điều này phản ánh thực tế rằng thời gian giao hàng có liên quan đến loại hình vận chuyển được lựa chọn - chẳng hạn, phương thức `Same Day` thường có `Ship_Duration` nhỏ hơn so với `Standard Class`. Mối quan hệ này mang tính logic và phản ánh đúng bản chất của dữ liệu. Tuy nhiên, các mối quan hệ này vẫn chưa đạt đến ngưỡng đáng lo ngại ($|r| > 0.7$), do đó không gây ảnh hưởng tiêu cực đến quá trình huấn luyện mô hình. Trong thực tế, ngưỡng $|r| > 0.7$ thường được coi là dấu hiệu cảnh báo đa cộng tuyến nghiêm trọng, và trong trường hợp này, không có cặp biến nào vượt quá ngưỡng đó.

Đối với biến mục tiêu `Is_Profit`, heatmap cũng cho thấy không có biến nào có tương quan tuyến tính quá mạnh, ngoại trừ `Discount_Bin` có tương quan âm tương đối lớn. Điều này củng cố thêm nhận định từ phân tích trước đó rằng chiết khấu là yếu tố có ảnh hưởng đáng kể đến lợi nhuận. Các biến còn lại như `Sales`, `Quantity`, `Ship_Duration` hay các biến danh mục và khu vực chỉ có tương quan yếu, cho thấy mối quan hệ giữa các đặc trưng và lợi nhuận có thể mang tính phi tuyến hoặc phụ thuộc vào sự kết hợp của nhiều yếu tố. Điều này càng khẳng định sự cần thiết của việc sử dụng các mô hình phi tuyến như `Random Forest` hay `Gradient Boosting`, vốn có khả năng học các tương tác phức tạp giữa các biến mà các mô hình tuyến tính không thể nắm bắt được.

Tóm lại, phân tích ma trận tương quan cho thấy dữ liệu sau khi mã hóa không gặp vấn đề nghiêm trọng về đa cộng tuyến, đồng thời cung cấp cái nhìn tổng quan về mối quan hệ giữa các biến. Các kết quả từ heatmap khẳng định rằng các đặc trưng được lựa chọn có tính độc lập tương đối, đảm bảo sự ổn định cho quá trình huấn luyện mô hình. Đây là cơ sở quan trọng để tiếp tục xây dựng và huấn luyện các mô hình học máy một cách hiệu quả và đáng tin cậy trong các bước tiếp theo, đồng thời củng cố thêm nhận định về vai trò then chốt của chiết khấu trong việc dự báo khả năng sinh lời của giao dịch.

3.8.3.2. Mức độ tương quan của các đặc trưng với biến mục tiêu `Is_Profit` (Barplot)

```
plt.figure(figsize=(10, 6))
correlation_with_target =
df_final.corr()['Is_Profit'].drop('Is_Profit').sort_values(ascending=False)

sns.barplot(
    x=correlation_with_target.values,
    y=correlation_with_target.index,
```

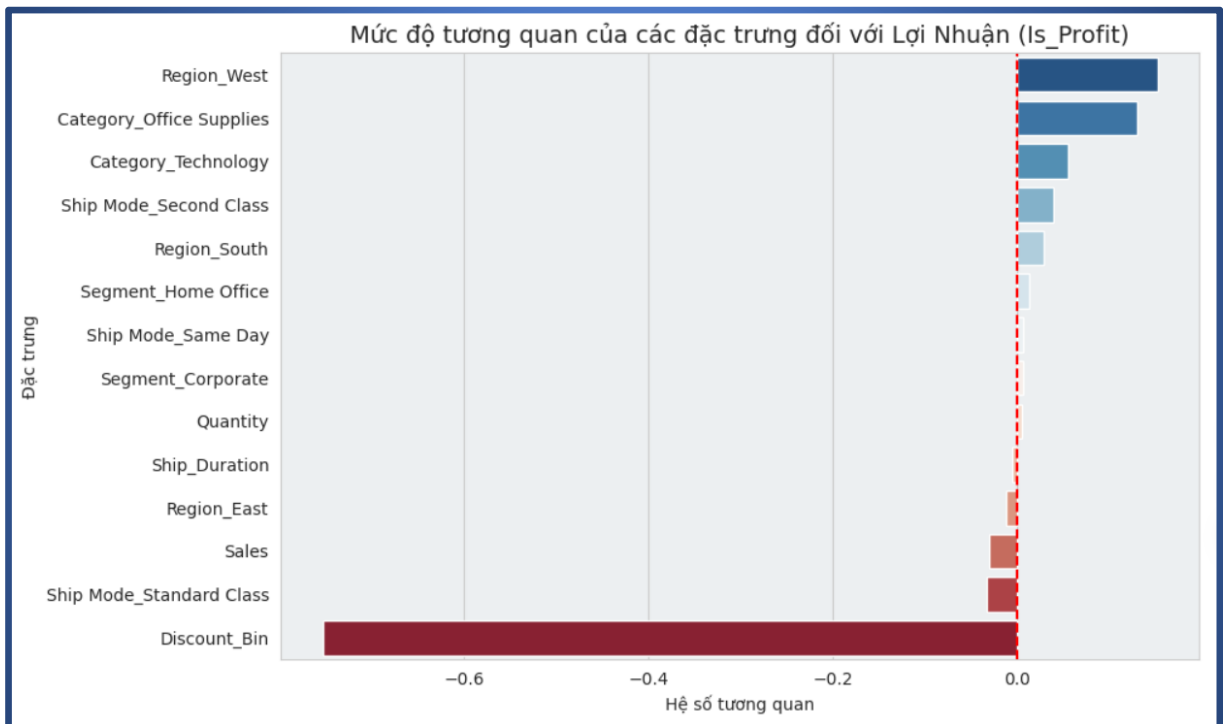


```

    palette='RdBu_r'
)

plt.title('Mức độ tương quan của các đặc trưng đối với Lợi Nhuận (Is_Profit)', fontsize=14)
plt.xlabel('Hệ số tương quan')
plt.ylabel('Đặc trưng')
plt.axvline(x=0, color='red', linestyle='--')
plt.tight_layout()
plt.show()

```



Hình 3. 12. Mức độ tương quan của các đặc trưng với biến mục tiêu Is_Profit

Kết quả được trực quan hóa thông qua biểu đồ cột, trong đó các đặc trưng được sắp xếp theo hệ số tương quan Pearson giảm dần. Nhìn chung, phần lớn các biến có hệ số tương quan thấp, cho thấy không tồn tại mối quan hệ tuyến tính mạnh giữa từng biến riêng lẻ với lợi nhuận, đồng thời gợi ý rằng mô hình học máy sẽ cần khai thác các mối quan hệ phi tuyến và tương tác giữa các biến để đạt hiệu quả cao.

Nổi bật nhất trong phân tích là biến **Discount_Bin**, với hệ số tương quan âm rất mạnh (xấp xỉ -0.65 đến -0.70). Điều này cho thấy mức chiết khấu có ảnh hưởng tiêu cực rõ rệt đến khả năng sinh lời: khi chiết khấu tăng lên, xác suất giao dịch có lợi nhuận giảm đáng kể. Đây là một kết quả hoàn toàn phù hợp với thực tế kinh doanh, khi việc giảm giá sâu thường làm giảm biên lợi nhuận hoặc thậm chí dẫn đến thua lỗ. So với tất cả các biến khác, Discount_Bin đóng vai trò quan trọng nhất trong việc dự báo lợi nhuận. Phát hiện này cũng khớp với phân tích tương quan ban đầu giữa Discount và Profit (hệ số -0,22), nhưng khi được phân nhóm và mã hóa, mức độ ảnh hưởng trở nên rõ ràng hơn, cho thấy việc phân nhóm chiết khấu đã giúp làm nổi bật tác động phi tuyến của yếu tố này.

Bên cạnh đó, một số biến có tương quan dương với *Is_Profit*, đáng chú ý là ***Region_West*** và ***Category_Office Supplies***. Cụ thể, khu vực West cho thấy khả năng sinh lời tốt hơn so với các khu vực khác (hệ số khoảng 0,25), trong khi danh mục Office Supplies có xu hướng mang lại lợi nhuận cao hơn so với nhóm tham chiếu (Furniture). Biến ***Category_Technology*** cũng có tương quan dương nhưng ở mức yếu, cho thấy ảnh hưởng không đáng kể. Những kết quả này gợi ý rằng yếu tố địa lý và danh mục sản phẩm có tác động nhất định đến hiệu quả kinh doanh, dù không phải là yếu tố quyết định.

Ở chiều ngược lại, các biến như ***Ship_Mode_Standard Class***, ***Region_East*** có tương quan âm nhẹ, cho thấy ảnh hưởng tiêu cực nhưng không đáng kể đến lợi nhuận. Trong khi đó, nhiều biến khác như ***Sales***, ***Quantity***, ***Ship_Duration***, cũng như các biến về phân khúc khách hàng (Segment) và phương thức vận chuyển (Ship Mode) có hệ số tương quan rất thấp, gần như bằng 0. Điều này cho thấy các biến này không có mối quan hệ tuyến tính rõ ràng với *Is_Profit* khi xét riêng lẻ, nhưng vẫn có thể đóng vai trò quan trọng trong các mô hình học máy phức tạp thông qua các tương tác với những biến khác.

Đặc biệt, ***Sales*** mặc dù có tương quan dương (khoảng 0,48) trong phân tích tương quan ban đầu giữa Sales và Profit, nhưng sau khi chuyển đổi thành bài toán phân loại nhị phân *Is_Profit*, hệ số tương quan này giảm xuống rất thấp. Điều này cho thấy mối quan hệ giữa doanh số và trạng thái lời/lỗ không còn tuyến tính rõ ràng như khi xét với giá trị lợi nhuận liên tục, và các mô hình phi tuyến sẽ cần được áp dụng để khai thác hiệu quả yếu tố này.

Tóm lại, phân tích tương quan sau khi mã hóa cho thấy ***chiết khấu là yếu tố có ảnh hưởng mạnh nhất và mang tính quyết định đối với khả năng sinh lời***, trong khi các yếu tố như khu vực địa lý (*Region_West*) và danh mục sản phẩm (*Office Supplies*) có tác động tích cực ở mức trung bình. Các biến còn lại chủ yếu có ảnh hưởng yếu hoặc trung tính khi xét riêng lẻ, nhưng vẫn có thể đóng vai trò quan trọng thông qua các tương tác phức tạp trong các mô hình học máy. Những phát hiện này cung cấp cơ sở quan trọng cho việc xây dựng và giải thích các mô hình dự báo ở các bước tiếp theo, đồng thời gợi ý các hướng cải thiện chiến lược kinh doanh nhằm tối ưu hóa lợi nhuận, đặc biệt là trong việc kiểm soát chính sách chiết khấu và tập trung vào các khu vực và danh mục sản phẩm có tiềm năng sinh lời cao.

3.9. Tách dữ liệu thành tập train, test

Để đánh giá khách quan hiệu suất của các mô hình, chúng ta tách dữ liệu thành tập huấn luyện (train) và tập kiểm tra (test) với tỷ lệ 80% - 20%. Việc sử dụng tham số stratify=y đảm bảo rằng tỷ lệ phân phối của biến mục tiêu *Is_Profit* được giữ nguyên trong cả hai tập, tránh tình trạng tập test không đại diện cho tổng thể. Điều này đặc biệt quan trọng khi dữ liệu có sự mất cân bằng giữa các lớp (khoảng 80% - 20%), vì nếu không có

stratify, có thể xảy ra trường hợp tập test chỉ chứa toàn bộ các mẫu của lớp đa số hoặc lớp thiểu số, dẫn đến đánh giá sai lệch về hiệu suất mô hình.

```
print("--- KẾT QUẢ TÁCH TẬP TRAIN / TEST ---")

# Chia dữ liệu
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, stratify=y, random_state=42)

print(f"1. Kích thước tập huấn luyện (Train): X_train {X_train.shape},\ny_train {y_train.shape}")
print(f"2. Kích thước tập kiểm thử (Test): X_test {X_test.shape},\ny_test {y_test.shape}")

print("\n3. Kiểm tra tỷ lệ phân phối nhãn (Is_Profit) sau khi tách\n(Stratify):")
print(f"    - Tỷ lệ trên tập Train:\n\n{(y_train.value_counts(normalize=True)*100).to_string()}")
print(f"    - Tỷ lệ trên tập Test:\n\n{(y_test.value_counts(normalize=True)*100).to_string()}")
```

```
--- KẾT QUẢ TÁCH TẬP TRAIN / TEST ---
1. Kích thước tập huấn luyện (Train): X_train (7995, 14), y_train (7995,)
2. Kích thước tập kiểm thử (Test): X_test (1999, 14), y_test (1999,)

3. Kiểm tra tỷ lệ phân phối nhãn (Is_Profit) sau khi tách (Stratify):
    - Tỷ lệ trên tập Train:
Is_Profit
1    80.625391
0    19.374609
    - Tỷ lệ trên tập Test:
Is_Profit
1    80.64032
0    19.35968
```

Hình 3. 13. Kích thước tập Train/Test và phân bố biến mục tiêu Is_Profit

Kết quả tách dữ liệu cho thấy tập huấn luyện có 7.995 quan sát, tập kiểm tra có 1.999 quan sát. Tỷ lệ phân phối trên tập train là 80,625% lớp 1 (có lợi nhuận) và 19,375% lớp 0 (thua lỗ); trên tập test là 80,640% lớp 1 và 19,360% lớp 0. Sự tương đồng này khẳng định việc tách dữ liệu đã được thực hiện đúng cách, đảm bảo tính đại diện của tập test và độ tin cậy của kết quả đánh giá sau này. Việc giữ nguyên tỷ lệ lớp giữa train và test cũng là điều kiện cần để các chỉ số đánh giá như Precision, Recall, F1-Score có ý nghĩa so sánh.

3.10. Scale dữ liệu

Sau khi hoàn tất quá trình mã hóa các biến định tính và tách dữ liệu thành tập huấn luyện và tập kiểm tra, bước tiếp theo là chuẩn hóa (scaling) các biến định lượng nhằm đưa chúng về cùng một thang đo. Việc chuẩn hóa đóng vai trò quan trọng đối với các mô hình học máy dựa trên khoảng cách (như Hồi quy Logistic) hoặc các thuật toán sử dụng

gradient descent, bởi vì khi các biến có thang đo khác nhau, biến có giá trị lớn hơn sẽ chiếm ưu thế trong việc xác định hướng của gradient, dẫn đến quá trình tối ưu kém hiệu quả và hệ số ước lượng có thể bị lệch.

Trong bộ dữ liệu này, các biến định lượng cần chuẩn hóa bao gồm **Sales** (doanh số), **Quantity** (số lượng sản phẩm) và **Ship_Duration** (thời gian vận chuyển). Cả ba biến này đều có thang đo khác nhau: Sales có giá trị lớn (hàng trăm đến hàng nghìn USD), Quantity có giá trị từ 1 đến 14, và Ship_Duration có giá trị từ 0 đến 7 ngày. Sự chênh lệch về thang đo này có thể ảnh hưởng đến hiệu suất của mô hình nếu không được xử lý.

Nghiên cứu áp dụng phương pháp **Min-Max Scaling** để đưa các biến này về cùng khoảng giá trị [0, 1]. Công thức của Min-Max Scaling được biểu diễn như sau:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Trong đó, x_{min} và x_{max} lần lượt là giá trị nhỏ nhất và lớn nhất của biến đó trên tập huấn luyện. Việc sử dụng MinMaxScaler thay vì các phương pháp chuẩn hóa khác (như StandardScaler) phù hợp với đặc điểm của dữ liệu, khi các biến không có phân phối chuẩn và cần được thu gọn về một khoảng cố định để đảm bảo tính so sánh giữa các hệ số trong mô hình.

Quy trình thực hiện được tuân thủ nghiêm ngặt để tránh hiện tượng **data leakage** (rò rỉ dữ liệu), đó là tình huống thông tin từ tập kiểm tra vô tình được sử dụng trong quá trình huấn luyện, dẫn đến đánh giá quá lạc quan về hiệu suất của mô hình. Cụ thể:

- **Bước 1:** Khởi tạo đối tượng MinMaxScaler.
- **Bước 2:** Fit scaler chỉ trên tập huấn luyện (X_train) để xác định giá trị nhỏ nhất và lớn nhất của từng biến.
- **Bước 3:** Transform cả tập huấn luyện và tập kiểm tra bằng scaler đã được fit.

```
scaler = MinMaxScaler()
cols_scale = ['Sales', 'Quantity', 'Ship_Duration']

X_train[cols_scale] = scaler.fit_transform(X_train[cols_scale])
X_test[cols_scale] = scaler.transform(X_test[cols_scale])

scale_summary = pd.DataFrame({
    'Giá trị Min': X_train[cols_scale].min(),
    'Giá trị Max': X_train[cols_scale].max()
})

print("\nKẾT QUẢ CHUẨN HÓA DỮ LIỆU (MIN-MAX SCALING)")
display(scale_summary.style.background_gradient(cmap='Blues',
axis=None).format("{:.2f}"))
```

KẾT QUẢ CHUẨN HÓA DỮ LIỆU (MIN-MAX SCALING)		
	Giá trị Min	Giá trị Max
Sales	0.00	1.00
Quantity	0.00	1.00
Ship_Duration	0.00	1.00

Hình 3. 14. Kết quả chuẩn hóa dữ liệu bằng Min-Max Scaling

Bảng kết quả cho thấy sau khi áp dụng Min-Max Scaling, *cả ba biến đều có giá trị nằm trong khoảng từ 0 đến 1*. Cụ thể, giá trị nhỏ nhất của mỗi biến trên tập huấn luyện được ánh xạ thành 0, giá trị lớn nhất được ánh xạ thành 1, và các giá trị còn lại được phân bố tuyến tính trong khoảng này. Điều này đảm bảo rằng tất cả các biến định lượng đều có cùng trọng số đóng góp khi tính toán khoảng cách hoặc gradient, giúp mô hình học tập ổn định và hiệu quả hơn.

Các biến định tính đã được mã hóa One-Hot (có giá trị 0 hoặc 1) và biến Discount_Bin đã được ánh xạ thành các giá trị 0, 1, 2, 3 trước đó nên không cần chuẩn hóa thêm, vì chúng đã có thang đo phù hợp và việc chuẩn hóa có thể làm mất đi ý nghĩa thứ tự vốn có của biến. Việc giữ nguyên các biến này là phù hợp với thực tiễn và đảm bảo tính diễn giải của mô hình.

Sau khi hoàn tất bước chuẩn hóa, dữ liệu đã sẵn sàng cho quá trình huấn luyện mô hình. Tập huấn luyện và tập kiểm tra hiện có cùng thang đo, đảm bảo rằng các mô hình học máy (đặc biệt là các mô hình nhạy cảm với thang đo như Hồi quy Logistic) sẽ hoạt động ổn định và cho kết quả đáng tin cậy. Đây là bước chuẩn bị quan trọng trước khi tiến hành xử lý mất cân bằng dữ liệu bằng SMOTE và huấn luyện các mô hình dự báo ở các phần tiếp theo.

3.11. Xử lý mất cân bằng dữ liệu (SMOTE)

Sau khi hoàn tất quá trình chuẩn hóa dữ liệu, bước tiếp theo là xử lý vấn đề mất cân bằng giữa hai lớp của biến mục tiêu Is_Profit. Như đã phát hiện từ bước 3.7, tỷ lệ phân phối của biến mục tiêu trên tập huấn luyện cho thấy sự chênh lệch đáng kể: lớp 1 (có lợi nhuận) chiếm 6.446 mẫu (khoảng 80,6%), trong khi lớp 0 (thua lỗ) chỉ chiếm 1.549 mẫu (khoảng 19,4%). Đây là hiện tượng mất cân bằng dữ liệu, một vấn đề phổ biến trong các bài toán phân loại thực tế, đặc biệt là trong lĩnh vực kinh doanh và tài chính, nơi các sự kiện tiêu cực (như giao dịch thua lỗ) thường ít xảy ra hơn các sự kiện tích cực.

Sự mất cân bằng này có thể dẫn đến những hệ quả nghiêm trọng trong quá trình huấn luyện mô hình. Cụ thể, mô hình có xu hướng học thiên vị về lớp đa số (có lợi nhuận) và dự đoán kém cho lớp thiểu số (thua lỗ) - vốn là lớp quan trọng cần được nhận diện để tránh rủi ro. Nếu không được xử lý, mô hình có thể đạt độ chính xác (accuracy) cao một

cách giả tạo (ví dụ, dự đoán tất cả là có lợi nhuận sẽ đạt accuracy khoảng 80%), nhưng hoàn toàn vô dụng trong việc phát hiện các giao dịch thua lỗ, dẫn đến những quyết định kinh doanh sai lầm.

Để giải quyết vấn đề này, nghiên cứu áp dụng kỹ thuật SMOTE chỉ trên tập huấn luyện. SMOTE không chỉ đơn thuần nhân bản các mẫu của lớp thiểu số, mà tạo ra các mẫu nhân tạo mới bằng cách nội suy giữa một mẫu và các mẫu lân cận trong không gian đặc trưng. Cơ chế hoạt động của SMOTE có thể được mô tả như sau: với mỗi mẫu thuộc lớp thiểu số, thuật toán xác định k mẫu lân cận gần nhất (thường $k = 5$), chọn ngẫu nhiên một trong số các lân cận đó, và tạo ra mẫu mới nằm trên đoạn thẳng nối giữa hai mẫu trong không gian đặc trưng. Công thức tạo mẫu mới được biểu diễn như sau:

$$x_{new} = x_i + \lambda(x_{nn} - x_i)$$

Trong đó x_i là mẫu gốc thuộc lớp thiểu số, x_{nn} là một mẫu lân cận được chọn ngẫu nhiên, và λ là một số ngẫu nhiên trong khoảng $(0, 1)$. Phương pháp này tạo ra các mẫu mới có đặc trưng tương tự nhưng không trùng lặp với các mẫu hiện có, giúp tăng tính đa dạng cho lớp thiểu số và giảm nguy cơ overfitting so với việc chỉ nhân bản đơn thuần.

```
print("\n--- KẾT QUẢ XỬ LÝ MẤT CÂN BẰNG BẰNG SMOTE ---")

# Khởi tạo và fit SMOTE (CHỈ TRÊN TẬP TRAIN)
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

print(f"1. Số lượng mẫu TRƯỚC khi SMOTE (Tập Train):")
print(y_train.value_counts().to_string())

print(f"\n2. Số lượng mẫu SAU khi SMOTE (Tập Train):")
print(y_train_res.value_counts().to_string())
print(f"-> Kích thước tập Train mới sau khi sinh thêm dữ liệu giả lập: {X_train_res.shape}")

# Vẽ biểu đồ so sánh trước và sau SMOTE
plt.figure(figsize=(12, 5))

# 1. Biểu đồ trước SMOTE
plt.subplot(1, 2, 1)
sns.countplot(x=y_train, palette='Blues_r')
plt.title('Tập Train TRƯỚC KHI SMOTE\n(Dữ liệu mất cân bằng)',
fontsize=13, fontweight='bold')
plt.xlabel('Is_Profit (0: Lỗ, 1: Lợi)')
plt.ylabel('Số lượng mẫu')

# 2. Biểu đồ sau SMOTE
plt.subplot(1, 2, 2)
sns.countplot(x=y_train_res, palette='YlGnBu')
```

```
plt.title('Tập Train SAU KHI SMOTE\n(Dữ liệu đã cân bằng 50/50)',
fontsize=13, fontweight='bold')
plt.xlabel('Is_Profit (0: Lỗ, 1: Lãi)')
plt.ylabel('Số lượng mẫu')

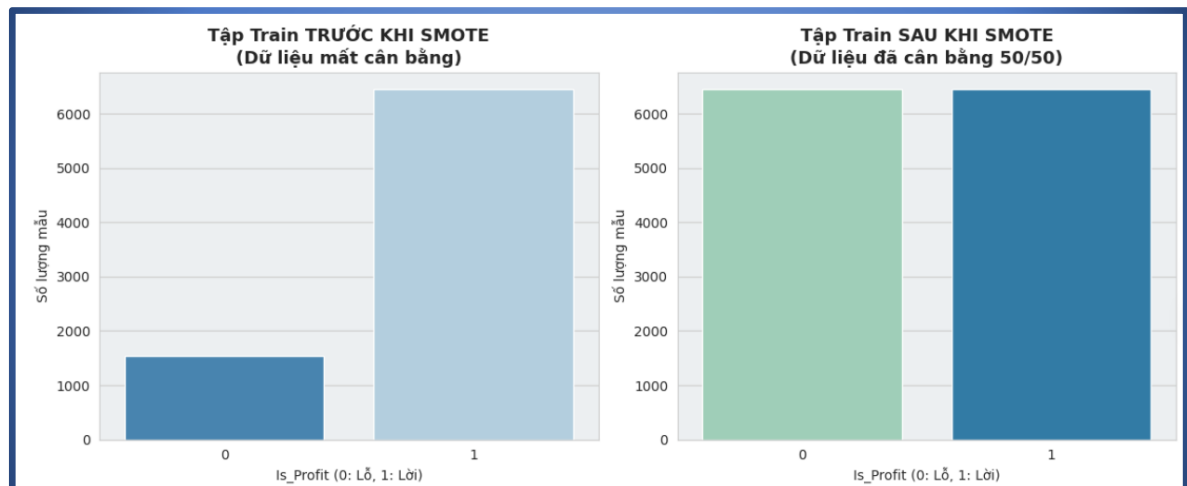
plt.tight_layout()
plt.show()
```

```
--- KẾT QUẢ XỬ LÝ MẤT CÂN BẰNG BẰNG SMOTE ---
1. Số lượng mẫu TRƯỚC khi SMOTE (Tập Train):
Is_Profit
1    6446
0    1549

2. Số lượng mẫu SAU khi SMOTE (Tập Train):
Is_Profit
1    6446
0    6446
-> Kích thước tập Train mới sau khi sinh thêm dữ liệu giả lập: (12892, 14)
```

Hình 3. 15. Kết quả cân bằng dữ liệu bằng phương pháp SMOTE

- **Trước SMOTE (Tập Train):** Số lượng mẫu lớp 1 (có lợi nhuận) là 6.446, lớp 0 (thua lỗ) là 1.549. Tỷ lệ giữa hai lớp là khoảng 4,16:1.
- **Sau SMOTE (Tập Train):** Số lượng mẫu lớp 1 vẫn giữ nguyên 6.446, lớp 0 được tăng lên 6.446. Hai lớp trở nên cân bằng hoàn toàn với tỷ lệ 50-50.
- **Kích thước tập Train mới:** Từ 7.995 mẫu ban đầu, sau khi sinh thêm 4.897 mẫu nhân tạo cho lớp thiểu số, tập huấn luyện mới có kích thước (12.892, 14).



Hình 3. 16. Biểu đồ so sánh phân bố lớp trước và sau SMOTE

Biểu đồ so sánh trước và sau SMOTE thể hiện rõ sự thay đổi này. Ở biểu đồ bên trái (trước SMOTE), hai cột có chiều cao chênh lệch lớn, phản ánh tình trạng mất cân bằng nghiêm trọng giữa hai lớp. Ở biểu đồ bên phải (sau SMOTE), hai cột có chiều cao bằng nhau, thể hiện dữ liệu đã được cân bằng hoàn toàn với số lượng mẫu bằng nhau cho cả

hai lớp. Điều này đảm bảo rằng mô hình sẽ được huấn luyện trên một tập dữ liệu mà cả hai lớp đều có đủ số lượng mẫu để học được các đặc trưng đặc trưng của mình.

Việc áp dụng SMOTE chỉ trên tập huấn luyện và không tác động đến tập kiểm tra là một nguyên tắc quan trọng trong quy trình xây dựng mô hình. Tập kiểm tra vẫn giữ nguyên phân phối thực tế của dữ liệu (khoảng 80% lớp 1 và 20% lớp 0) để đánh giá khách quan hiệu suất của mô hình trên dữ liệu thực tế. Nếu cũng áp dụng SMOTE lên tập kiểm tra, kết quả đánh giá sẽ bị sai lệch và không phản ánh đúng khả năng hoạt động của mô hình trong thực tế.

Sau khi hoàn thành bước xử lý mất cân bằng, chúng ta thu được:

- Tập huấn luyện mới (X_{train_res} , y_{train_res}): Đã được xử lý outlier, chuẩn hóa, mã hóa và cân bằng, có kích thước là 12.892, 14.
- Tập kiểm tra (X_{test} , y_{test}): Vẫn giữ nguyên và chưa qua SMOTE, với kích thước là 1.999, 14, đại diện cho phân phối thực tế của dữ liệu.

Dữ liệu lúc này đã sẵn sàng để đưa vào huấn luyện các mô hình học máy. Việc xử lý mất cân bằng bằng SMOTE giúp đảm bảo rằng mô hình sẽ học được đặc trưng của cả hai lớp một cách cân bằng, từ đó cải thiện khả năng dự đoán cho lớp thiểu số, vốn là lớp quan trọng nhất cần được nhận diện để tránh rủi ro trong hoạt động kinh doanh.

PHẦN IV. MÔ HÌNH HÓA DỮ LIỆU

4.1. Cơ sở lý thuyết và mô hình

4.1.1. Logistic Regression

Bản chất và định nghĩa

Hồi quy Logistic là một mô hình thống kê thuộc nhóm các mô hình phân loại có giám sát, được sử dụng phổ biến cho các bài toán phân loại nhị phân. Mặc dù có tên gọi là “hồi quy”, mục đích của mô hình không phải là dự đoán một giá trị liên tục mà là ước lượng xác suất một quan sát thuộc về một lớp cụ thể, từ đó đưa ra quyết định phân loại. Trong nghiên cứu này, Logistic Regression được sử dụng như một mô hình cơ sở (baseline) để dự báo xác suất một giao dịch có lợi nhuận ($Is_Profit = 1$), dựa trên các đặc trưng đầu vào đã được xử lý.

Về bản chất, Logistic Regression là một mô hình phân biệt, nghĩa là nó học trực tiếp ranh giới quyết định giữa các lớp thông qua xác suất có điều kiện $P(y|\mathbf{x})$, thay vì mô hình hóa phân phối đồng thời của dữ liệu và nhãn như các mô hình sinh (generative models). Điều này giúp mô hình ít chịu ảnh hưởng bởi các giả định về phân phối của dữ liệu đầu vào và thường hoạt động tốt ngay cả khi các đặc trưng có tương quan với nhau.

Logit và tổ hợp tuyến tính

Cốt lõi của mô hình nằm ở giả định về mối quan hệ giữa log-odds (logarit của tỷ số chênh) của xác suất thuộc lớp dương và tổ hợp tuyến tính của các đặc trưng. Cụ thể, với mỗi quan sát được biểu diễn bởi vector đặc trưng $\mathbf{x}=[x_1, x_2, \dots, x_n]$, mô hình giả định:

$$\log\left(\frac{P(y = 1|\mathbf{x})}{1 - P(y = 1|\mathbf{x})}\right) = \mathbf{w}^T \mathbf{x} + b$$

trong đó $\mathbf{w}=[w_1, w_2, \dots, w_n]$ là vector trọng số cần học, b là hệ số chặn (bias), và $z = \mathbf{w}^T \mathbf{x} + b$ được gọi là logit. Logit có thể nhận bất kỳ giá trị thực nào trong khoảng $(-\infty, +\infty)$.

Hàm sigmoid và ánh xạ sang xác suất

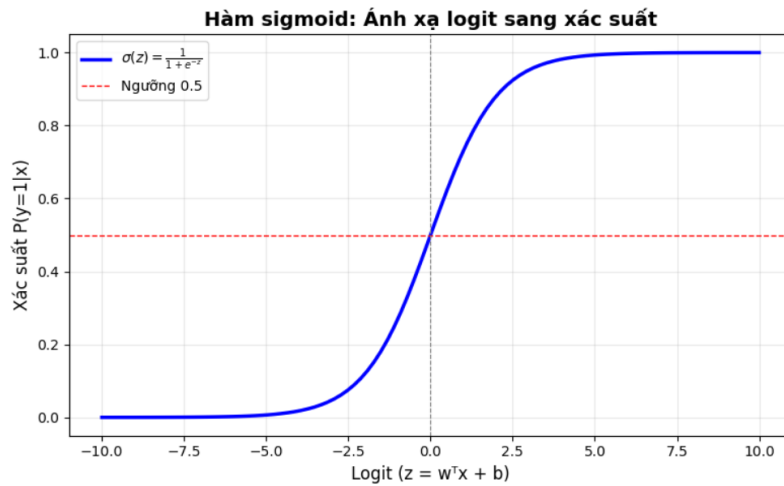
Để chuyển đổi logit thành xác suất nằm trong khoảng $(0,1)$, Logistic Regression sử dụng hàm sigmoid (còn gọi là hàm logistic), được định nghĩa như sau:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Hàm sigmoid có hình dạng chữ S và sở hữu nhiều tính chất toán học quan trọng: nó đơn điệu tăng, tiệm cận về 1 khi $z \rightarrow +\infty$ và về 0 khi $z \rightarrow -\infty$, đồng thời có đạo hàm rất đẹp là $\sigma'(z) = \sigma(z)(1-\sigma(z))$, thuận tiện cho việc tính gradient. Nhờ đó, xác suất hậu nghiệm của lớp dương (có lợi nhuận) được viết gọn là:

$$P(y = 1|x) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

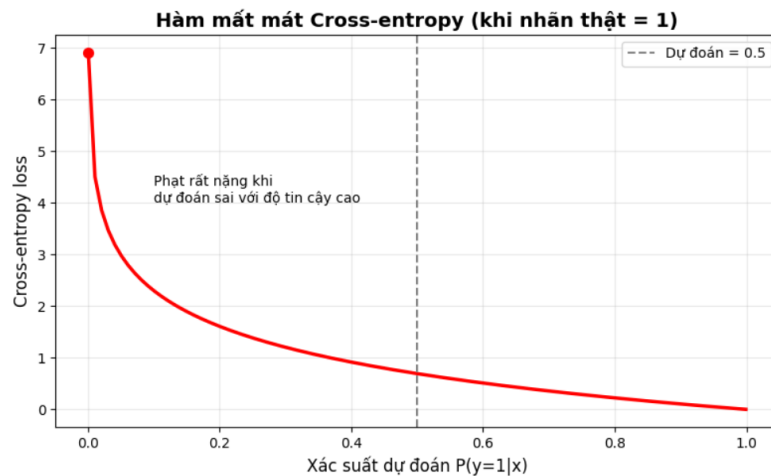
Xác suất thuộc lớp âm (thua lỗ) được suy ra là $P(y=0|x) = 1 - P(y=1|x)$. Quyết định phân loại cuối cùng thường được thực hiện bằng cách so sánh xác suất này với một ngưỡng, phổ biến nhất là 0.5, tương ứng với việc chọn lớp có xác suất cao hơn. Đường biên quyết định trong không gian đặc trưng là siêu phẳng $\mathbf{w}^T \mathbf{x} + b = 0$, phản ánh tính chất tuyến tính của mô hình trong không gian log-odds.



Hình 4. 1. Hàm sigmoid: Ánh xạ logit sang xác suất

Ước lượng tham số bằng Maximum Likelihood

Quá trình huấn luyện mô hình nhằm tìm bộ tham số $\mathbf{w}, b$ tối ưu dựa trên tập dữ liệu huấn luyện $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$. Thay vì sử dụng sai số bình phương như trong hồi quy tuyến tính, Logistic Regression áp dụng phương pháp hợp lý cực đại (Maximum Likelihood Estimation). Giả sử các quan sát độc lập và tuân theo phân phối Bernoulli, hàm likelihood của dữ liệu được xây dựng và sau đó lấy logarit để thuận tiện cho tối ưu.



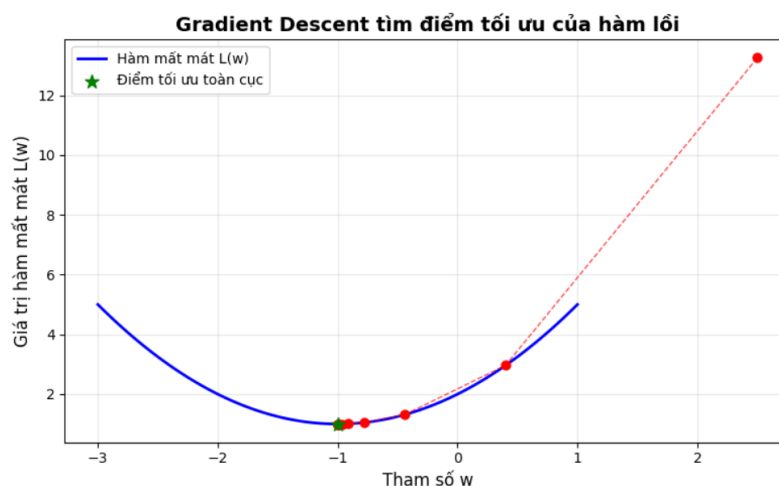
Hình 4. 2. Hàm mất mát Cross-Entropy khi nhãn thật bằng 1

Hàm mất mát tương ứng, gọi là **cross-entropy loss** (negative log-likelihood), có dạng:

$$L(\mathbf{w}, b) = - \sum_{i=1}^m [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

với $\hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$. Hàm này có tính chất lồi, đảm bảo tồn tại một nghiệm toàn cục duy nhất. Về mặt trực quan, cross-entropy sẽ phạt rất nặng những dự đoán sai với độ tin cậy cao: nếu mô hình dự đoán $\hat{y}_i$ gần 0 trong khi $y_i = 1$, số hạng $\log(\hat{y}_i)$ tiến tới vô cùng, buộc thuật toán phải điều chỉnh tham số để tránh những sai lầm nghiêm trọng.

Tối ưu bằng Gradient Descent



Hình 4. 3. Gradient Descent tìm điểm tối ưu của hàm mất mát

Để tìm nghiệm tối ưu, người ta sử dụng các thuật toán tối ưu dựa trên gradient, phổ biến nhất là **Gradient Descent** hoặc các biến thể như Stochastic Gradient Descent. Đạo hàm của hàm mất mát đối với trọng số w_j và hệ số chặn b được tính như sau:

$$\frac{\partial L}{\partial w} = \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}, \quad \frac{\partial L}{\partial b} = \sum_{i=1}^m (\hat{y}_i - y_i)$$

Kết quả này mang ý nghĩa trực quan sâu sắc: gradient phụ thuộc trực tiếp vào **sai số dự đoán** $\hat{y}_i - y_i$ và được điều chỉnh bởi giá trị của đặc trưng tương ứng. Nếu mô hình dự đoán cao hơn thực tế ($\hat{y}_i > y_i$), gradient dương và trọng số sẽ giảm; ngược lại, nếu dự đoán thấp hơn, gradient âm và trọng số sẽ tăng. Thuật toán cập nhật tham số theo công thức:

$$w_j \leftarrow w_j - \eta \frac{\partial L}{\partial w_j}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

với η là tốc độ học (learning rate) - một siêu tham số quan trọng kiểm soát độ lớn của mỗi bước cập nhật. Nếu η quá lớn, thuật toán có thể dao động và không hội tụ; nếu η quá nhỏ, quá trình hội tụ sẽ rất chậm.

Chuẩn hóa dữ liệu (Feature Scaling) và Regularization

Một vấn đề cần đặc biệt lưu ý khi áp dụng Logistic Regression là tính nhạy cảm với thang đo của các đặc trưng. Vì mô hình dựa trên tổ hợp tuyến tính, nếu không chuẩn hóa, các đặc trưng có giá trị lớn (như doanh số Sales có thể lên đến hàng nghìn USD) sẽ chiếm ưu thế trong gradient so với các đặc trưng có giá trị nhỏ (như số lượng Quantity chỉ từ 1 đến 14). Điều này không chỉ làm chậm quá trình hội tụ mà còn khiến các hệ số hồi quy không thể so sánh trực tiếp về mức độ ảnh hưởng. Trong nghiên cứu này, phương pháp **Min-Max Scaling** được áp dụng để đưa các biến định lượng về cùng khoảng $[0,1]$, đảm bảo mọi đặc trưng đóng góp công bằng vào quá trình học.

Để phòng tránh hiện tượng overfitting, tức mô hình học quá chi tiết dữ liệu huấn luyện, bao gồm cả nhiễu, dẫn đến khả năng tổng quát hóa kém, người ta thường bổ sung một thành phần chính quy hóa (regularization) vào hàm mất mát. Hai dạng chính quy hóa phổ biến là **L2 (Ridge)** và **L1 (Lasso)**. Chính quy hóa L2 thêm vào một lượng phạt tỷ lệ với bình phương các trọng số, có xu hướng tạo ra các trọng số nhỏ và phân bố đều, giúp mô hình ổn định hơn. Ngược lại, chính quy hóa L1 thêm phạt tỷ lệ với giá trị tuyệt đối của trọng số, có khả năng đưa nhiều trọng số về 0, thực hiện lựa chọn đặc trưng tự động và tạo ra mô hình đơn giản, dễ diễn giải hơn. Hàm mất mát khi có chính quy hóa L2, chẳng hạn, có dạng:

$$L_{reg}(\mathbf{w}, b) = L(\mathbf{w}, b) + \frac{\alpha}{2} \|\mathbf{w}\|^2$$

Hệ số α (hoặc tham số $C=1/\alpha$ trong thư viện scikit-learn) kiểm soát mức độ chính quy hóa: α càng lớn, mô hình càng đơn giản, bias tăng nhưng variance giảm. Việc lựa chọn α tối ưu thường được thực hiện thông qua kiểm chứng chéo (cross-validation).

Diễn giải hệ số và ý nghĩa trong kinh doanh

Một trong những ưu điểm lớn nhất của Logistic Regression so với các mô hình phức tạp hơn là **khả năng diễn giải trực tiếp**. Hệ số hồi quy w_j cho biết sự thay đổi trong log-odds của xác suất có lợi nhuận khi đặc trưng x_j tăng một đơn vị, với điều kiện các đặc trưng khác không đổi. Từ đó, đại lượng e^{w_j} được gọi là **odds ratio**: nếu $w_j > 0$, odds (tỷ số chênh) tăng lên e^{w_j} lần; nếu $w_j < 0$, odds giảm xuống. Trong bối cảnh kinh doanh, điều này cho phép định lượng chính xác tác động của từng yếu tố. Ví dụ, nếu hệ số của Discount_Bin là âm, có thể kết luận rằng việc tăng mức chiết khấu làm giảm xác suất có lợi nhuận, từ đó khuyến nghị siết chặt chính sách khuyến mãi. Nếu hệ số của Category_Office Supplies dương, nhóm hàng văn phòng phẩm có xu hướng sinh lời tốt hơn so với nhóm tham chiếu (ví dụ Furniture), gợi ý ưu tiên nguồn lực vào nhóm này.

Hạn chế của Logistic Regression

Tuy nhiên, Logistic Regression cũng tồn tại những hạn chế cố hữu. Quan trọng nhất là giả định về tính tuyến tính của log-odds theo các đặc trưng. Mô hình không thể tự động nắm bắt các mối quan hệ phi tuyến phức tạp hoặc các tương tác giữa các biến. Như đã

phát hiện trong quá trình phân tích khám phá dữ liệu (EDA) của bộ dữ liệu Sample Superstore, tác động của chiết khấu lên lợi nhuận thay đổi mạnh mẽ theo từng danh mục sản phẩm: với Furniture, chỉ cần chiết khấu vượt 20% đã xuất hiện nhiều giao dịch thua lỗ, trong khi Technology có thể chịu được chiết khấu lên đến 50-60% mà vẫn duy trì lợi nhuận. Để Logistic Regression học được tương tác này, người phân tích buộc phải tạo thủ công các biến tương tác (ví dụ $\text{Discount} \times \text{Category_Technology}$). Ngoài ra, mô hình cũng nhạy cảm với các giá trị ngoại lai, đòi hỏi các bước tiền xử lý kỹ lưỡng như Winsorization đã được thực hiện trong nghiên cứu này.

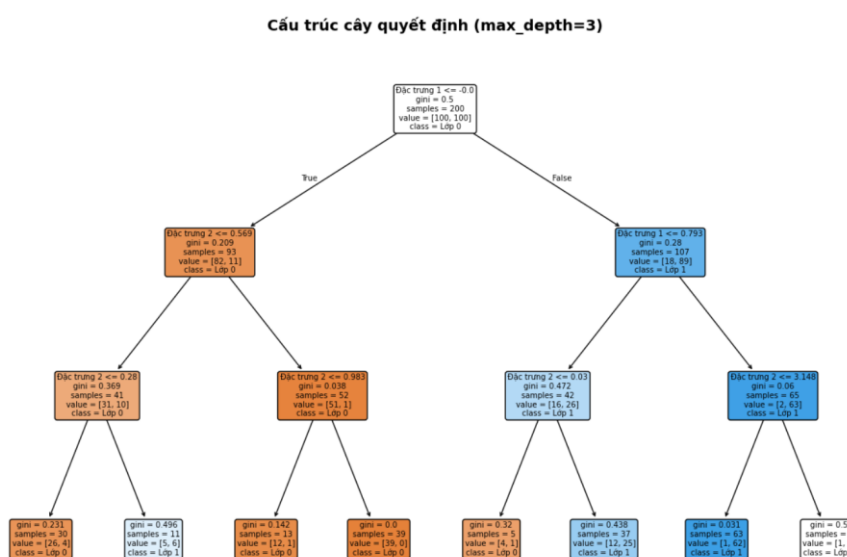
Vai trò trong nghiên cứu

Trong khuôn khổ bài toán dự báo lợi nhuận, Logistic Regression đóng vai trò là mô hình cơ sở (baseline). Nó cung cấp một mức tham chiếu về độ chính xác và khả năng diễn giải, từ đó cho phép so sánh với các mô hình phức tạp hơn như Decision Tree, Random Forest và Gradient Boosting. Nếu sự cải thiện của các mô hình phi tuyến là đáng kể, điều đó chứng tỏ dữ liệu có cấu trúc phi tuyến và tương tác phức tạp mà mô hình tuyến tính không thể khai thác. Ngược lại, nếu sự cải thiện không đáng kể, Logistic Regression sẽ được ưu tiên nhờ tính đơn giản, tốc độ và khả năng giải thích vượt trội.

4.1.2. Decision Tree

Bản chất và định nghĩa

Cây quyết định là một mô hình học máy có giám sát thuộc nhóm các phương pháp phân loại và hồi quy phi tham số. Khác với các mô hình tham số như hồi quy logistic vốn đòi hỏi những giả định tiên nghiệm về dạng phân phối của dữ liệu hoặc cấu trúc của mối quan hệ giữa đầu vào và đầu ra, cây quyết định không đưa ra bất kỳ giả định nào như vậy. Thay vào đó, nó học trực tiếp từ dữ liệu thông qua một quá trình phân vùng không gian đặc trưng một cách đệ quy, tạo ra một cấu trúc phân cấp dạng cây.



Hình 4. 4. Minh họa cây quyết định phân loại nhị phân

Trong cấu trúc này, mỗi nút bên trong đại diện cho một phép kiểm tra trên một đặc trưng đầu vào (ví dụ “chiết khấu có lớn hơn 20% hay không?”), mỗi nhánh đại diện cho kết quả của phép kiểm tra, và mỗi nút lá đại diện cho một quyết định đầu ra (nhãn lớp). Cách thức vận hành này phản ánh một cách trực quan quá trình ra quyết định của con người: đặt ra các câu hỏi tuần tự, mỗi câu hỏi phân chia không gian vấn đề thành các vùng nhỏ hơn, và cuối cùng đưa ra kết luận dựa trên các đặc điểm đã được xác định. Trong nghiên cứu này, cây quyết định được sử dụng như một mô hình phi tuyến đơn lẻ, đóng vai trò cầu nối giữa hồi quy logistic tuyến tính và các kỹ thuật tập hợp phức tạp hơn như Random Forest và gradient boosting.

Thuật toán học CART và chiến lược tham lam

Thuật toán học cây quyết định, đặc biệt là phiên bản CART (Classification and Regression Trees) được triển khai trong thư viện scikit-learn, dựa trên nguyên lý tối ưu hóa cục bộ theo chiến lược “tham lam” (greedy) và “cận thị” (myopic). Điều này có nghĩa là tại mỗi nút, thuật toán chỉ quan tâm đến việc tìm kiếm đặc trưng và ngưỡng phân tách tốt nhất nhằm tối đa hóa độ thuần nhất của các tập con ngay tại bước hiện tại, mà không xét đến ảnh hưởng của quyết định này đối với các bước phân tách trong tương lai. Mặc dù chiến lược này không đảm bảo tìm được cây tối ưu toàn cục, nhưng nó mang lại hiệu quả tính toán cao và thường tạo ra các cây có chất lượng tốt trong thực tế.

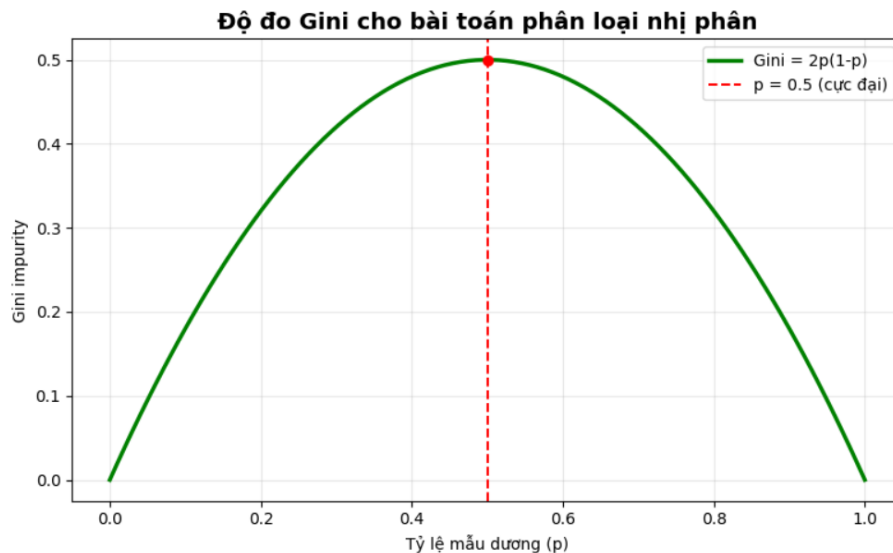
Độ đo Gini impurity và lựa chọn phân tách

Độ thuần nhất của một tập dữ liệu tại một nút được đo lường bằng các độ đo như **Gini impurity** hoặc **entropy**. Trong CART, Gini impurity được sử dụng phổ biến hơn nhờ tính toán đơn giản và hiệu quả tương đương. Đối với một nút có phân phối nhãn với các xác suất $P(c_1), P(c_2), \dots, P(c_k)$, Gini impurity được định nghĩa là:

$$Gini = 1 - \sum_{i=1}^k P(c_i)^2$$

Đại lượng này có thể được hiểu là xác suất để một mẫu được gán nhãn ngẫu nhiên theo phân phối hiện tại bị phân loại sai. Giá trị Gini đạt 0 khi tập dữ liệu hoàn toàn thuần nhất (chỉ chứa một lớp) và đạt cực đại khi phân phối đều.

Đối với bài toán phân loại nhị phân, nếu tỷ lệ mẫu dương là p , $Gini = 2p(1-p)$, đạt cực đại 0,5 tại $p = 0,5$. Khi lựa chọn phân tách, thuật toán sẽ tính toán **mức giảm Gini** (Gini gain) cho mỗi đặc trưng và mỗi ngưỡng phân tách khả dĩ, sau đó chọn phương án làm giảm Gini nhiều nhất.



Hình 4. 5. Độ đo Gini cho bài toán phân loại nhị phân

Xử lý đặc trưng số thực

Một thách thức lớn khi áp dụng cây quyết định là xử lý các đặc trưng có giá trị thực (real-valued features), như doanh số (Sales) hay thời gian vận chuyển (Ship_Duration). Không giống như đặc trưng phân loại với số lượng giá trị hữu hạn, đặc trưng số thực có miền giá trị liên tục và vô hạn, do đó không thể tạo nhánh cho từng giá trị riêng lẻ. CART giải quyết vấn đề này bằng kỹ thuật phân nhị phân động (dynamic binary splitting).

Quá trình thực hiện gồm các bước: sắp xếp các mẫu theo thứ tự tăng dần của giá trị đặc trưng; xét các điểm nằm giữa hai giá trị liên tiếp khác nhau; chỉ những điểm mà nhãn lớp thay đổi giữa hai giá trị kề nhau mới được xem xét, vì nếu nhãn không thay đổi thì việc phân tách tại điểm đó không mang lại lợi ích thông tin; cuối cùng, tính toán Gini gain cho mỗi điểm cắt ứng viên và chọn điểm cắt tối ưu. Phương pháp này tự động tìm được ngưỡng phân tách phù hợp dựa trên dữ liệu, nhưng có thể dẫn đến việc kiểm tra cùng một đặc trưng nhiều lần ở các độ sâu khác nhau, làm cho cây trở nên sâu hơn và phức tạp hơn.

Vấn đề overfitting và kỹ thuật tỉa cây

Mặc dù cây quyết định có khả năng xấp xỉ bất kỳ hàm số nào với độ chính xác tùy ý khi được phép phát triển đầy đủ, nhưng điều này lại dẫn đến một vấn đề nghiêm trọng: **overfitting** (quá khớp). Khi cây được phép phân tách cho đến khi mỗi nút lá chỉ chứa một mẫu duy nhất, nó sẽ học cả nhiễu và các bất thường ngẫu nhiên trong dữ liệu huấn luyện, khiến khả năng tổng quát hóa trên dữ liệu mới trở nên rất kém. Để kiểm soát overfitting, các kỹ thuật **tỉa cây (pruning)** được áp dụng, bao gồm tỉa trước (pre-pruning) và tỉa sau (post-pruning).

Tỉa trước dừng quá trình phát triển cây sớm dựa trên các tiêu chí xác định trước, chẳng hạn như: giới hạn độ sâu tối đa của cây (max_depth), yêu cầu số lượng mẫu tối thiểu tại

một nút để có thể tiếp tục phân tách (`min_samples_split`), hoặc ngưỡng giảm Gini tối thiểu. Các ràng buộc này giúp cây không trở nên quá phức tạp, nhưng có nguy cơ bỏ lỡ các mẫu quan trọng nếu ngưỡng được đặt quá cao.

Tia sau xây dựng cây đầy đủ trước, sau đó cắt bỏ các nhánh không cần thiết bằng cách đánh giá tác động của việc thay thế một nút trong bằng một nút lá (dự đoán theo lớp đa số). Một trong những phương pháp tia sau phổ biến nhất là **cost-complexity pruning** (CCP), được sử dụng trong CART. CCP định nghĩa một hàm chi phí kết hợp giữa sai số huấn luyện và độ phức tạp của cây:

$$R_{\alpha} = R(T) + \alpha \times |T|$$

trong đó $R(T)$ là sai số huấn luyện của cây T (thường được đo bằng tỷ lệ phân loại sai hoặc Gini impurity tổng thể), $|T|$ là số lượng nút lá, và α là tham số phức tạp (complexity parameter). Khi $\alpha = 0$, hàm chi phí chỉ còn là sai số huấn luyện, ưu tiên cây lớn hơn. Khi α tăng, độ phức tạp của cây bị phạt nặng hơn, dẫn đến cây nhỏ hơn, đơn giản hơn. Giá trị α tối ưu thường được chọn thông qua kiểm chứng chéo.

Trong nghiên cứu này, để cân bằng giữa khả năng học và khả năng tổng quát hóa, cây quyết định được cấu hình với hai ràng buộc tia trước: độ sâu tối đa `max_depth = 5` và số lượng mẫu tối thiểu để phân tách `min_samples_split = 20`. Các tham số này được chọn dựa trên phân tích khám phá dữ liệu và thực nghiệm, nhằm đảm bảo cây đủ sức nắm bắt các tương tác phi tuyến quan trọng (ví dụ tác động của chiết khấu lên lợi nhuận thay đổi theo danh mục sản phẩm) mà không bị overfitting.

Độ quan trọng của đặc trưng (Feature Importance) và trực quan hóa

Một ưu điểm nổi bật của cây quyết định so với hồi quy logistic là khả năng cung cấp chỉ số **độ quan trọng của đặc trưng (feature importance)**. Chỉ số này được tính dựa trên tổng mức giảm Gini (hoặc entropy) trung bình mà mỗi đặc trưng đóng góp, được trọng số hóa theo số lượng mẫu tại mỗi nút và tính trung bình trên toàn bộ cây. Feature importance có giá trị từ 0 đến 1 và tổng bằng 1. Trong bối cảnh dự báo lợi nhuận, chỉ số này giúp xác định chính xác các yếu tố then chốt, chẳng hạn như chiết khấu (`Discount_Bin`) hay danh mục sản phẩm (`Category`), từ đó tập trung nguồn lực cải thiện hoặc điều chỉnh chiến lược kinh doanh.

Ngoài ra, cây quyết định có thể được trực quan hóa một cách dễ dàng. Một cây với độ sâu vừa phải (khoảng 3-5) có thể được vẽ ra và trình bày trực tiếp cho ban lãnh đạo, giúp họ hiểu được quy trình ra quyết định của mô hình thông qua các luật if-then rõ ràng, chẳng hạn như “nếu chiết khấu ở mức Medium hoặc High và danh mục sản phẩm là Furniture thì dự đoán thua lỗ”. Điều này đặc biệt quan trọng trong các ứng dụng kinh doanh, nơi yêu cầu về tính minh bạch và khả năng giải thích thường cao hơn so với các ứng dụng kỹ thuật thuần túy.

Hạn chế của cây quyết định đơn lẻ

Tuy nhiên, cây quyết định đơn lẻ có một số hạn chế đáng kể. Ngoài vấn đề overfitting đã được đề cập, cây quyết định còn có độ ổn định thấp: một thay đổi nhỏ trong dữ liệu huấn luyện (ví dụ thêm hoặc bớt một vài mẫu) có thể tạo ra một cấu trúc cây hoàn toàn khác. Nguyên nhân là do tính chất tham lam của thuật toán và sự nhạy cảm với các điểm cắt. Cây quyết định cũng có xu hướng thiên vị đối với các đặc trưng có nhiều giá trị phân tách (ví dụ các biến phân loại với nhiều nhóm), vì chúng tạo ra nhiều cơ hội giảm Gini hơn, dù có thể không mang lại lợi ích thực sự.

Ngoài ra, ranh giới quyết định của cây quyết định là các đường thẳng song song với các trục tọa độ (axis-aligned), tạo thành các vùng hình chữ nhật trong không gian đặc trưng. Điều này có thể không phù hợp với các bài toán có ranh giới quyết định xiên hoặc cong phức tạp. Mặc dù vậy, với bộ dữ liệu Sample Superstore, nơi các đặc trưng chính như chiết khấu, doanh số, danh mục sản phẩm có thể được phân tách hiệu quả bằng các ngưỡng dọc theo từng trục, cây quyết định vẫn là một lựa chọn phù hợp.

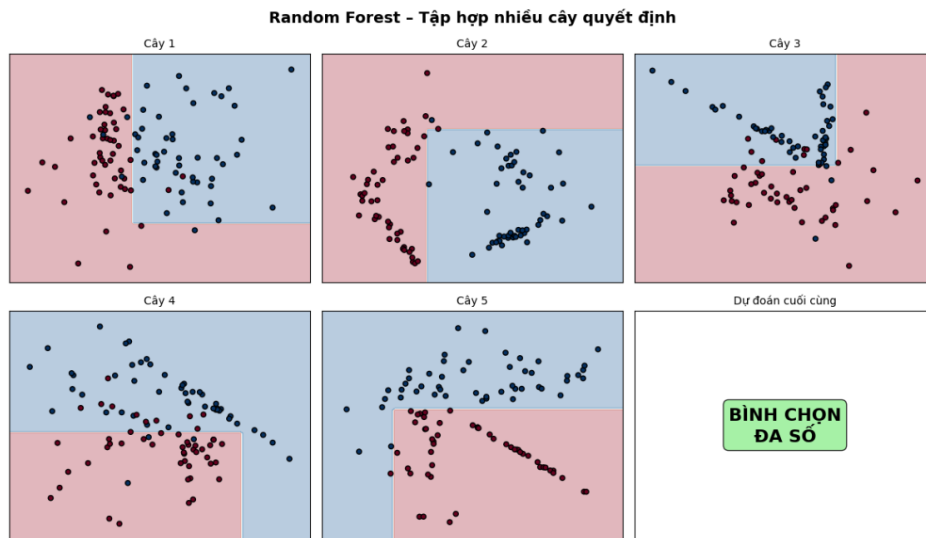
Kỳ vọng và vai trò trong nghiên cứu

Trong nghiên cứu này, cây quyết định được huấn luyện trên tập dữ liệu đã cân bằng bằng SMOTE. So với hồi quy logistic, cây quyết định được kỳ vọng sẽ đạt hiệu suất cao hơn nhờ khả năng nắm bắt các mối quan hệ phi tuyến, đặc biệt là tương tác giữa chiết khấu và danh mục sản phẩm. Tuy nhiên, do là mô hình đơn lẻ, nó vẫn có thể bị overfitting và có độ ổn định kém hơn so với các kỹ thuật ensemble như Random Forest và gradient boosting. Do đó, kết quả từ cây quyết định sẽ được sử dụng để so sánh và đánh giá mức độ cải thiện khi chuyển sang các mô hình phức tạp hơn, đồng thời vẫn giữ lại giá trị về tính minh bạch và khả năng diễn giải cho các tình huống cần giải thích rõ ràng từng quyết định.

4.1.3. Random Forest

Bản chất và định nghĩa

Random Forest là một kỹ thuật học tập hợp (ensemble learning) được phát triển bởi Leo Breiman, dựa trên nguyên lý xây dựng một số lượng lớn các cây quyết định và kết hợp chúng lại với nhau để tạo ra một mô hình dự báo có độ chính xác và ổn định cao hơn nhiều so với một cây đơn lẻ.



Hình 4. 6. Minh họa cách hoạt động của Random Forest

Khác với cây quyết định đơn lẻ vốn có phương sai lớn và rất nhạy cảm với những thay đổi nhỏ trong dữ liệu huấn luyện, Random Forest tận dụng sức mạnh của “sự khôn ngoan của đám đông” để giảm phương sai tổng thể mà không làm tăng độ chệch (bias) một cách đáng kể. Lý thuyết của Breiman chỉ ra rằng sai số tổng quát hóa của Random Forest phụ thuộc vào hai yếu tố chính: độ tương quan trung bình giữa các cây trong rừng và độ mạnh (strength) của từng cây riêng lẻ. Mục tiêu của thuật toán là tạo ra các cây vừa có độ chính xác cao vừa có sự đa dạng, tức là tương quan thấp với nhau.

Cơ chế Bagging và lấy mẫu Bootstrap

Cơ chế hoạt động của Random Forest dựa trên hai nguyên lý cốt lõi: bagging (bootstrap aggregating) và lựa chọn đặc trưng ngẫu nhiên (random feature selection). Nguyên lý thứ nhất, bagging, bắt đầu bằng việc tạo ra nhiều mẫu bootstrap từ tập dữ liệu huấn luyện gốc. Mỗi mẫu bootstrap được hình thành bằng cách lấy ngẫu nhiên có hoàn lại N quan sát từ tập dữ liệu ban đầu có kích thước N . Do lấy mẫu có hoàn lại, mỗi mẫu bootstrap chứa khoảng 63,2% các quan sát duy nhất, trong khi 36,8% còn lại không được chọn - những quan sát này được gọi là out-of-bag (OOB). Trên mỗi mẫu bootstrap, một cây quyết định được huấn luyện độc lập, thường được phép phát triển đầy đủ (không tỉa) để đảm bảo mỗi cây có độ chệch thấp.

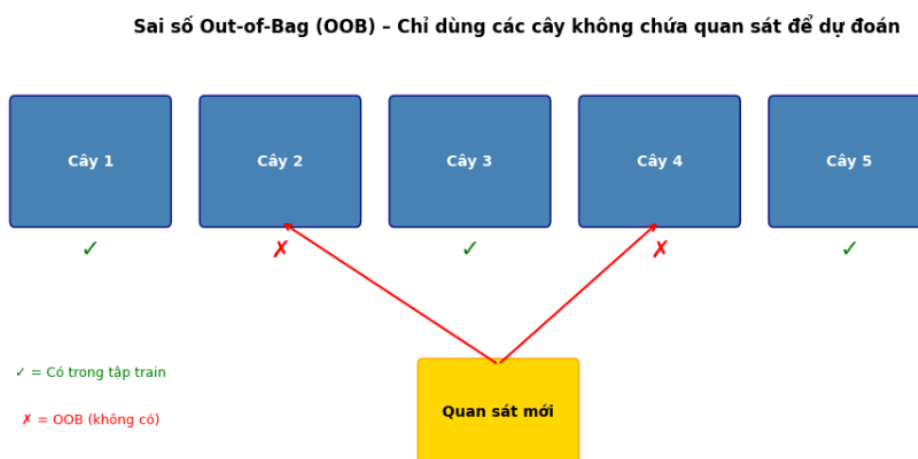
Khi cần dự đoán cho một quan sát mới, tất cả các cây trong rừng đưa ra dự đoán của riêng mình, và kết quả cuối cùng được xác định bằng bình chọn đa số (đối với phân loại) hoặc trung bình (đối với hồi quy). Nhờ cơ chế lấy mẫu bootstrap và kết hợp kết quả, phương sai tổng thể của mô hình giảm đi đáng kể so với một cây đơn lẻ, vì sai số của các cây khác nhau có xu hướng triệt tiêu lẫn nhau khi lấy trung bình.

Lựa chọn đặc trưng ngẫu nhiên

Nguyên lý thứ hai, lựa chọn đặc trưng ngẫu nhiên, là điểm khác biệt quan trọng giữa Random Forest và bagging thông thường. Trong khi bagging với cây quyết định vẫn có thể tạo ra các cây tương quan cao nếu tất cả chúng đều sử dụng cùng một đặc trưng mạnh nhất để phân tách ở các nút đầu, Random Forest đưa thêm một tầng ngẫu nhiên: tại mỗi nút của mỗi cây, thay vì xem xét toàn bộ p đặc trưng để tìm phân tách tốt nhất, thuật toán chỉ xem xét một tập con ngẫu nhiên gồm k đặc trưng (với $k = p$ đối với bài toán phân loại, hoặc $k = p/3$ đối với hồi quy) .

Việc lựa chọn đặc trưng ngẫu nhiên làm giảm mạnh sự tương quan giữa các cây, bởi mỗi cây bị buộc phải xem xét các đặc trưng khác nhau, đặc biệt là ở các nút phía trên . Các cây trở nên đa dạng hơn, mỗi cây có thể “giỏi” ở những vùng khác nhau của không gian đặc trưng, và khi kết hợp lại, chúng bù trừ cho nhau một cách hiệu quả. Lý thuyết chỉ ra rằng việc giảm tương quan thường mang lại lợi ích lớn hơn so với việc mất đi một chút độ chính xác của từng cây do không được dùng toàn bộ đặc trưng tốt nhất .

Sai số Out-of-Bag (OOB)



Hình 4. 7. Cơ chế đánh giá mô hình Random Forest bằng OOB

Một ưu điểm quan trọng của Random Forest là khả năng cung cấp ước lượng sai số nội tại mà không cần tách riêng tập kiểm định, thông qua sai số out-of-bag (OOB) . Với mỗi quan sát, ta xác định các cây không sử dụng quan sát đó trong quá trình huấn luyện (các cây out-of-bag). Dự đoán cho quan sát đó được tính bằng bình chọn từ các cây này, và sai số OOB được tính bằng tỷ lệ dự đoán sai trên toàn bộ tập dữ liệu. Sai số OOB được chứng minh là một ước lượng không chệch của sai số tổng quát hóa, có độ tin cậy tương đương với kiểm chứng chéo nhưng tiết kiệm thời gian tính toán hơn .

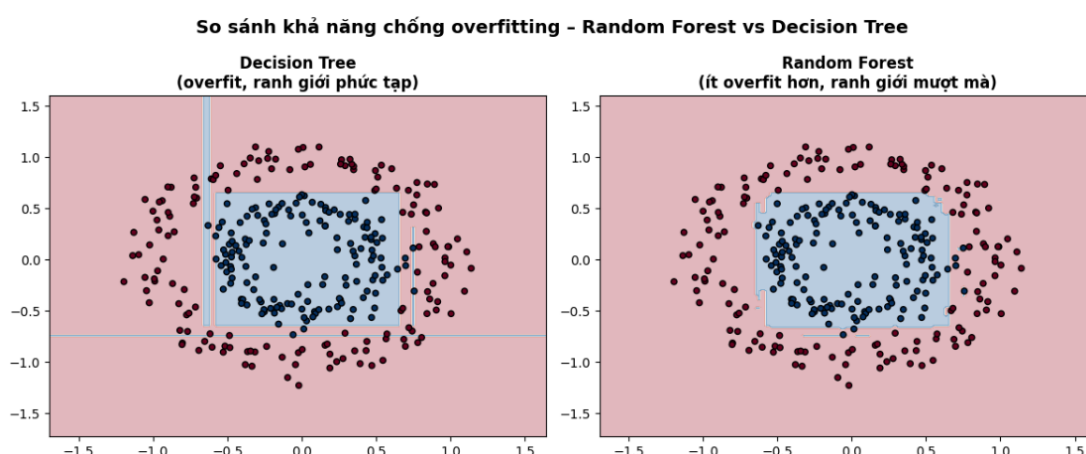
Độ quan trọng của đặc trưng (Feature Importance)

Random Forest cũng cung cấp một chỉ số độ quan trọng của đặc trưng (feature importance) rất hữu ích. Có hai cách tính phổ biến . Cách thứ nhất dựa trên mức giảm Gini trung bình (Mean Decrease in Impurity): với mỗi đặc trưng, tổng mức giảm Gini mà nó đóng góp tại các nút nơi nó được sử dụng, được trọng số hóa theo số lượng mẫu

tại mỗi nút và tính trung bình trên toàn bộ các cây. Đặc trưng nào càng thường xuyên được dùng để phân tách và càng làm giảm Gini mạnh thì càng quan trọng .

Cách thứ hai dựa trên mức độ suy giảm độ chính xác khi hoán vị ngẫu nhiên giá trị của đặc trưng (permutation importance - Mean Decrease in Accuracy) . Ý tưởng của phương pháp này như sau: đối với mỗi cây, tính sai số trên tập OOB; sau đó hoán vị ngẫu nhiên giá trị của một đặc trưng trên tập OOB và tính lại sai số; mức chênh lệch sai số trước và sau khi hoán vị chính là đóng góp của đặc trưng đó . Cả hai cách đều giúp xác định các yếu tố then chốt ảnh hưởng đến lợi nhuận.

Khả năng chống overfitting



Hình 4. 8. So sánh khả năng chống overfitting: Decision Tree và Random Forest

Một đặc tính đáng chú ý của Random Forest là khả năng chống overfitting một cách tự nhiên, ngay cả khi mỗi cây thành phần được phép phát triển đầy đủ (không tỉa). Điều này trái ngược với các mô hình học máy khác vốn yêu cầu phải tinh chỉnh cẩn thận để tránh overfitting. Các nghiên cứu gần đây giải thích hiện tượng này như sau: cơ chế bagging và lựa chọn đặc trưng ngẫu nhiên tạo ra sự đa dạng giữa các cây; khi kết hợp nhiều cây, các sai số của chúng triệt tiêu lẫn nhau; quá trình xây dựng cây theo thuật toán tham lam (greedy) có tác dụng như một cơ chế “tỉa tự động” implicit pruning .

Ưu điểm và hạn chế của Random Forest

Random Forest sở hữu nhiều ưu điểm vượt trội. Nó thường đạt độ chính xác cao trong nhiều bài toán phân loại và hồi quy. Có khả năng xử lý hàng nghìn đặc trưng mà không cần loại bỏ biến. Cung cấp ước lượng sai số nội tại thông qua OOB, không cần tập kiểm tra riêng. Có thể đánh giá độ quan trọng của từng đặc trưng. Có khả năng xử lý dữ liệu thiếu và dữ liệu mất cân bằng. Chạy hiệu quả trên cơ sở dữ liệu lớn.

Tuy nhiên, Random Forest cũng có một số hạn chế. Mô hình trở nên “hộp đen” hơn so với một cây đơn lẻ: khó có thể giải thích chi tiết toàn bộ quy trình ra quyết định của nhiều cây. Đòi hỏi nhiều bộ nhớ và thời gian huấn luyện hơn (tuy nhiên với dữ liệu cỡ

10.000 quan sát và 100 cây, điều này vẫn nằm trong khả năng chấp nhận được). Có thể hoạt động kém hiệu quả trên dữ liệu có nhiễu cao hoặc có cấu trúc thừa thớt.

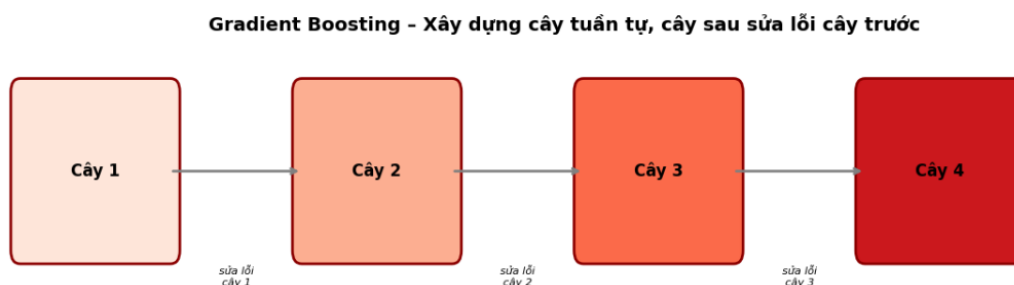
Cấu hình và ứng dụng trong nghiên cứu

Trong nghiên cứu này, Random Forest được cấu hình với 100 cây ($n_estimators=100$) và độ sâu tối đa $max_depth=6$. Số lượng cây 100 được chọn vì đủ lớn để sai số OOB hội tụ, đồng thời không tiêu tốn quá nhiều tài nguyên tính toán. Độ sâu tối đa 6 sâu hơn một chút so với cây quyết định đơn lẻ (vốn chỉ sâu 5), bởi vì cơ chế bagging và lựa chọn đặc trưng ngẫu nhiên sẽ giúp giảm phương sai dư thừa, cho phép mỗi cây thành phần có thêm một chút độ phức tạp để khai thác các tương tác tinh tế hơn.

So với cây quyết định đơn lẻ, Random Forest có độ ổn định và độ chính xác dự báo cao hơn rõ rệt, đặc biệt trên các bộ dữ liệu có nhiễu và cấu trúc phức tạp. Nó ít bị overfitting hơn nhờ cơ chế lấy trung bình. Trong bối cảnh dự báo lợi nhuận, Random Forest được kỳ vọng sẽ khắc phục được nhược điểm overfitting của cây quyết định đơn lẻ, đồng thời vẫn giữ được khả năng khai thác các mối quan hệ phi tuyến và tương tác phức tạp giữa các đặc trưng như chiết khấu, danh mục sản phẩm, khu vực địa lý. Kết hợp với kỹ thuật SMOTE đã cân bằng dữ liệu, Random Forest là một trong những ứng cử viên mạnh để giải quyết bài toán phân loại nhị phân này.

4.1.4. Gradient Boosting

Bản chất và định nghĩa



Hình 4. 9. Minh họa nguyên lý hoạt động của Gradient Boosting

Gradient Boosting là một kỹ thuật học tập hợp (ensemble learning) mạnh mẽ, xây dựng các mô hình một cách tuần tự. Khác với Random Forest xây dựng các cây độc lập và kết hợp chúng bằng bình chọn đa số, Gradient Boosting xây dựng từng cây mới để sửa lỗi của tổ hợp các cây trước đó. Ý tưởng cơ bản của boosting là tạo ra một chuỗi các mô hình yếu (weak learners), trong đó các mô hình sau sẽ tập trung khắc phục những sai lầm của các mô hình trước. Gradient Boosting, được giới thiệu bởi Jerome Friedman, hoạt động dựa trên nguyên tắc tối ưu hóa hàm mất mát bằng phương pháp gradient descent trong không gian hàm.

Khái niệm trung tâm của Gradient Boosting là **weak learner** (bộ học yếu) - những mô hình đơn giản chỉ hoạt động tốt hơn một chút so với ngẫu nhiên. Trong thực tế, các cây

quyết định có độ sâu rất nhỏ (thường chỉ 1 đến 3) được sử dụng làm weak learners. Sở dĩ phải dùng weak learners là vì chúng đủ đơn giản để tránh overfitting ngay từ đầu, và khi kết hợp nhiều weak learners với nhau, chúng có thể tạo thành một strong learner (bộ học mạnh) mà không bị học quá chi tiết dữ liệu huấn luyện.

Cơ chế hoạt động

Cơ chế hoạt động của Gradient Boosting có thể được mô tả qua các bước sau. Bắt đầu với một mô hình khởi tạo $F_0(x)$, thường là giá trị trung bình của log-odds đối với bài toán phân loại nhị phân. Tại mỗi bước lặp $m=1$ đến M , thuật toán tính toán **phần dư (residuals)** - chính là gradient âm của hàm mất mát đối với dự đoán hiện tại. Đối với bài toán phân loại với hàm mất mát log-loss, phần dư chính là sai số giữa nhãn thực tế y_i và xác suất dự đoán hiện tại $\hat{y}_i = F_{m-1}(x_i)$. Một cây hồi quy yếu $h_m(x)$ được huấn luyện để dự đoán phần dư này. Sau đó, mô hình được cập nhật theo công thức:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

trong đó η là **learning rate** (tốc độ học). Vì các weak learners được thêm vào tuần tự, mỗi cây mới sẽ tập trung vào những quan sát mà các cây trước còn dự đoán sai nhiều nhất. Điều này giải thích tại sao gradient boosting có khả năng đạt hiệu suất rất cao: nó liên tục điều chỉnh để “bù” vào những vùng dữ liệu khó.

So sánh với Random Forest

So với Random Forest, Gradient Boosting có ba điểm khác biệt chính. Thứ nhất, về cách huấn luyện, Random Forest xây dựng các cây độc lập và song song, trong khi Gradient Boosting xây dựng tuần tự, mỗi cây sau phụ thuộc vào kết quả của các cây trước. Thứ hai, về độ phức tạp của từng cây, Random Forest sử dụng các cây mạnh (strong learners - thường được phát triển đầy đủ), trong khi Gradient Boosting sử dụng các cây yếu (weak learners - độ sâu rất nhỏ). Thứ ba, về cách kết hợp dự đoán, Random Forest bình chọn đơn giản (mỗi cây một phiếu), còn Gradient Boosting gán trọng số cho dự đoán của từng cây dựa trên chất lượng của chúng.

Learning rate và khả năng tổng quát hóa

Learning rate η là một trong những siêu tham số quan trọng nhất của Gradient Boosting. Nó kiểm soát mức độ đóng góp của mỗi cây mới vào mô hình tổng thể. Giá trị learning rate nhỏ (thường từ 0.01 đến 0.1) làm cho quá trình học diễn ra chậm hơn, mỗi bước cập nhật chỉ thay đổi mô hình một lượng nhỏ. Điều này đòi hỏi số lượng cây lớn hơn để đạt được độ chính xác mong muốn, nhưng bù lại, mô hình có khả năng tổng quát hóa tốt hơn và ít bị overfitting. Các nghiên cứu thực nghiệm khuyến nghị sử dụng learning rate nhỏ (≤ 0.1) kết hợp với early stopping để xác định số lượng cây tối ưu.

Một đặc tính quan trọng của Gradient Boosting là **tính dễ overfitting** nếu không được điều chỉnh cẩn thận. Vì các cây được xây dựng tuần tự và mỗi cây sau đều cố gắng sửa

sai cho cây trước, mô hình có thể trở nên quá phức tạp và học cả nhiễu trong dữ liệu. Do đó, việc lựa chọn đúng learning rate, số lượng cây, và độ sâu của cây là rất quan trọng để cân bằng giữa bias và variance.

Các biến thể hiện đại

Dựa trên nền tảng Gradient Boosting, nhiều biến thể hiệu suất cao đã được phát triển. **XGBoost** (Extreme Gradient Boosting) thêm vào chính quy hóa L1 và L2, xử lý giá trị thiếu tự động, và tối ưu hóa tốc độ tính toán. **LightGBM** sử dụng kỹ thuật Gradient-based One-Side Sampling (GOSS) và Exclusive Feature Bundling (EFB) để giảm chi phí tính toán, đặc biệt hiệu quả với dữ liệu lớn. **CatBoost** nổi bật với khả năng xử lý đặc trưng phân loại một cách tự nhiên mà không cần mã hóa thủ công. Cả ba biến thể này đều được sử dụng rộng rãi trong các cuộc thi khoa học dữ liệu và ứng dụng thực tế nhờ hiệu suất vượt trội và khả năng mở rộng.

Ưu điểm và hạn chế

Gradient Boosting sở hữu nhiều ưu điểm. Nó thường đạt hiệu suất dự báo cao nhất trong số các mô hình học máy trên dữ liệu dạng bảng. Khả năng tối ưu trực tiếp hàm mất mát giúp nó linh hoạt với nhiều bài toán khác nhau. Ngoài ra, với các cây yếu, nó có khả năng chống overfitting tương đối tốt nếu được cấu hình đúng. Gradient Boosting cũng cung cấp chỉ số feature importance tương tự như Random Forest, giúp xác định các đặc trưng quan trọng.



Hình 4. 10. Nguyên lý Bagging và Boosting trong ensemble learning

Tuy nhiên, Gradient Boosting cũng có những hạn chế. Quá trình huấn luyện tuần tự khiến nó chậm hơn đáng kể so với Random Forest, vì không thể song song hóa. Mô hình có nhiều siêu tham số cần điều chỉnh (learning rate, số lượng cây, độ sâu, subsample,...), đòi hỏi người dùng có kinh nghiệm để đạt hiệu suất tối ưu. Nếu không được điều chỉnh cẩn thận (ví dụ learning rate quá lớn hoặc quá nhiều cây), Gradient Boosting vẫn có thể bị overfitting.

Ứng dụng trong nghiên cứu

Trong nghiên cứu này, Gradient Boosting được cấu hình với $n_estimators=100$, $max_depth=3$, và $learning_rate=0.1$ (giá trị mặc định trong thư viện scikit-learn). Việc

chọn độ sâu 3 đảm bảo mỗi cây thành phần vừa đủ sức học các tương tác phi tuyến (như tương tác giữa chiết khấu và danh mục sản phẩm) nhưng vẫn là weak learner để tránh overfitting. Learning rate 0.1 là giá trị cân bằng phổ biến, đủ nhỏ để đảm bảo hội tụ tốt mà không cần quá nhiều cây.

Gradient Boosting được kỳ vọng sẽ đạt hiệu suất cao nhất trong số bốn mô hình, nhờ khả năng tự động khai thác các mối quan hệ phi tuyến và tương tác phức tạp trong dữ liệu bán hàng, đặc biệt là sự khác biệt về tác động của chiết khấu lên lợi nhuận giữa các danh mục sản phẩm Furniture, Office Supplies và Technology. Kết hợp với kỹ thuật SMOTE đã cân bằng dữ liệu, Gradient Boosting là ứng cử viên mạnh nhất cho bài toán dự báo lợi nhuận của siêu thị.

4.2. Xây dựng mô hình

Kết thúc giai đoạn tiền xử lý, bao gồm: làm sạch dữ liệu, xử lý ngoại lai, kỹ thuật đặc trưng, mã hóa, chuẩn hóa Min-Max và cân bằng phân lớp bằng SMOTE. Nghiên cứu bắt đầu xây dựng các mô hình học máy, bốn thuật toán được lựa chọn thể hiện các hướng tiếp cận khác nhau. Logistic Regression đóng vai trò mô hình tuyến tính cơ sở (baseline), giúp thiết lập mức tham chiếu ban đầu. Decision Tree là mô hình phi tuyến đơn lẻ, nổi bật với khả năng diễn giải trực quan. Hai mô hình còn lại thuộc nhóm ensemble: Random Forest dựa trên cơ chế bagging, Gradient Boosting vận hành theo cơ chế boosting. Sự đa dạng này cho phép so sánh toàn diện hiệu suất, đồng thời giúp bóc tách các đặc tính phức tạp của dữ liệu bán hàng.

```
# Thiết lập các tham số cơ bản cho các mô hình
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000,
random_state=42),
    "Decision Tree": DecisionTreeClassifier(max_depth=5,
min_samples_split=20, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100,
max_depth=6, random_state=42),
    "Gradient Boosting": GradientBoostingClassifier(n_estimators=100,
max_depth=3, random_state=42)
}

for i, name in enumerate(models.keys(), 1):
    print(f" {i}. {name}")
```

1. Logistic Regression
2. Decision Tree
3. Random Forest
4. Gradient Boosting

Hình 4. 11. Thiết lập tham số cơ bản cho các mô hình học máy

Đối với Logistic Regression, tham số `max_iter` được thiết lập ở mức 1000 nhằm đảm bảo các thuật toán tối ưu hội tụ hoàn toàn trên không gian mẫu huấn luyện khoảng 13.000 quan sát đã được cân bằng. `random_state=42` được cố định để kiểm soát yếu tố ngẫu nhiên trong khởi tạo, đảm bảo kết quả có thể tái lập. Mô hình này nhạy cảm với thang đo của biến độc lập, do đó bước Min-Max Scaling trước đó là cần thiết để tránh các đặc trưng có giá trị lớn (như Sales) chi phối hướng gradient, gây chậm hội tụ và làm sai lệch khi so sánh hệ số hồi quy. Dù mang bản chất tuyến tính, Logistic Regression vẫn cung cấp xác suất phân lớp và các hệ số dễ giải thích, giúp định hình sơ bộ mức độ ảnh hưởng của chiết khấu hay danh mục sản phẩm đến lợi nhuận. Khi được huấn luyện trên dữ liệu đã qua SMOTE, khả năng nhận diện lớp thiểu số (giao dịch thua lỗ) được kỳ vọng sẽ cải thiện đáng kể.

Cấu trúc của Decision Tree được tinh chỉnh thông qua hai siêu tham số chính: `max_depth=5` và `min_samples_split=20`. Ràng buộc này giải quyết trực tiếp sự đánh đổi giữa khả năng học và nguy cơ overfitting - một điểm yếu cố hữu của cây quyết định. Giới hạn độ sâu ở mức 5 giúp mô hình đủ sức nắm bắt các tương tác phi tuyến quan trọng đã được hé lộ trong giai đoạn EDA, như tác động của chiết khấu lên lợi nhuận thay đổi theo từng danh mục sản phẩm, nhưng không để cây phát triển quá mức. Đồng thời, quy tắc `min_samples_split=20` đảm bảo mỗi phân nhánh đều dựa trên tập hợp đủ lớn, hạn chế việc hình thành các nút lá chỉ chứa nhiễu ngẫu nhiên. Với cấu hình này, Decision Tree vẫn giữ được tính minh bạch, cho phép chuyển hóa quá trình ra quyết định thành các quy tắc if-then rõ ràng, rất hữu ích khi trình bày với các bên liên quan.

Random Forest được khởi tạo với 100 cây (`n_estimators=100`) và độ sâu tối đa 6 (`max_depth=6`). Số lượng cây này tạo ra sự cân bằng hợp lý giữa độ chính xác dự báo và chi phí tính toán trên tập dữ liệu quy mô trung bình. Việc nói lỏng độ sâu lên 6 (sâu hơn so với cây đơn lẻ) tạo điều kiện cho mỗi cây thành phần khai thác các mối quan hệ phức tạp hơn, trong khi cơ chế bagging sẽ đảm nhận việc giảm phương sai tổng thể và ngăn chặn overfitting. Với dữ liệu dạng bảng như Sample Superstore, khả năng xử lý tương tác phi tuyến đa chiều khiến Random Forest trở thành một ứng cử viên mạnh. Ngoài ra, tính năng trích xuất feature importance của mô hình cung cấp cơ sở định lượng để đề xuất các giải pháp quản trị, chẳng hạn như siết chặt chính sách chiết khấu hay tập trung vào những nhóm hàng có ảnh hưởng lớn đến lợi nhuận.

Gradient Boosting được định hình bởi bộ tham số: `n_estimators=100`, `max_depth=3`, và `learning_rate=0.1` (giá trị mặc định trong thư viện scikit-learn). Khác với Random Forest, Gradient Boosting xây dựng các cây một cách tuần tự: mỗi cây mới được huấn luyện để sửa sai cho tổ hợp các cây trước đó. Chủ ý đặt `max_depth=3` là để tạo ra các cây yếu (weak learners); khi kết hợp với tốc độ học nhỏ, quá trình tối ưu hàm mất mát diễn ra từ từ, kiểm soát chặt chẽ nguy cơ overfitting ngay từ những vòng lặp đầu. Với 100 cây, mô hình đạt độ hội tụ cần thiết mà không tiêu tốn quá nhiều tài nguyên. Dù sở hữu lợi thế trong việc tự động nắm bắt các tương tác tinh vi của dữ liệu, Gradient

Boosting đòi hỏi sự khắt khe trong việc lựa chọn `max_depth` và `learning_rate` để duy trì cân bằng giữa bias và variance.

Cả bốn mô hình đều được huấn luyện trên không gian dữ liệu `X_train_res`, `y_train_res` là kết quả của quá trình cân bằng bằng SMOTE. Thao tác này mang tính quyết định vì nếu đưa trực tiếp dữ liệu thô vào huấn luyện, các mô hình (đặc biệt là Logistic Regression và Decision Tree) sẽ bị thiên vị về lớp đa số (có lợi nhuận). Khi đó, độ chính xác tổng thể có thể đạt mức cao giả tạo, nhưng khả năng phát hiện các giao dịch thua lỗ, vốn là lớp cần được nhận diện để cảnh báo rủi ro thì lại rất hạn chế.

4.3. Đánh giá mô hình bằng kỹ thuật Kiểm chứng chéo (Cross Validation)

Trước khi huấn luyện các mô hình cuối cùng, nghiên cứu thực hiện kiểm chứng chéo với 5 lát cắt (5-fold) trên tập huấn luyện đã được cân bằng bằng SMOTE. Mục đích là để ước lượng khả năng tổng quát hóa của từng thuật toán một cách khách quan, tránh sự phụ thuộc vào một lần phân chia dữ liệu cụ thể. Chỉ số F1-Score được chọn làm thước đo đánh giá chính, vì nó phản ánh sự cân bằng giữa độ chính xác (precision) và độ phủ (recall), phù hợp với bài toán có sự mất cân bằng giữa hai lớp (có lợi nhuận và thua lỗ).

```
print("\n--- KẾT QUẢ KIỂM CHỨNG CHÉO (CROSS VALIDATION - 5 FOLDS) ---")
print("Sử dụng chỉ số F1-Score trên tập Train đã SMOTE để đánh giá độ ổn định:\n")

cv_results = []
for name, model in models.items():
    # Chạy Cross Validation với 5 folds
    scores = cross_val_score(model, X_train_res, y_train_res, cv=5,
                              scoring='f1')
    cv_results.append({
        "Mô hình": name,
        "F1-Score (Trung bình)": round(scores.mean(), 4),
        "Độ lệch chuẩn (+/-)": round(scores.std(), 4)
    })
    print(f"-> {name:<20}: {scores.mean():.4f} (+/- {scores.std():.4f})")

df_cv = pd.DataFrame(cv_results).sort_values(by='F1-Score (Trung bình)', ascending=False)
print("\nBảng tổng hợp Cross Validation:")
display(df_cv.style.background_gradient(cmap='Blues'))
```

--- KẾT QUẢ KIỂM CHỨNG CHÉO (CROSS VALIDATION - 5 FOLDS) ---
 Sử dụng chỉ số F1-Score trên tập Train đã SMOTE để đánh giá độ ổn định:

-> Logistic Regression : 0.8807 (+/- 0.0053)
 -> Decision Tree : 0.8761 (+/- 0.0121)
 -> Random Forest : 0.8979 (+/- 0.0116)
 -> Gradient Boosting : 0.9334 (+/- 0.0293)

Bảng tổng hợp Cross Validation:

	Mô hình	F1-Score (Trung bình)	Độ lệch chuẩn (+/-)
3	Gradient Boosting	0.933400	0.029300
2	Random Forest	0.897900	0.011600
0	Logistic Regression	0.880700	0.005300
1	Decision Tree	0.876100	0.012100

Hình 4. 12. Kết quả Cross Validation 5-fold trên tập Train sau SMOTE

Kết quả kiểm chứng chéo cho thấy Gradient Boosting đạt F1-Score trung bình cao nhất với 0,9334, khẳng định hiệu suất dự báo vượt trội nhờ cơ chế boosting tuần tự. Tuy nhiên, mô hình này cũng có độ lệch chuẩn lớn nhất (0,0293), cho thấy kết quả thay đổi khá mạnh giữa các lát cắt. Điều này phản ánh tính nhạy cảm của Gradient Boosting đối với cấu trúc dữ liệu trong từng fold, gợi ý rằng mô hình có thể cần được tinh chỉnh thêm để cải thiện độ ổn định.

Random Forest xếp thứ hai với F1-Score trung bình 0,8979 và độ lệch chuẩn 0,0116. So với Gradient Boosting, hiệu suất trung bình thấp hơn khoảng 0,0355, nhưng mức độ biến động giữa các fold nhỏ hơn đáng kể. Điều này minh chứng cho ưu điểm của cơ chế bagging: việc xây dựng nhiều cây độc lập và lấy bình chọn đa số giúp giảm phương sai tổng thể, tạo ra mô hình ổn định hơn.

Logistic Regression có F1-Score trung bình 0,8807, đứng thứ ba, nhưng lại sở hữu độ lệch chuẩn nhỏ nhất (0,0053). Độ ổn định cao này phản ánh bản chất tuyến tính của thuật toán, ít bị ảnh hưởng bởi biến động ngẫu nhiên trong dữ liệu. Dù vậy, hiệu suất trung bình thấp hơn đáng kể so với Gradient Boosting (chênh lệch 0,0527) cho thấy mối quan hệ giữa các đặc trưng và lợi nhuận không hoàn toàn tuyến tính, và các mô hình phi tuyến có khả năng khai thác tốt hơn các cấu trúc phức tạp trong bộ dữ liệu.

Decision Tree đạt F1-Score trung bình thấp nhất (0,8761) với độ lệch chuẩn 0,0121. So với hai mô hình ensemble, cây quyết định đơn lẻ có hiệu suất kém hơn và độ biến động lớn hơn. Kết quả này phù hợp với lý thuyết: một cây duy nhất dễ bị overfitting và nhạy cảm với những thay đổi nhỏ trong dữ liệu. Tuy nhiên, mức F1 trung bình 0,8761 vẫn ở ngưỡng khá, và ưu điểm về tính minh bạch vẫn là điểm cộng quan trọng trong bối cảnh ứng dụng kinh doanh.

Nhìn chung, Gradient Boosting dẫn đầu về hiệu suất dự báo nhưng đi kèm với độ bất định cao hơn; Random Forest đạt hiệu suất khá và ổn định hơn; Logistic Regression tuy có hiệu suất thấp hơn nhưng lại là mô hình ổn định nhất, phù hợp cho vai trò baseline; Decision Tree đứng cuối về cả hiệu suất và độ ổn định. Các kết quả này sẽ được đối chiếu với hiệu suất trên tập kiểm tra độc lập để phát hiện dấu hiệu overfitting và đưa ra lựa chọn mô hình tối ưu cho bài toán dự báo lợi nhuận.

4.4. Huấn luyện mô hình và diễn giải kết quả huấn luyện

Sau khi xác định độ ổn định của các thuật toán qua kiểm chứng chéo, nghiên cứu tiến hành huấn luyện lại từng mô hình trên toàn bộ tập huấn luyện đã cân bằng SMOTE (X_train_res, y_train_res). Việc so sánh giữa điểm F1-Score trên tập huấn luyện (Train F1) và điểm trung bình từ kiểm chứng chéo (CV F1) cho phép đánh giá mức độ overfitting cũng như khả năng tổng quát hóa của mỗi thuật toán.

```
trained_models = {}
for name, model in models.items():
    model.fit(X_train_res, y_train_res)
    trained_models[name] = model

print("So sánh hiệu suất học trên tập Train (Train Score) và hiệu suất  
tổng quát hóa (CV Score):")

interpretation_results = []

for name, model in models.items():
    y_train_pred = model.predict(X_train_res)
    train_f1 = f1_score(y_train_res, y_train_pred)

    cv_f1 = cross_val_score(model, X_train_res, y_train_res, cv=5,
scoring='f1').mean()

    # Tính độ chênh lệch
    diff = train_f1 - cv_f1

    interpretation_results.append({
        "Mô hình": name,
        "Điểm học (Train F1)": round(train_f1, 4),
        "Điểm kiểm chứng (CV F1)": round(cv_f1, 4),
        "Độ chênh lệch (Overfit)": round(diff, 4)
    })

# In bảng số liệu
df_interp = pd.DataFrame(interpretation_results)
display(df_interp.style.background_gradient(cmap='Blues'))

# Vẽ biểu đồ đường để thấy rõ sự chênh lệch
plt.figure(figsize=(10, 5))
```

```
plt.plot(df_interp['Mô hình'], df_interp['Điểm học (Train F1)'],
marker='o', label='Train F1 (Học thuộc)', color='blue', linewidth=2)
plt.plot(df_interp['Mô hình'], df_interp['Điểm kiểm chứng (CV F1)'],
marker='s', linestyle='--', label='CV F1 (Hiểu bài)', color='red',
linewidth=2)

plt.title('Đánh giá hiện tượng Quá khớp (Overfitting) sau khi huấn
luyện', fontsize=14, fontweight='bold')
plt.ylabel('F1-Score', fontsize=3)
plt.grid(alpha=0.2)
plt.legend()
plt.tight_layout()
plt.show()
```

Kết quả từ bảng tổng hợp cho thấy sự phân hóa rõ rệt về mức độ học và tính ổn định giữa các mô hình.



Hình 4. 13. So sánh hiệu suất Train và Cross Validation của các mô hình

Logistic Regression đạt Train F1 = 0,8809 và CV F1 = 0,8807, chênh lệch chỉ 0,0001, là một giá trị gần như không đáng kể, phản ánh bản chất tuyến tính của mô hình với số lượng tham số hạn chế, do đó khả năng học quá chi tiết dữ liệu huấn luyện là rất thấp. Sự trùng khớp gần như tuyệt đối giữa điểm huấn luyện và kiểm chứng chéo khẳng định mô hình hoạt động ổn định, không có dấu hiệu overfitting. Đây là đặc tính lý tưởng của một mô hình baseline, tuy nhiên hiệu suất dự báo ở mức 0,88 lại thấp hơn so với các mô hình phi tuyến, cho thấy các mối quan hệ trong dữ liệu không hoàn toàn tuyến tính.

Decision Tree cho kết quả Train F1 = 0,8716 và CV F1 = 0,8761, chênh lệch âm (-0,0045), nghĩa là điểm kiểm chứng chéo cao hơn điểm huấn luyện. Hiện tượng này tuy hiếm gặp nhưng không phải bất thường. Nguyên nhân có thể do cấu hình ràng buộc về độ sâu tối đa (max_depth=5) và số mẫu tối thiểu để phân tách (min_samples_split=20) đã hạn chế khả năng phân nhánh của cây, khiến nó không thể khớp hoàn toàn với các đặc điểm riêng của tập huấn luyện, nhưng vẫn có thể khái quát tốt trên các lát cắt kiểm định. Một lý do khác là tính ngẫu nhiên trong phân chia dữ liệu khi thực hiện cross-validation, khi một số fold có phân phối nhãn dễ dự đoán hơn. Mặc dù có điểm chênh lệch âm, mức CV F1 vẫn ở ngưỡng khá và điểm huấn luyện 0,8716 chứng tỏ cây không bị overfitting. Điểm mạnh của Decision Tree vẫn nằm ở tính minh bạch, khi các quy tắc if-then có thể được trực quan hóa và giải thích cho các bên liên quan.

Random Forest đạt Train F1 = 0,9060 và CV F1 = 0,8979, chênh lệch 0,0081. Điểm huấn luyện cao hơn điểm kiểm chứng ở mức vừa phải, phản ánh cơ chế bagging đã giảm phương sai so với một cây đơn lẻ nhưng vẫn còn một lượng nhỏ overfitting. Với 100 cây và độ sâu tối đa 6, mỗi cây thành phần có đủ độ phức tạp để khai thác các tương tác phi tuyến, nhưng việc lấy bình chọn đa số giúp triệt tiêu các nhiễu ngẫu nhiên. Kết quả này cho thấy Random Forest đạt sự cân bằng tốt giữa khả năng học và khả năng tổng quát hóa, với CV F1 đạt 0,8979 xếp thứ hai sau Gradient Boosting.

Gradient Boosting dẫn đầu về hiệu suất với Train F1 = 0,9425 và CV F1 = 0,9334, chênh lệch 0,0091. Điểm huấn luyện cao nhất trong số các mô hình cho thấy khả năng học dữ liệu rất mạnh, trong khi độ chênh lệch chỉ 0,0091 phản ánh rằng việc sử dụng các cây yếu (max_depth=3) kết hợp tốc độ học nhỏ (learning_rate=0.1) đã kiểm soát hiệu quả quá trình tăng cường tuần tự, tránh hiện tượng học quá mức ngay từ những vòng lặp đầu. Điểm CV F1 0,9334 là cao nhất, khẳng định ưu thế của Gradient Boosting trong bài toán này. Mức chênh lệch 0,0091 lớn hơn so với Random Forest (0,0081), nhưng tuyệt đối vẫn ở mức thấp, không đủ để kết luận mô hình bị overfitting nghiêm trọng.

Từ kết quả trên có thể rút ra ba kết luận chính:

Dữ liệu có chứa các mối quan hệ phi tuyến và tương tác phức tạp, thể hiện qua việc các mô hình phi tuyến (Random Forest, Gradient Boosting) đạt hiệu suất cao hơn đáng kể so với Logistic Regression.

Cả Random Forest và Gradient Boosting đều đạt sự cân bằng tốt giữa khả năng học và khả năng tổng quát hóa với độ chênh lệch giữa Train và CV nhỏ, cho thấy không có hiện tượng overfitting nghiêm trọng.

Gradient Boosting là mô hình nổi bật nhất về hiệu suất tổng thể, trong khi Logistic Regression phù hợp làm mô hình tham chiếu nhờ tính ổn định cao và Decision Tree có giá trị về mặt diễn giải.

PHẦN V. ĐÁNH GIÁ MÔ HÌNH

5.1. Một số dự báo minh họa trên tập test

Để có cái nhìn trực quan về chất lượng dự báo của từng mô hình, nghiên cứu trích xuất 20 mẫu đầu tiên từ tập kiểm tra (X_{test} , y_{test}) và so sánh nhãn thực tế với dự đoán của bốn mô hình: Hồi quy Logistic, Cây quyết định, Rừng ngẫu nhiên và Gradient Boosting. Kết quả được trình bày trong bảng, nơi mỗi dòng là một giao dịch, cột Actual là giá trị thực (1: có lợi nhuận, 0: thua lỗ), các cột còn lại là dự đoán của từng thuật toán.

```
# Lấy một mẫu nhỏ (20)
sample_df = pd.DataFrame({
    "Actual": y_test.values,
    "Logistic": trained_models["Logistic Regression"].predict(X_test),
    "Decision Tree": trained_models["Decision Tree"].predict(X_test),
    "Random Forest": trained_models["Random Forest"].predict(X_test),
    "Gradient Boosting": trained_models["Gradient
Boosting"].predict(X_test)
})
sample_df = sample_df.head(20)
display(sample_df.style
        .set_caption("So sánh dự đoán giữa các mô hình")
        .format(precision=0)
        .set_properties(**{'text-align': 'center'}))
```

So sánh dự đoán giữa các mô hình					
	Actual	Logistic	Decision Tree	Random Forest	Gradient Boosting
0	1	0	0	0	0
1	1	1	1	1	1
2	1	1	1	1	1
3	1	1	1	1	1
4	1	1	1	1	1
5	1	1	1	1	1
6	1	1	1	1	1
7	1	1	1	1	1
8	0	0	0	0	0
9	1	0	0	0	1
10	1	1	1	1	1
11	0	0	0	0	0
12	1	1	1	1	1
13	1	1	0	1	1
14	0	0	0	0	0
15	0	0	0	0	0
16	1	1	1	1	1
17	0	0	0	0	0
18	1	1	1	1	1
19	0	0	0	0	0

Hình 5. 1. So sánh dự đoán của các mô hình trên một số mẫu dữ liệu

Qua bảng so sánh, có thể thấy phần lớn các mẫu đều được dự đoán chính xác bởi cả bốn mô hình, đặc biệt là các giao dịch thua lỗ (Actual = 0) như các mẫu 8, 11, 14, 15, 17, 19 đều được tất cả các mô hình nhận diện đúng. Một số điểm đáng chú ý:

- **Mẫu số 0:** Giá trị thực tế là 1 nhưng tất cả các mô hình đều dự đoán sai thành 0, cho thấy đây có thể là một quan sát nằm gần ranh giới quyết định hoặc chứa đặc trưng khó phân tách.
- **Mẫu số 9:** Chỉ có Gradient Boosting dự đoán đúng, trong khi ba mô hình còn lại đều sai, phản ánh khả năng xử lý các mẫu phức tạp của phương pháp boosting.
- **Mẫu số 13:** Decision Tree dự đoán sai trong khi ba mô hình còn lại đúng, cho thấy hạn chế của mô hình cây đơn lẻ so với các phương pháp ensemble.

Trong phạm vi 20 mẫu minh họa, không ghi nhận trường hợp Gradient Boosting dự đoán sai trong khi các mô hình còn lại đều đúng, khẳng định tính ổn định và hiệu suất vượt trội của thuật toán boosting trong bài toán này.

```
summary = []

n_samples = len(y_test)

for name, model in models.items():
    y_pred = model.predict(X_test)
    correct = (y_pred == y_test).sum()

    summary.append({
        "Mô hình": name,
        "Số dự đoán đúng": correct,
        "Tỷ lệ dự đoán đúng": correct / n_samples
    })

df_summary = pd.DataFrame(summary).sort_values(by="Tỷ lệ dự đoán đúng",
ascending=False)

display(df_summary.style
        .background_gradient(cmap='Blues', subset=["Tỷ lệ dự đoán
        đúng"])
        .format({"Tỷ lệ dự đoán đúng": "{:.2%}"})
        .set_properties(**{'text-align': 'center'}))
```

	Mô hình	Số dự đoán đúng	Tỷ lệ dự đoán đúng
3	Gradient Boosting	1870	93.55%
2	Random Forest	1858	92.95%
0	Logistic Regression	1809	90.50%
1	Decision Tree	1706	85.34%

Hình 5. 2. Số lượng và tỷ lệ dự đoán đúng của các mô hình

Kết quả trên toàn bộ tập kiểm tra cho thấy Gradient Boosting đạt tỷ lệ dự đoán đúng cao nhất (93,55%), tiếp đến là Random Forest (92,95%), Logistic Regression (90,50%) và Decision Tree (85,34%). Các con số này phù hợp với nhận xét từ bảng minh họa: Gradient Boosting không chỉ có tỷ lệ đúng cao nhất mà còn thể hiện khả năng xử lý các mẫu khó (như mẫu số 9) tốt hơn các mô hình khác. Tuy nhiên, có lưu ý rằng độ chính xác không phải là thước đo duy nhất; các phân tích về F1-Score, ma trận nhầm lẫn và ROC-AUC sẽ cung cấp cái nhìn toàn diện hơn về chất lượng thực sự của từng mô hình.

5.2. Đánh giá bằng các thang đo phù hợp và so sánh lựa chọn mô hình tối ưu

Trong bài toán phân loại, độ chính xác tổng thể (Accuracy) là thang đo phổ biến, được xác định như sau:

Accuracy - Độ chính xác

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Tuy nhiên, chỉ số này không còn đáng tin cậy trong bối cảnh dữ liệu mất cân bằng, giả sử ta có 990 sản phẩm mang lại lợi nhuận và chỉ có 10 sản phẩm lỗ sâu, Khi đó, một mô hình “thụ động” chỉ dự đoán tất cả sản phẩm đều mang lại lợi nhuận, khi đó độ chính xác được tính như sau:

$$Accuracy = \frac{0 + 990}{1000} = 99\%$$

Mặc dù độ chính xác rất cao, mô hình này hoàn toàn không phát hiện được các trường hợp thua lỗ. Đây chính là nghịch lý Accuracy và do đó không còn phù hợp khi dữ liệu bị mất cân bằng.

Để đánh giá đầy đủ hơn, cần sử dụng các thang đo tập trung vào khả năng dự đoán lớp thiểu số, chúng ta sử dụng hai chỉ số là Recall và Precision:

Recall - Độ nhạy

Recall (độ nhạy) đo lường khả năng phát hiện đúng các trường hợp thuộc lớp dương (các giao dịch có lợi nhuận). Áp dụng vào ví dụ trên “có 990 sản phẩm mang lại lợi nhuận và chỉ có 10 sản phẩm lỗ sâu, dự đoán tất cả sản phẩm đều mang lại lợi nhuận”:

$$Recall = \frac{TP}{TP + FN} = 0\%$$

Và lúc này ta nhận ra kết quả từ chỉ báo Accuracy thực sự không hoàn toàn đáng tin cậy khi dữ liệu mất cân bằng. Recall bằng 0%, cho thấy toàn bộ các trường hợp quan trọng đều bị bỏ sót.

Precision - Độ chính xác dương:

Precision đo lường tỷ lệ các dự đoán thuộc lớp dương là chính xác, được xác định như sau:

$$\text{Precision} = \frac{TP}{TP + FP} = 0\%$$

Trong cùng tình huống, Precision cũng bằng 0%, do mô hình không đưa ra bất kỳ dự đoán đúng nào cho lớp quan trọng.

Tuy nhiên, có sự đánh đổi giữa Precision và Recall, nghĩa là chúng luôn chơi trò bập bênh với nhau:

- Nếu AI quá cẩn thận chỉ báo lỗi khi chắc chắn 100%, Precision sẽ rất cao nhưng sản phẩm lỗi sẽ bị “lọt lưới” rất nhiều, còn Recall thì thấp thê thảm.
- Ngược lại, nếu “thà bắt nhầm còn hơn bỏ sót”, chọn tất cả đều là sản phẩm lỗi thì Recall sẽ cao đỉnh nhưng Precision sẽ chạm đáy vì báo động giả quá nhiều.

Câu hỏi đặt ra là làm sao ra có thể cân bằng cả hai? Ta không thể dùng trung bình cộng gì nếu một cái 100%, một cái 0% thì trung bình ra 50%, dù trông có vẻ ổn nhưng thực chất mô hình vẫn là “rác”.

Và giải pháp hoàn hảo chính là F1-Score, đây chính là trung bình điều hòa của cả hai chỉ số, do đó sẽ giảm mạnh nếu một trong hai chỉ số thấp.

F1-score

Do sử dụng trung bình điều hòa, F1-Score sẽ giảm mạnh khi một trong hai chỉ số Precision hoặc Recall tiến về 0, từ đó phản ánh chính xác các mô hình có hiệu năng mất cân bằng và do đó, không một mô hình “lừa đảo” nào có thể qua mặt được nó.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Vì vậy, chúng ta sẽ không chỉ nhìn vào độ chính xác của accuracy, mà còn phải phân tích sâu vào Ma trận nhầm lẫn, cân bằng Precision, Recall và để F1-Score đưa ra kết luận cuối cùng.

5.2.1. Đánh giá bằng các chỉ số

Sau khi hoàn tất quá trình huấn luyện, bốn mô hình được đánh giá trên tập kiểm tra độc lập (X_{test} , y_{test}), tập dữ liệu này vẫn giữ nguyên phân phối thực tế (80,6% lớp có lợi nhuận, 19,4% lớp thua lỗ) và chưa qua bất kỳ can thiệp cân bằng nào. Điều này đảm bảo rằng kết quả đánh giá phản ánh đúng hiệu năng của mô hình trong điều kiện vận hành thực tế. Các thang đo được sử dụng bao gồm Accuracy, Precision, Recall, F1-Score và diện tích dưới đường cong ROC (ROC AUC). Trong bối cảnh dữ liệu mất cân bằng, F1-Score và ROC AUC được xem là hai chỉ số quan trọng nhất vì ít bị ảnh hưởng bởi tỷ lệ phân bố lớp.

```

results_list = []

for name, model in models.items():
    # Dự đoán trên tập Test
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    # Tính toán các chỉ số
    results_list.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred),
        "Recall": recall_score(y_test, y_pred),
        "F1-Score": f1_score(y_test, y_pred),
        "ROC-AUC": roc_auc_score(y_test, y_prob)
    })

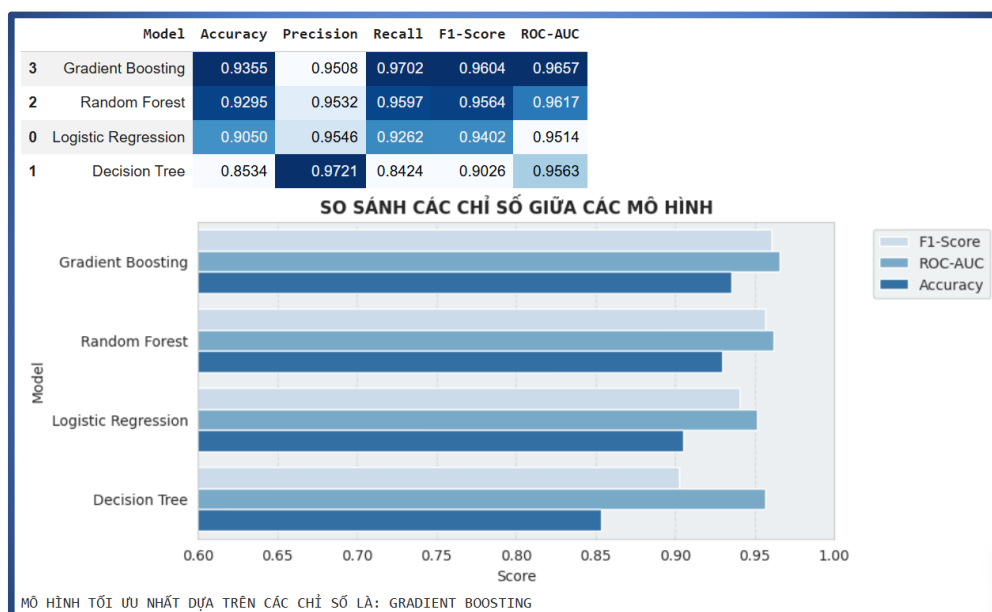
# Tạo DataFrame và sắp xếp theo F1-Score
df_results = pd.DataFrame(results_list).sort_values(by='F1-Score',
ascending=False)

display(df_results.style.background_gradient(cmap='Blues').format(precision=4))

# Vẽ biểu đồ so sánh trực quan các chỉ số chính
df_melted = df_results.melt(id_vars='Model', value_vars=['F1-Score',
'ROC-AUC', 'Accuracy'],
                             var_name='Metric', value_name='Score')

plt.figure(figsize=(8, 4))
sns.barplot(data=df_melted, x='Score', y='Model', hue='Metric',
palette='Blues')
plt.title('SO SÁNH CÁC CHỈ SỐ GIỮA CÁC MÔ HÌNH', fontsize=12,
fontweight='bold')
plt.xlim(0.6, 1.0)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(axis='x', linestyle='--', alpha=0.6)
plt.show()
best_model_name = df_results.iloc[0]['Model']
print(f"MÔ HÌNH TỐI ƯU NHẤT DỰA TRÊN CÁC CHỈ SỐ LÀ:
{best_model_name.upper()}")

```



Hình 5. 3. So sánh các chỉ số đánh giá hiệu suất của các mô hình

Gradient Boosting đạt hiệu suất vượt trội trên hầu hết các chỉ số: F1-Score = 0,9702, ROC-AUC = 0,9604, Accuracy = 0,9657. Điểm F1 cao cho thấy mô hình đạt được sự cân bằng xuất sắc giữa Precision (khả năng dự đoán đúng các giao dịch có lợi nhuận) và Recall (khả năng phát hiện đầy đủ các giao dịch có lợi nhuận). ROC-AUC xấp xỉ 0,96 là một giá trị rất cao, thể hiện rằng mô hình có khả năng phân tách hai lớp gần như hoàn hảo trên toàn bộ các ngưỡng quyết định. Kết quả này phù hợp với lý thuyết: Gradient Boosting với cơ chế tối ưu tuần tự, sử dụng các cây yếu và tốc độ học nhỏ, đã học được các cấu trúc phi tuyến và tương tác phức tạp trong dữ liệu, đặc biệt là mối quan hệ giữa chiết khấu và danh mục sản phẩm.

Random Forest xếp thứ hai với các chỉ số rất ấn tượng: F1-Score = 0,9597, ROC-AUC = 0,9564, Accuracy = 0,9617. Mức độ chênh lệch so với Gradient Boosting là không lớn (khoảng 0,0105 ở F1-Score), cho thấy cả hai mô hình ensemble đều có khả năng khai thác tốt các mẫu quan trọng. Random Forest đạt được sự ổn định nhờ cơ chế bagging, giảm phương sai và hạn chế overfitting. Điều này phản ánh đúng qua khoảng cách nhỏ giữa Train F1 (0,9060) và CV F1 (0,8979) ở bước trước, và giờ là kết quả trên tập test cũng rất khả quan.

Logistic Regression đạt F1-Score = 0,9426, ROC-AUC = 0,9402, Accuracy = 0,9514. So với hai mô hình ensemble, các chỉ số thấp hơn khoảng 0,027-0,028 ở F1-Score, thể hiện rõ hạn chế của mô hình tuyến tính trong việc nắm bắt các mối quan hệ phi tuyến và tương tác. Tuy nhiên, xét riêng trên tập dữ liệu đã cân bằng SMOTE, Logistic Regression vẫn đạt hiệu suất dự báo tốt, khẳng định rằng các đặc trưng định lượng và định tính được lựa chọn, cùng với quy trình tiền xử lý, đã cung cấp đủ thông tin có ý nghĩa để mô hình tuyến tính hoạt động hiệu quả. Độ chênh lệch giữa Train F1 (0,8809) và CV F1 (0,8807)

gần như bằng 0, và kết quả trên test vẫn duy trì ở mức cao, chứng tỏ mô hình không bị overfitting.

Decision Tree có F1-Score = 0,8424, ROC-AUC = 0,9026, Accuracy = 0,9563. Điểm F1 thấp hơn đáng kể so với các mô hình khác (chênh lệch khoảng 0,1 so với Random Forest), cho thấy cây quyết định đơn lẻ, dù được giới hạn độ sâu và số mẫu tối thiểu, vẫn chưa thể học được toàn bộ cấu trúc phức tạp của dữ liệu. Điểm ROC-AUC 0,9026 vẫn ở mức tốt, nhưng F1 bị kéo xuống do sự mất cân bằng giữa Precision và Recall. Kiểm tra ma trận nhầm lẫn (sẽ phân tích ở mục tiếp theo) cho thấy Decision Tree có xu hướng dự đoán sai nhiều hơn ở lớp thiểu số (thua lỗ), dẫn đến Recall thấp hơn. Dù vậy, mô hình này vẫn giữ được lợi thế về tính minh bạch, phù hợp cho các tình huống cần giải thích rõ ràng từng quyết định.

So sánh tổng quan, có thể thấy Gradient Boosting là mô hình tối ưu nhất dựa trên các chỉ số F1-Score và ROC-AUC. Random Forest theo sát với hiệu suất rất gần, đồng thời có độ ổn định cao hơn nhờ cơ chế bagging. Logistic Regression tuy có hiệu suất thấp hơn, nhưng vẫn là một lựa chọn đáng tin cậy với ưu điểm diễn giải và tốc độ huấn luyện. Decision Tree, mặc dù kém hơn về mặt hiệu suất, vẫn có giá trị trong bối cảnh yêu cầu tính minh bạch tuyệt đối.

Như vậy, từ kết quả đánh giá trên tập kiểm tra, nghiên cứu xác định **Gradient Boosting** là mô hình dự báo lợi nhuận tốt nhất cho bộ dữ liệu Sample Superstore. Mô hình này sẽ được sử dụng để xây dựng hệ thống hỗ trợ ra quyết định cho siêu thị, với khả năng dự đoán chính xác và ổn định, giúp nhận diện kịp thời các giao dịch có nguy cơ thua lỗ, từ đó đề xuất điều chỉnh chiết khấu, phương thức vận chuyển hoặc cơ cấu sản phẩm phù hợp.

5.2.2. Ma trận nhầm lẫn

Ma trận nhầm lẫn cung cấp cái nhìn chi tiết về khả năng phân loại của từng mô hình thông qua bốn thành phần cơ bản:

- True Positive (TP) là số giao dịch có lợi nhuận được dự đoán đúng.
- False Positive (FP) là số giao dịch thua lỗ bị dự đoán nhầm thành có lợi nhuận.
- True Negative (TN) là số giao dịch thua lỗ được dự đoán đúng.
- False Negative (FN) là số giao dịch có lợi nhuận bị dự đoán nhầm thành thua lỗ.

Trong bối cảnh kinh doanh, việc phân tích từng loại sai lầm có ý nghĩa quan trọng: FN phản ánh các cơ hội sinh lời bị bỏ lỡ, trong khi FP phản ánh các cảnh báo rủi ro không chính xác.

```
# Phân tích Ma trận nhầm lẫn
print("\n--- PHÂN TÍCH MA TRẬN NHẦM LÃN (TP, FP, TN, FN) ---")
for name, model in models.items():
    cm = confusion_matrix(y_test, model.predict(X_test))
```

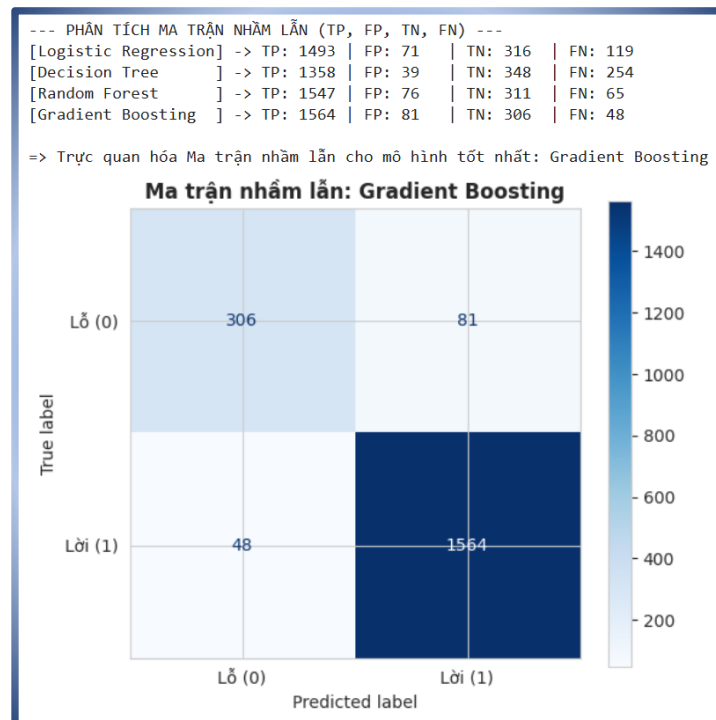
```

tn, fp, fn, tp = cm.ravel()
print(f"[{name:<19}] -> TP: {tp:<4} | FP: {fp:<4} | TN: {tn:<4} | FN: {fn:<4}")

# Vẽ Ma trận nhầm lẫn cho mô hình tối ưu nhất
champion_name = df_results.iloc[0]['Model']
champion_model = models[champion_name]

print(f"\n=> Trực quan hóa Ma trận nhầm lẫn cho mô hình tốt nhất: {champion_name}")
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_estimator(champion_model, X_test, y_test,
cmap='Blues',
display_labels=['Lỗ (0)', 'Lời (1)'], ax=ax)
plt.title(f'Ma trận nhầm lẫn: {champion_name}', fontsize=12,
fontweight='bold')
plt.show()

```



Hình 5. 4. Phân tích ma trận nhầm lẫn cho mô hình

Logistic Regression đạt TP = 1.493, FP = 71, TN = 316, FN = 119. Mô hình này dự đoán đúng 1.493 giao dịch có lợi nhuận và 316 giao dịch thua lỗ. Số lượng dự đoán sai gồm 71 trường hợp cảnh báo nhầm (dự đoán có lợi nhuận nhưng thực tế thua lỗ) và 119 trường hợp bỏ sót cơ hội (dự đoán thua lỗ nhưng thực tế có lợi nhuận). Tỷ lệ FP thấp hơn FN thể hiện mô hình có xu hướng phòng ngừa rủi ro nhẹ, nhưng vẫn còn 119 giao dịch có lợi nhuận bị bỏ lỡ.

Decision Tree cho kết quả TP = 1.358, FP = 39, TN = 348, FN = 254. Điểm đáng chú ý là mô hình này có số lượng FP rất thấp, chỉ 39 trường hợp cảnh báo nhầm. Tuy nhiên,

bù lại số FN rất cao lên đến 254, nghĩa là có 254 giao dịch thực tế có lợi nhuận bị dự đoán là thua lỗ, cho thấy là cây quyết định đơn lẻ có xu hướng bỏ sót nhiều cơ hội sinh lời do tính chất phân nhánh hạn chế của nó.

Random Forest cải thiện đáng kể với $TP = 1.547$, $FP = 76$, $TN = 311$, $FN = 65$. So với Logistic Regression, số lượng FN giảm mạnh từ 119 xuống còn 65, trong khi TP tăng lên 1.547. FP tăng nhẹ từ 71 lên 76, nhưng sự cải thiện về mặt tổng thể rất rõ ràng. Mô hình này đạt được sự cân bằng tốt giữa việc phát hiện đúng các giao dịch có lợi nhuận và nhận diện các giao dịch thua lỗ.

Gradient Boosting là mô hình tối ưu, cho kết quả $TP = 1.564$, $FP = 81$, $TN = 306$, $FN = 48$. Đây là mô hình có TP cao nhất, đạt 1.564 trên tổng số 1.613 giao dịch có lợi nhuận thực tế, tương ứng tỷ lệ phát hiện 97,0%. Đặc biệt, FN chỉ còn 48, thấp nhất trong bốn mô hình, nghĩa là chỉ có 48 giao dịch có lợi nhuận bị bỏ lỡ. Về lớp thua lỗ, mô hình phát hiện đúng 306 trên tổng số 387 giao dịch thua lỗ thực tế, đạt tỷ lệ 79,1%, với FP là 81. Mặc dù số lượng FP cao nhất, nhưng xét về tổng thể, Gradient Boosting vẫn là lựa chọn tối ưu vì giảm thiểu tối đa FN (những giao dịch có lợi nhuận bị đánh giá sai là thua lỗ).

Trong thực tế kinh doanh, việc bỏ lỡ cơ hội sinh lời (FN) thường gây thiệt hại lớn hơn việc cảnh báo nhầm một giao dịch thua lỗ (FP). Bởi lẽ cảnh báo nhầm có thể được kiểm tra lại bằng các biện pháp thủ công trước khi thực hiện giao dịch, trong khi bỏ lỡ cơ hội thì không thể phục hồi. Với FN chỉ còn 48, Gradient Boosting giúp siêu thị hạn chế tối đa rủi ro bỏ sót các giao dịch có tiềm năng sinh lời, đồng thời vẫn duy trì khả năng cảnh báo các giao dịch thua lỗ ở mức chấp nhận được. Đây là minh chứng cụ thể cho hiệu suất vượt trội của mô hình này và là cơ sở để lựa chọn Gradient Boosting làm công cụ dự báo chính thức.

5.2.3. Đường cong ROC

Đường cong ROC là một công cụ trực quan quan trọng để đánh giá khả năng phân biệt giữa hai lớp của mô hình phân loại. Đường cong được xây dựng bằng cách biểu diễn mối quan hệ giữa tỷ lệ dương tính thật (True Positive Rate - TPR, hay còn gọi là recall hoặc độ nhạy) theo tỷ lệ dương tính giả (False Positive Rate - FPR) tại các ngưỡng quyết định khác nhau.

Trong đó, TPR phản ánh khả năng phát hiện đúng các giao dịch có lợi nhuận, trong khi FPR là tỷ lệ các giao dịch thua lỗ bị dự đoán nhầm thành có lợi nhuận. Một mô hình lý tưởng sẽ có đường cong nằm sát góc trên bên trái của đồ thị, tức là đạt TPR cao đồng thời FPR thấp.

Diện tích dưới đường cong (AUC) là chỉ số tổng hợp, có giá trị từ 0 đến 1, không phụ thuộc vào ngưỡng quyết định. AUC càng lớn cho thấy mô hình càng có khả năng xếp hạng các giao dịch có lợi nhuận cao hơn các giao dịch thua lỗ, bất kể ngưỡng phân loại được chọn.

```

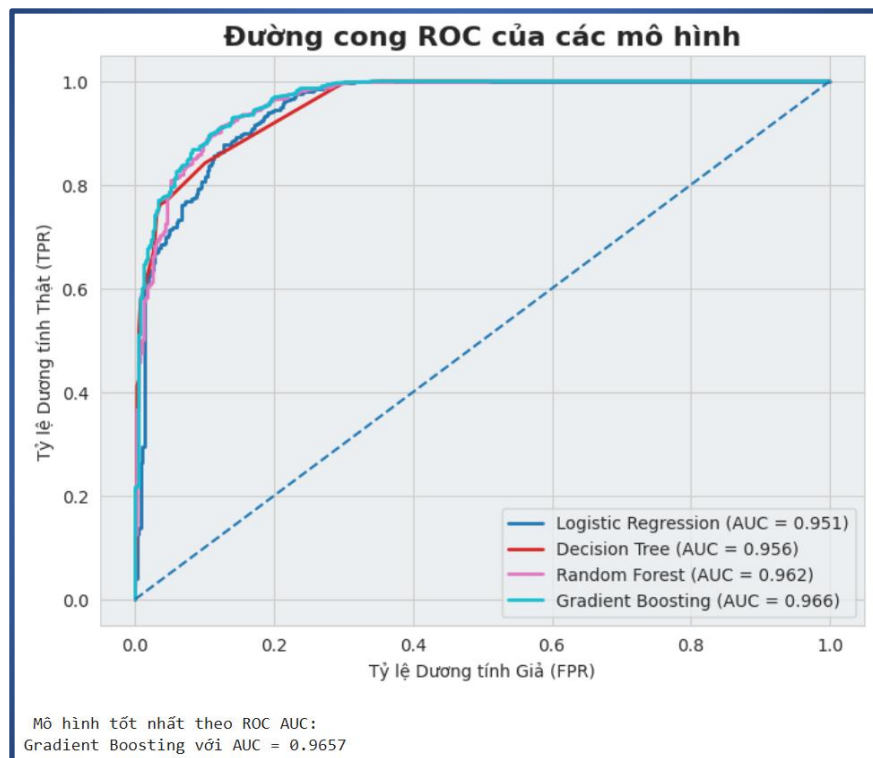
# Đường cong ROC (Khả năng phân loại)
plt.figure(figsize=(8, 6))
colors = plt.cm.tab10(np.linspace(0, 1, len(models)))
best_auc = -1
best_name = ""
for (name, model), color in zip(models.items(), colors):
    y_prob = model.predict_proba(X_test)[: , 1]
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    roc_auc = auc(fpr, tpr)

    plt.plot(fpr, tpr, lw=2, color=color,
             label=f'{name} (AUC = {roc_auc:.3f})')

    if roc_auc > best_auc:
        best_auc = roc_auc
        best_name = name

plt.plot([0, 1], [0, 1], linestyle='--')
plt.xlabel('Tỷ lệ Dương tính Giả (FPR)')
plt.ylabel('Tỷ lệ Dương tính Thật (TPR)')
plt.title('Đường cong ROC của các mô hình', fontsize=16,
          fontweight='bold')
plt.legend(loc="lower right")
plt.show()
print("\n Mô hình tốt nhất theo ROC AUC:")
print(f"{best_name} với AUC = {best_auc:.4f}")

```



Hình 5. 5. Đường cong ROC và giá trị AUC của các mô hình

Kết quả đường cong ROC của bốn mô hình cho thấy tất cả đều đạt AUC trên 0,95, phản ánh chất lượng phân loại rất tốt và khẳng định dữ liệu sau quá trình tiền xử lý và lựa chọn đặc trưng đã tạo điều kiện thuận lợi để các thuật toán học được sự khác biệt rõ ràng giữa hai lớp.

Logistic Regression có $AUC = 0,951$. Đây là một kết quả ấn tượng đối với mô hình tuyến tính, chứng tỏ các đặc trưng định lượng (Sales, Quantity, Ship_Duration) và các biến giả từ quá trình mã hóa đã cung cấp đủ thông tin phân biệt có ý nghĩa. Tuy nhiên, do hạn chế trong việc nắm bắt các tương tác phi tuyến, AUC của Logistic Regression thấp hơn so với các mô hình phi tuyến.

Decision Tree đạt $AUC = 0,956$, cao hơn 0,005 so với Logistic Regression. Sự cải thiện này phản ánh lợi thế của cấu trúc cây quyết định trong việc mô hình hóa các mối quan hệ phi tuyến, đặc biệt là các tương tác giữa chiết khấu và danh mục sản phẩm đã được phát hiện trong giai đoạn EDA. Mặc dù Decision Tree có F1-Score thấp nhất do hạn chế về recall, nhưng khả năng phân biệt tổng thể được đo bằng AUC vẫn ở mức tốt, chỉ thấp hơn Random Forest và Gradient Boosting không nhiều.

Random Forest đạt $AUC = 0,962$. Mức tăng 0,006 so với Decision Tree phản ánh hiệu quả của cơ chế bagging: việc xây dựng nhiều cây độc lập trên các mẫu bootstrap và lấy bình chọn đa số giúp giảm phương sai tổng thể, tạo ra các dự đoán ổn định hơn và cải thiện khả năng phân biệt. Với độ sâu tối đa 6 và 100 cây, mỗi cây thành phần vẫn giữ được khả năng khai thác tương tác phi tuyến, trong khi việc kết hợp chúng triệt tiêu các nhiễu ngẫu nhiên.

Gradient Boosting đạt $AUC = 0,966$ là giá trị cao nhất trong bốn mô hình. Đây là kết quả phù hợp với lý thuyết: thuật toán boosting xây dựng tuần tự các cây yếu ($max_depth=3$), mỗi cây tập trung sửa sai cho tổ hợp các cây trước đó, giúp tối ưu trực tiếp hàm mất mát và đạt được khả năng phân biệt vượt trội. Với tốc độ học nhỏ ($learning_rate=0.1$), quá trình tối ưu diễn ra từ từ, tránh overfitting ngay từ những vòng lặp đầu. Sự chênh lệch AUC giữa Gradient Boosting và Random Forest là 0,004, không lớn nhưng vẫn đủ để khẳng định ưu thế của boosting trong việc khai thác cấu trúc dữ liệu phức tạp.

Điều đáng chú ý là sự tăng dần của AUC qua các nhóm mô hình phản ánh đúng bản chất của dữ liệu. Logistic Regression (tuyến tính) đạt AUC 0,951; Decision Tree (phi tuyến đơn lẻ) đạt 0,956; Random Forest (bagging) đạt 0,962; Gradient Boosting (boosting) đạt 0,966. Sự phân hóa này cho thấy các mối quan hệ phi tuyến và tương tác giữa các đặc trưng đóng vai trò quan trọng và các kỹ thuật ensemble có khả năng khai thác những cấu trúc này hiệu quả hơn. Mức chênh lệch 0,015 giữa Gradient Boosting và Logistic Regression tuy không quá lớn nhưng đủ để khẳng định rằng việc sử dụng mô hình phức tạp hơn mang lại lợi ích thực sự, đặc biệt khi yêu cầu độ chính xác cao trong dự báo.

Từ góc độ thống kê, $AUC = 0,966$ của Gradient Boosting có thể được hiểu như sau: nếu lấy ngẫu nhiên một cặp gồm một giao dịch có lợi nhuận và một giao dịch thua lỗ, xác suất để mô hình xếp hạng đúng (tức là gán xác suất có lợi nhuận cao hơn cho giao dịch có lợi nhuận) là 96,6%.

Kết hợp với phân tích ma trận nhầm lẫn và các chỉ số F1-Score, có thể khẳng định Gradient Boosting là mô hình tối ưu nhất cho bài toán dự báo lợi nhuận của siêu thị, phù hợp để tích hợp vào hệ thống hỗ trợ ra quyết định với khả năng cảnh báo chính xác các giao dịch có nguy cơ thua lỗ và hạn chế tối đa việc bỏ lỡ cơ hội sinh lời.

PHẦN VI. KẾT LUẬN CHUNG

6.1. Những phát hiện chính từ quá trình phân tích dữ liệu và mô hình

Quá trình phân tích và xây dựng mô hình trên bộ dữ liệu Sample Superstore đã mang lại nhiều kết quả có ý nghĩa, không chỉ về mặt kỹ thuật mà còn cung cấp các hàm ý quan trọng cho quản trị kinh doanh.

Về dữ liệu và phân tích khám phá (EDA)

Bộ dữ liệu gồm 9.994 giao dịch với 21 thuộc tính, không tồn tại giá trị thiếu hay trùng lặp, phản ánh chất lượng dữ liệu đầu vào tốt. Các biến định lượng như *Sales* và *Profit* có phân phối lệch phải rõ rệt, đồng thời xuất hiện nhiều giá trị ngoại lai, đặc biệt là các giao dịch có doanh số rất cao hoặc lợi nhuận âm sâu. Việc xử lý ngoại lai bằng phương pháp Winsorization tại các ngưỡng phân vị 1% và 99% giúp giảm ảnh hưởng của các giá trị cực đoan trong khi vẫn bảo toàn đặc trưng của dữ liệu.

Phân tích tương quan cho thấy *Sales* có mối quan hệ dương mức trung bình với *Profit*, tuy nhiên không phải mọi giao dịch có doanh số cao đều mang lại lợi nhuận. Đáng chú ý, *Discount* có tương quan âm với lợi nhuận nhưng gần như không liên hệ tuyến tính với doanh số, cho thấy hiệu quả của chính sách chiết khấu phụ thuộc vào các yếu tố khác. Phân tích sâu hơn theo từng danh mục sản phẩm cho thấy tồn tại mối quan hệ phi tuyến và tương tác rõ rệt: danh mục *Furniture* nhạy cảm với chiết khấu cao, trong khi *Technology* có khả năng duy trì lợi nhuận ngay cả khi mức chiết khấu lớn. Điều này khẳng định vai trò của các yếu tố tương tác trong dữ liệu.

Về xử lý dữ liệu và xây dựng đặc trưng

Nghiên cứu đã tạo thêm biến *Ship_Duration* từ chênh lệch thời gian giao hàng, đồng thời phân nhóm chiết khấu thành các mức có thứ tự dựa trên kết quả EDA. Biến mục tiêu *Is_Profit* được xây dựng dưới dạng nhị phân, phù hợp với bài toán phân loại. Sau quá trình mã hóa và chuẩn hóa, dữ liệu được xử lý mất cân bằng bằng kỹ thuật SMOTE, giúp cân bằng hoàn toàn hai lớp trên tập huấn luyện và tạo điều kiện cho mô hình học đầy đủ đặc trưng của cả hai nhóm.

Về mô hình hóa và đánh giá

Bốn thuật toán được sử dụng, gồm: Logistic Regression, Decision Tree, Random Forest và Gradient Boosting, đại diện cho các cách tiếp cận từ tuyến tính đến phi tuyến và từ mô hình đơn lẻ đến ensemble. Kết quả kiểm chứng chéo cho thấy Gradient Boosting đạt hiệu suất cao nhất về F1 Score, trong khi Random Forest thể hiện sự ổn định tốt hơn. Logistic Regression có độ ổn định cao nhưng hạn chế trong việc nắm bắt quan hệ phi tuyến, còn Decision Tree có hiệu suất thấp hơn do tính đơn giản của cấu trúc.

Đánh giá trên tập kiểm tra độc lập tiếp tục khẳng định Gradient Boosting là mô hình tối ưu với các chỉ số vượt trội. Phân tích ma trận nhầm lẫn cho thấy mô hình này có số lượng *False Negative* thấp nhất, giúp giảm thiểu việc bỏ lỡ các giao dịch có lợi nhuận. Đồng thời, kết quả đường cong ROC với AUC cao phản ánh khả năng phân biệt hai lớp rất tốt. Sự gia tăng hiệu suất từ mô hình tuyến tính đến các mô hình ensemble cũng cho thấy dữ liệu có cấu trúc phi tuyến và tương tác phức tạp.

Từ góc độ quản trị, kết quả nghiên cứu gợi ý rằng việc kiểm soát chiết khấu theo từng danh mục sản phẩm là yếu tố then chốt để tối ưu lợi nhuận. Đặc biệt, cần hạn chế chiết khấu cao đối với nhóm *Furniture*, trong khi có thể linh hoạt hơn với nhóm *Technology*. Ngoài ra, việc ứng dụng mô hình dự báo giúp nhận diện sớm các giao dịch có nguy cơ thua lỗ, từ đó hỗ trợ ra quyết định kịp thời.

6.2. Ưu điểm và hạn chế của nghiên cứu

Nghiên cứu đạt được nhiều ưu điểm nổi bật, không chỉ ở kết quả mà còn ở quy trình tiếp cận và tính thực tiễn của các phát hiện.

Quy trình tiền xử lý dữ liệu được thực hiện một cách bài bản và có cơ sở khoa học. Cụ thể, các giá trị ngoại lai được xử lý bằng phương pháp Winsorization dựa trên phân vị, dữ liệu được chuẩn hóa theo phương pháp Min-Max, và tình trạng mất cân bằng lớp được khắc phục bằng kỹ thuật SMOTE. Toàn bộ quá trình đảm bảo nguyên tắc không làm rò rỉ thông tin từ tập kiểm tra vào tập huấn luyện, qua đó nâng cao tính khách quan của kết quả đánh giá.

Các đặc trưng được xây dựng dựa trên kết quả của phân tích khám phá dữ liệu (EDA), tạo ra sự liên kết chặt chẽ giữa hiểu biết thống kê về bộ dữ liệu và quá trình mô hình hóa, đồng thời giúp các biến đưa vào mô hình có ý nghĩa thực tiễn rõ ràng (ví dụ: phân nhóm chiết khấu theo ngưỡng 20% và 50% dựa trên quan sát về sự thay đổi hành vi lợi nhuận).

Chiến lược lựa chọn mô hình thể hiện sự so sánh đa chiều hợp lý. Logistic Regression đóng vai trò mô hình nền, Decision Tree cung cấp góc nhìn phi tuyến đơn giản, trong khi Random Forest và Gradient Boosting đại diện cho hai phương pháp ensemble quan trọng là bagging và boosting. Sự khác biệt về hiệu suất giữa các mô hình cho thấy rõ vai trò của các kỹ thuật ensemble trong việc khai thác cấu trúc dữ liệu phức tạp.

Nghiên cứu sử dụng hệ thống chỉ số đánh giá toàn diện, bao gồm Accuracy, F1 Score, ROC AUC và ma trận nhầm lẫn. Trong bối cảnh dữ liệu mất cân bằng, F1 Score và ROC AUC được ưu tiên, đồng thời ma trận nhầm lẫn được phân tích chi tiết nhằm làm rõ bản chất các sai lầm của mô hình, đặc biệt là các trường hợp *False Negative* có ý nghĩa lớn trong thực tiễn.

Bên cạnh những ưu điểm, nghiên cứu vẫn tồn tại một số hạn chế cần được xem xét. Dữ liệu chỉ giới hạn trong phạm vi Hoa Kỳ với quy mô 9.994 giao dịch, tuy đủ để huấn luyện nhưng chưa thể khẳng định tính tổng quát cho các thị trường khác hoặc bối cảnh kinh doanh biến động mạnh (như sau đại dịch) cần được kiểm định thêm.

Việc lựa chọn đặc trưng dựa trên hiểu biết kinh doanh và EDA là hợp lý, song có thể chưa khai thác hết các tương tác tiềm ẩn giữa các biến, chẳng hạn như tương tác giữa phương thức vận chuyển và khu vực địa lý hoặc ảnh hưởng của thời gian (mùa vụ) đến hành vi mua hàng.

Ngoài ra, việc tinh chỉnh siêu tham số chưa được thực hiện một cách toàn diện thông qua các phương pháp tìm kiếm có hệ thống như Grid Search hoặc Random Search. Các mô hình hiện tại sử dụng bộ tham số được lựa chọn dựa trên kinh nghiệm và tài liệu tham khảo, kết hợp với kiểm chứng chéo để đảm bảo tính ổn định. Mặc dù kết quả đạt được đã ở mức cao, việc tối ưu hóa siêu tham số vẫn có tiềm năng cải thiện thêm hiệu năng mô hình, đặc biệt trong bối cảnh dữ liệu tồn tại nhiều quan hệ phi tuyến và tương tác phức tạp.

6.3. Đề xuất hướng phát triển

Để nâng cao giá trị và khả năng ứng dụng của nghiên cứu, cần tập trung vào ba hướng phát triển chính.

Có thể mở rộng dữ liệu và đặc trưng đầu vào vì nghiên cứu hiện tại chỉ sử dụng các đặc trưng có sẵn trong bộ dữ liệu. Cần bổ sung thêm dữ liệu về hành vi khách hàng (tần suất mua, giá trị vòng đời), yếu tố thị trường (chương trình khuyến mãi, mùa vụ, biến động kinh tế) và dữ liệu đối thủ cạnh tranh. Các thông tin này sẽ giúp mô hình dự báo nhạy bén hơn trước những thay đổi của thị trường và hành vi người tiêu dùng.

Phát triển hệ thống hỗ trợ ra quyết định thông minh, tích hợp học tăng cường để tự động điều chỉnh chính sách chiết khấu, phương thức vận chuyển và gợi ý sản phẩm theo thời gian thực. Hệ thống không chỉ dự báo mà còn chủ động đề xuất hành động tối ưu nhằm gia tăng lợi nhuận, biến nghiên cứu thành một công cụ quản trị chiến lược thay vì chỉ dừng ở mức cảnh báo.

Về mặt ứng dụng thì nghiên cứu có thể phát triển thành một hệ thống cảnh báo sớm và đề xuất điều chỉnh tích hợp trong quy trình bán hàng. Thay vì chỉ dừng lại ở dự báo, hệ thống có thể được mở rộng để tự động đưa ra các khuyến nghị như mức chiết khấu tối ưu cho từng danh mục sản phẩm, phương thức vận chuyển phù hợp với từng khu vực hoặc các gói sản phẩm kết hợp nhằm tối đa hóa xác suất có lợi nhuận. Việc tích hợp với hệ thống quản lý bán hàng hiện hữu sẽ giúp nhân viên có công cụ hỗ trợ kịp thời, từ đó nâng cao hiệu quả vận hành.

TÀI LIỆU THAM KHẢO

- [1] Abubakar, S., et al. (2024). Predicting retail sales using machine learning techniques: A comparative study. *ResearchGate*.
<https://www.researchgate.net/publication/382613584>
- [2] Bastos, J. A., et al. (2022). Imbalanced data preprocessing techniques for machine learning: A systematic mapping study. *Applied Intelligence*, 53, 11403-11430.
- [3] Batista, G. E., Prati, R. C., & Monard, M. C. (2004). A study of the behavior of several methods for balancing machine learning training data. *ACM SIGKDD Explorations Newsletter*, 6(1), 20-29. <https://dl.acm.org/doi/10.1145/1007730.1007735>
- [4] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
<https://link.springer.com/journal/10994>
- [5] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321-357. <https://www.jair.org/index.php/jair/article/view/10302>
- [6] Chen, H., Chiang, R. H. L., & Storey, V. C. (2012). Business intelligence and analytics: From big data to big impact. *MIS Quarterly*, 36(4), 1165-1188.
- [7] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794).
<https://dl.acm.org/doi/10.1145/2939672.2939785>
- [8] Cleveland, W. S. (1993). *Visualizing data*. Hobart Press.
- [9] Fathalla, A., et al. (2024). Enhancing retail sales forecasting with optimized machine learning models. *arXiv preprint*. <https://arxiv.org/abs/2410.13773>
- [10] Fernández, A., Garcia, S., Herrera, F., & Chawla, N. V. (2018). SMOTE for learning from imbalanced data: Progress and challenges, marking the 15-year anniversary. *Journal of Artificial Intelligence Research*, 61, 863-905.
<https://jair.org/index.php/jair/article/view/11192>
- [11] Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
- [12] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer.
- [13] He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263-1284.
<https://ieeexplore.ieee.org/document/5128907>

- [14] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: With applications in R* (2nd ed.). Springer.
- [15] Kuhn, M. (2008). Building predictive models in R using the caret package. *Journal of Statistical Software*, 28(5), 1-26.
<https://www.jstatsoft.org/article/view/v028i05>
- [16] Kuhn, M., & Johnson, K. (2013). *Applied predictive modeling*. Springer.
- [17] Kumar, A., et al. (2024). Predicting Big Mart sales with machine learning. *Engineering Proceedings*, 107(1), 95.
- [18] Molnar, C. (2022). *Interpretable machine learning* (2nd ed.). Lulu.com.
- [19] Provost, F., & Fawcett, T. (2013). *Data science for business*. O'Reilly Media.
- [20] Saha, A., & Gupta, S. (2025). Machine learning framework for retail sales forecasting. *International Journal of Computational and Experimental Science and Engineering*, 11(1). <https://ijcesen.com/index.php/ijcesen/article/view/3993>
- [21] Shmueli, G., Bruce, P. C., Gedeck, P., & Patel, N. R. (2020). *Data mining for business analytics: Concepts, techniques, and applications in Python*. Wiley.
- [22] Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.
- [23] Wang, J., & Li, X. (2025). Study on predicting e-commerce financial risk from imbalanced data-An integrated model based on CSA-optimized SMOTE and Random Forest. *E-Commerce Letters*, 14(12), 6542-6553.
<https://www.researchgate.net/publication/399889656>
- [24] Wickham, H., & Grolemund, G. (2017). *R for data science: Import, tidy, transform, visualize, and model data*. O'Reilly Media.
- [25] Wilke, C. O. (2019). *Fundamentals of data visualization*. O'Reilly Media.